

Министерство науки и высшего образования Российской Федерации

Томский государственный университет
систем управления и радиоэлектроники

Кафедра компьютерных систем в управлении и проектировании

БЕЗОПАСНОСТЬ СИСТЕМ БАЗ ДАННЫХ

Лабораторный практикум

для студентов вузов, обучающихся по техническим направлениям подготовки всех форм
обучения

Издание 2-е, дополненное и исправленное

Томск
2024

УДК 004.056:004.6 (076.5)
ББК 16.84:16.3я7-5
Б400

Рецензенты:

Ходашинский И.А., профессор кафедры компьютерных систем в управлении
и проектировании ТУСУР, д. техн. наук
Массель Л.В., главный научный сотрудник ИСЭМ СО РАН, заведующая отделом Си-
стем искусственного интеллекта в энергетике, д. техн. наук

Авторы:

К.С. Сарин, А.С. Киселёв, Н.П. Корышев, М.Б. Бардамова, И.А. Тахтеев

Безопасность систем баз данных: лабораторный практикум / К.С. Сарин [и др.].
–Томск: Томск. гос ун-т систем упр. и радиоэлектроники, 2024. – 81 с.

Лабораторный практикум содержит описания лабораторных работ по дисциплине
«Базы данных» для студентов вузов, обучающихся по техническим направлениям подго-
товки всех форм обучения. Представлены задания, методические указания по выполне-
нию, требования по представлению отчётности, вопросы для самоконтроля.

УДК 004.056:004.6 (076.5)
ББК 16.84:16.3я7-5

© К.С. Сарин, А.С. Киселев, Н.П. Корышев,
М.Б. Бардамова, И.А. Тахтеев, 2024
© Томск. гос ун-т систем упр. и радиоэлек-
троники 2024

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
ЛАБОРАТОРНАЯ РАБОТА № 1 Шифрование и маскирование	5
Теоретический материал.....	5
Ход работы.....	10
Контрольные вопросы.....	20
Задание.....	21
ЛАБОРАТОРНАЯ РАБОТА № 2 Разделение прав доступа	24
Теоретический материал.....	24
Ход работы.....	28
Контрольные вопросы.....	31
Задание.....	32
ЛАБОРАТОРНАЯ РАБОТА № 3 Защита на уровне строк	34
Теоретический материал.....	34
Ход работы.....	37
Контрольные вопросы.....	43
Задание.....	43
ЛАБОРАТОРНАЯ РАБОТА № 4 Протокол уровня защищённых сокетов (SSL protocol)	45
Теоретический материал.....	45
Ход работы.....	45
Контрольные вопросы.....	51
Задание.....	51
ЛАБОРАТОРНАЯ РАБОТА № 5 Оценка уязвимостей и классификация данных ..	52
Теоретический материал.....	52
Ход работы.....	53
Контрольные вопросы.....	57
Задание.....	57
ЛАБОРАТОРНАЯ РАБОТА №6 Защита от SQL-инъекций	58
Теоретический материал.....	58
Ход работы.....	61
Контрольные вопросы.....	76
Задание.....	76
Заключение	80
Литература	81

ВВЕДЕНИЕ

Лабораторный практикум подготовлен с целью обучения студентов работе с базовыми механизмами администрирования безопасности баз данных. Выполнив лабораторные работы, студенты получают навыки защиты информации в системах управления базами данных и приобретут профессиональные компетенции, связанные с освоением методики использования программных средств для решения практических задач и установкой программного и аппаратного обеспечения для информационных и автоматизированных систем.

Для выполнения лабораторных студентами должны быть освоены основы разработки баз данных, включая проектирование и реализацию реляционных моделей данных, а также основы языка SQL.

Лабораторные работы выполняются на виртуальных машинах под управлением операционной системы Windows Server 2016 с предустановленной системой управления базами данных (СУБД) Microsoft SQL Server и тренировочными базами данных. Ссылки для скачивания образов дисков этих машин должны быть предоставлены преподавателем и/или приведены в электронном курсе.

Далее приводится инструкция по развёртыванию виртуальных машин.

а) Загрузите архив с необходимым образом диска виртуальной машины для выполнения работ по предоставленной ссылке. Затем распакуйте его: в папке должны оказаться или два файла с расширением .ovf (файл с настройками машины) и .vmdk (файл образа жёсткого диска), либо файл с расширением .ova (укомплектованный файл виртуальной машины в сжатом виде).

б) Для запуска виртуальных машин установите одну программу виртуализации: VMWare Workstation либо Oracle VM VirtualBox. Для выполнения работ достаточно использовать бесплатные версии этих приложений.

в) Импортируйте загруженную виртуальную машину (файл .ovf или .ova) в используемую систему виртуализации.

1) В VMWare Workstation:

- выберите «Open Virtual Machine» и укажите импортируемый файл;
- укажите название и место хранения виртуальной машины и нажмите «Import»;
- дождитесь конца процесса импорта;
- проверьте настройки виртуальной машины (найдите кнопку «Settings» в контекстном меню машины); во вкладке «Options» выберите в левом меню «General» и при необходимости скорректируйте выбор гостевой операционной системы: должна быть выбрана система «Microsoft Windows» версии «Windows Server 2016».

2) В Oracle VM VirtualBox:

- в меню «Файл» выберите «Импорт конфигураций»;
- выберите загруженный файл конфигурации формата .ovf и нажмите «Далее»;
- проверьте настройки виртуальной машины в появившемся списке: в строке «Тип гостевой ОС» выберите «Windows 2016 (64-bit)»; при необходимости можете поменять имя виртуальной машины; после проверки настроек нажмите «Готово»;
- дождитесь конца процесса импорта.

ЛАБОРАТОРНАЯ РАБОТА № 1

Шифрование и маскирование

Цель работы – изучение основ шифрования и маскирования, их целей и задач, а также практическая реализация шифрования и маскирования в СУБД MS SQL Server.

Теоретический материал

Шифрование – такое преобразование данных, которое делает прочтение этих данных невозможным без обратного преобразования – расшифрования. Преобразования выполняются на основе некоторого алгоритма шифрования с использованием секрета – ключа шифрования. Используется шифрование для поддержания конфиденциальности данных, поскольку прочтение данных становится невозможным посторонним лицам.

Существуют два вида схем шифрования: симметричные (используется один ключ для шифрования и для расшифрования) и асимметричные (используется открытый ключ для расшифрования и закрытый ключ для шифрования). Симметричное шифрование чаще всего применяется для непосредственного шифрования рабочих данных (отличается более быстрым процессом преобразования). Асимметричное шифрование обычно полезно в задачах подтверждения авторства (передача сообщений, электронная подпись, сертификаты) и шифрования ключевой информации.

В рамках Microsoft (MS) SQL Server реализовано три вида шифрования: общая схема шифрования, работающее в рамках этой схемы прозрачное шифрование данных (Transparent Data Encryption, TDE) и постоянное шифрование (Always Encrypted, AE).

Целью общей схемы шифрования является предоставление разработчикам баз данных возможности зашифровать свою базу так, как это представляется удобным организации и/или совместимым с местным законодательством по шифрованию.

В рамках общей схемы предоставляются симметричное и асимметричное шифрование, а также сертификаты (можно как использовать внешний сертификат, так и создать собственный). Шифрование по общей схеме реализуется согласно приведённому колесу иерархии шифрования (рисунок 1.1). Шифровать по общей схеме шифрования можно конкретные столбцы данных.

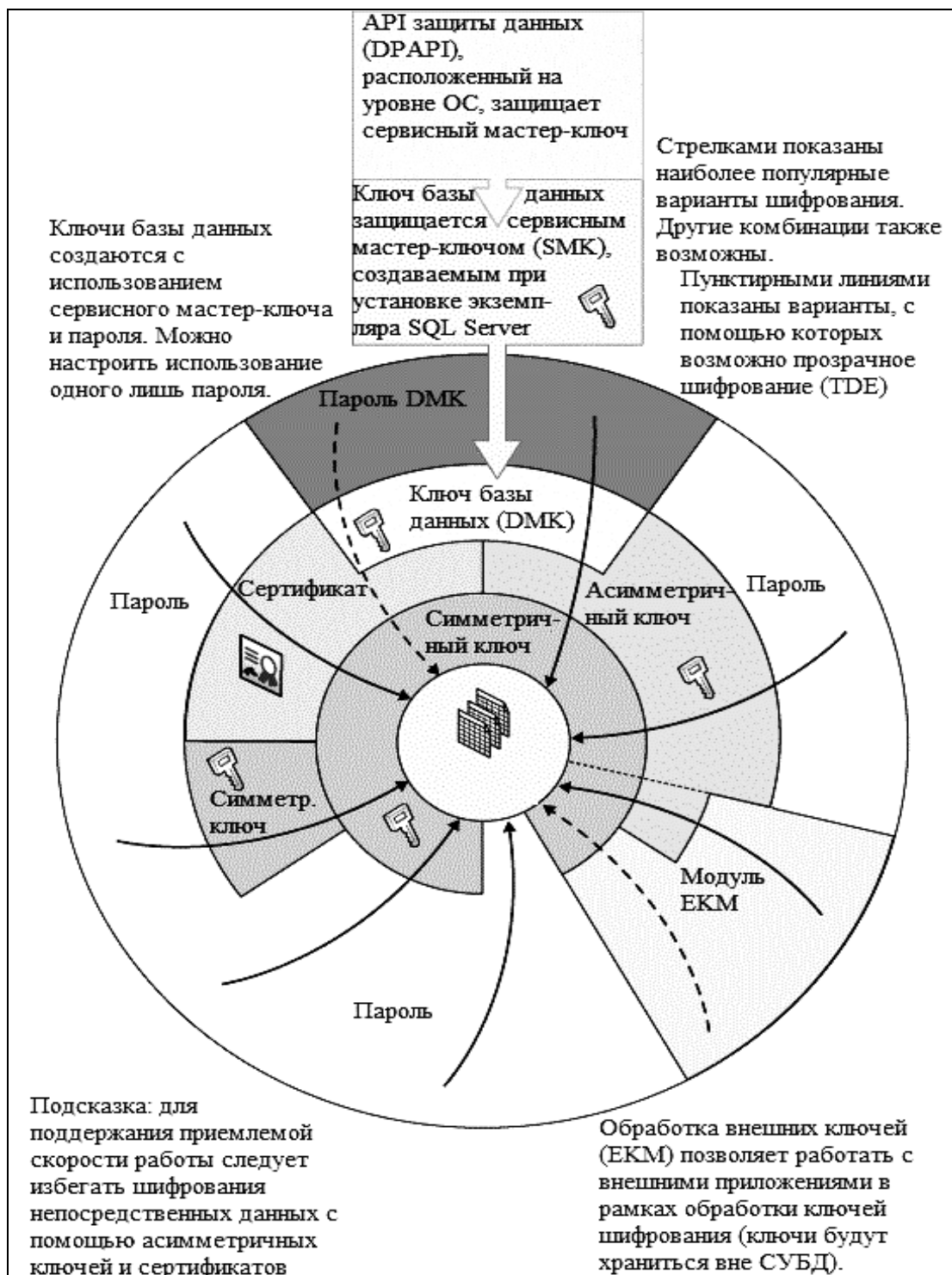


Рисунок 1.1 – Колесо иерархии шифрования (<https://learn.microsoft.com/>)

В рамках общей схемы шифрования существуют следующие виды механизмов шифрования.

- Transact-SQL: простейшее шифрование на основе парольной фразы с помощью 128-мибитного алгоритма Triple DES. Команды: ENCRYPTBYPASSPHRASE для шифрования и DECRYPTBYPASSPHRASE для расшифрования. Предназначен данный вид шифрования для преобразования строки данных (строка будет сохранена в зашифрованном виде).

- Сертификат: подписанное электронной подписью утверждение о принадлежности открытого ключа шифрования определённому лицу, устройству, сервису и т.д., у которого находится соответствующий закрытый ключ расшифрования. Сертифи-

каты содержат в себе публичный ключ субъекта, его идентификационную информацию, электронную подпись субъекта и имеют заданный срок действия. Сертификаты бывают подписанные доверенным сервисом и самоподписанные. В рамках SQL Server возможно использование внешних сертификатов и создание самоподписанных. Предназначен сертификат для создания доверенного соединения и используется, как правило, для получения необходимых секретных данных от доверенного сервера.

- Асимметричный ключ: пара открытый-закрытый ключи, используемая для зашифровывания секретной информации. Из-за медленной скорости выполнения асимметричных алгоритмов рекомендуется зашифровывать только ограниченный объём информации. Как правило, асимметричные ключи используются для шифрования другой секретной информации. В рамках SQL Server сертификат выполняет схожую с асимметричным ключом роль: они оба используют асимметричную схему шифрования и не имеют различий в надёжности алгоритма шифрования. Различие заключается в том, что к сертификату прилагается электронная подпись, на основании которой возможно создание доверенного соединения.

- Симметричный ключ: один ключ, используемый как для шифрования, так и для расшифрования. Предназначен для шифрования непосредственной информации. Алгоритмы симметричного шифрования достаточно быстры для обработки больших объёмов информации.

- Прозрачное шифрование данных: подробно описано далее.

- Обработка внешних ключей (Enterprise Key Management, EKM): схема, используемая для обработки внешних ключей (симметричных и асимметричных), предоставляемых внешними приложениями.

Прозрачное шифрование данных используется для зашифровывания всей базы данных наиболее удобным способом: для выполнения такого шифрования достаточно исполнить несколько запросов в Transact SQL. Данные в TDE шифруются симметрично, ключ шифрования защищается сертификатом либо асимметричным ключом, шифруется вся база данных вместе с журналами и мета-данными в реальном времени. Возможны два варианта зашифровывания с помощью TDE: с помощью мастер-ключа базы данных или с помощью внешнего ключа (оба варианта показаны на рис. 1.1 пунктирными линиями). Данные, как и в общей схеме, шифруются на носителе и расшифровываются при чтении. Обращаем внимание, что TDE поддерживается в Enterprise, Normal и Developer версиях MS SQL Server.

Важно отметить, что создаваемые в рамках SQL Server криптографические объекты (прежде всего это асимметричные ключи и сертификаты) предназначены для использования только в рамках самого SQL Server. Microsoft не рекомендует создавать эти криптографические данные для использования в других приложениях или операционной системой.

Постоянное шифрование предназначено для шифрования данных на ином уровне, нежели общая схема, и существует оно отдельно от общей схемы. В каком-то смысле постоянное шифрование является улучшением общей схемы, поскольку закрываются те недостатки, которые имели место в рамках общей схемы.

В рамках постоянного шифрования шифруются столбцы, данные зашифровываются вплоть до момента непосредственного чтения пользователем с особой привилегией (без таковой пользователь будет видеть только шифротекст) и установленным на клиенте приложением, поддерживающим Always Encrypted (например, SQL Server Management Studio, но могут быть и пользовательские приложения) и имеющим доступ к хранилищу, в котором расположен необходимый ключ. Такие ключи могут располагаться локально на клиентской машине (с помощью хранилища Windows Certificate Store) или удалённо на сервере (с помощью хранилища Azure Key Vault).

В рамках самой базы данных хранятся два вида ключей (оба зашифрованы): мастер-ключ столбца (Column Master Key) и ключ шифрования столбца (Column Encryption

Key). Мастер-ключ столбца используется для зашифровывания одного или нескольких ключей шифрования столбца, а ключ шифрования столбца используется для шифрования одного конкретного столбца.

Возможно шифрование определённое (одинаковые данные шифруются одинаковым шифротекстом) или случайное (одинаковые данные шифруются различным шифротекстом с помощью добавления случайного элемента); с определённым шифрованием возможны некоторые операции (структурирование, сортировка, точечный поиск, индексация), однако есть возможность вывести содержимое этих столбцов, если имеется достаточно внешней информации и/или вариантов содержимого столбца немного (например, да/нет, муж/жен и пр.).

Используется данный вид шифрования для засекречивания отдельных важных данных, которые следует раскрыть только весьма ограниченному кругу лиц (например, номер кредитной карты): такие данные не могут напрямую просматривать высоко привилегированные пользователи, а также к ним не удастся получить доступ злоумышленнику в том случае, если ему удастся взломать сервер базы данных, потому что ключевая информация, необходимая для расшифровывания, хранится не на нём (либо локально на клиенте, либо на специальном сервере).

На стороне сервера базы данных реализовывается постоянное шифрование с помощью графического интерфейса. На стороне клиента всё зависит от реализующей программы (в рамках SQL Server Management Studio необходимо в строке подключения указать параметр подключения `column encryption setting=enabled` и иметь доступ к хранилищу, содержащему необходимую ключевую информацию).

Маскирование данных – процесс преобразования конфиденциальных данных в форму, не позволяющую получить доступ к конфиденциальной части данных, но по-прежнему позволяющую работать с этими данными в иных, кроме ознакомления, целях. Примером таких целей могут быть сортировка данных, идентификация и аутентификация пользователя (например, по четырём первым цифрам номера кредитной карты), сбор статистической информации и пр. Маскирование применяется в тех случаях, когда необходимо раскрыть только часть данных, расположенных в рамках ячейки (например, первые четыре цифры кредитной карты).

Существуют два вида маскирования: статическое и динамическое. В SQL Server статическое маскирование планировалось Microsoft как полноценная возможность, однако разработка была отменена на уровне внедрения в предварительную версию SQL Server Management Studio (SSMS) - интегрированной среды для управления любой инфраструктурой СУБД Microsoft SQL Server. Тем не менее, концепция статического маскирования существует и возможна в рамках других СУБД, кроме того, Microsoft не исключают возможность возвращения к этой концепции в будущем. Об статическом маскировании у Microsoft осталась справочная информация, которую приведём ниже.

Статическое маскирование – процесс однонаправленного (без возможности направить процесс в обратную сторону) преобразования данных по заданному шаблону. Статическое маскирование происходит на уровне столбцов данных. Ко всем ячейкам в выбранном столбце применяется процесс преобразования данных по заданному шаблону. Применяется статическое маскирование в тех случаях, когда базу данных необходимо с какой-либо целью передать стороннему агенту или отделу организации; в зависимости от того, какую цель преследует получатель данных, будет определяться маскирующая функция. В результате статического маскирования создаётся копия таблицы, которая не содержит никакой информации о том, какие данные были в замаскированных столбцах до преобразования. Статическое маскирование планировалось реализовать с помощью мастера. Схема работы статического маскирования показана на рисунке 1.2.



Рисунок 1.2 – Статическое маскирование

Динамическое маскирование данных – процесс ограничения доступа к конфиденциальным данным за счёт сокрытия части этих данных от непривилегированных пользователей с помощью маски. Маска помогает скрыть часть данных от непривилегированных пользователей, защищая конфиденциальную информацию, но позволяя им каким-либо образом работать с этими данными. От статического маскирования данных динамическое отличается тем, что данные в самой таблице не меняются, меняется их представление для тех пользователей, которые не имеют доступа к демаскированной версии данных. Динамическое маскирование действует на уровне столбцов. Правила маскирования применяются к результатам запроса, таким образом логика действия приложений не меняется. Маскирование не запрещает доступа к данным (включая отсутствие запрета на запись и изменение данных), но закрывает данные от полного просмотра. Реализуется динамическое маскирование (в SQL Server) с помощью Transact-SQL. Для динамического маскирования доступны четыре функции:

- по умолчанию (default()) – полное маскирование данных в соответствии с типом: строка заменяется на xxxx (или меньшее число x, если строка меньше), число на 0, для даты ставится минимальная дата (1 января 1900 года), для двоичных данных ставится двоичный 0;
- Email (email()) – маскирование для адреса электронной почты (остаются открытыми первая буква электронного адреса и адреса 0-1 уровня доменного имени, остальные символы заменяются на X);
- случайное (random([start range], [end range])) – маскирование заменой на случайное число из приведённого диапазона [start range : end range] (работает, соответственно, только с числовыми значениями);
- пользовательское (partial(prefix, padding, suffix)) – маскирование, которое открывает заданное число символов по краям (prefix, suffix) строки данных и закрывает символы посередине заданной строкой (padding).

Невозможно динамическое маскирование для следующих столбцов:

- зашифрованные с помощью постоянного шифрования;
- FILESTREAM;
- часть набора столбцов COLUMN_SET;
- вычисляемый столбец (но немаскированный вычисляемый столбец может быть основан на маскированных столбцах, в таком случае он будет возвращать результат на основании маскированного значения);
- имеет зависимости (чтобы это обойти, можно удалить зависимости, создать маску и вернуть зависимости).

- также столбец с динамическим маскированием не может применяться в качестве ключа для полнотекстового индекса.

Важно обратить внимание на то, что динамическое маскирование не защищает от запросов с условиями. Выведенные данные будут замаскированными, но их можно вывести статистически, например, запросом:

```
SELECT ID, Name, Salary FROM Employees  
WHERE Salary > 99999 and Salary < 100001;
```

можно получить всех пользователей с зарплатой 10000, хотя в результативной таблице будет показана замаскированная зарплата. Необходимо применять маскирование в совокупности с другими средствами разграничения доступа и грамотно задавать права пользователям базы данных.

Ход работы

В процессе работы будет решена задача соблюдения конфиденциальности данных с помощью шифрования и маскирования конфиденциальной информации.

Практически в любой базе данных хранится информация, оставлять открытой которую было бы как минимум нежелательно. Простейший пример такой информации – номера кредитных карт клиентов. Для сокрытия такой информации от посторонних лиц используется разделение прав доступа к объектам базы данных. Однако эта мера работает только на уровне системы управления базами данных, а чтобы хранимые данные не было возможности считать посторонним пользователям в обход системы, их необходимо преобразовать (зашифровать) в форму, которая понятна только при обратном преобразовании (расшифровывании). Данные, используемые при таком преобразовании, можно хранить во внешних источниках или самой СУБД.

Реализация шифрования разделяется на следующие шаги:

- 1) определение информации в базе данных, подлежащей зашифровыванию;
- 2) определение степени конфиденциальности таковой информации (на основе как важности самой информации и требованиям к скорости работы с ней, так и среды, в рамках которой данная информация обрабатывается);
- 3) определение режима и параметров шифрования согласно строгости требований к поддержанию конфиденциальности;
- 4) применение указанного режима шифрования к информации.

Некоторым пользователям базы данных без полного уровня доступа не требуется знание всей информации, предоставленной в таблице. Когда необходимо защитить часть столбца данных, оставляя пользователям возможность работать с другой их частью, не имеющей в себе закрытой информации, применяется именно маскирование. Использование маскирования позволяет, например, сотрудникам в службе поддержки аутентифицировать пользователя, спросив у него первые четыре номера кредитной карты – при этом сами сотрудники полного номера знать не обязаны; или позволяет аналитическому отделу собирать статистику по наиболее используемым хостингам почтовых служб, не распространяя информацию об адресах и их владельцах.

Функционал маскирования возможно реализовать с помощью внешних средств – и в рамках однократной выборки данных это ещё можно назвать целесообразным, однако когда маскировать данные необходимо на регулярной основе (как уже в приведённом примере с сотрудником службы поддержки), осуществление этого маскирования на уровень выше (с уровня прикладного приложения на уровне СУБД) уменьшает поверхность атаки, тем самым уменьшая вероятность её выполнения (злоумышленник может перехватить данные ещё до обработки их приложением, однако если данные уже замаскиро-

ваны на уровне СУБД, это злоумышленнику ничего не даст). Если к этому добавить упрощение разработки прикладных приложений, которым более не нужно выполнять это маскирование, преимущества маскирования на уровне СУБД становятся ещё более очевидны. Недостатком такого подхода можно назвать только ограниченность маскирующих функций – что целиком и полностью зависит от разработчика СУБД.

Реализация маскирования разделяется на следующие шаги:

- 5) определение цели маскирования;
- 6) определение данных, подлежащих маскированию;
- 7) определение функции маскирования согласно определённой цели и виду данных;
- 8) осуществление процесса маскирования.

В рамках MS SQL Server будет продемонстрирован процесс маскирования на примере динамического маскирования.

Описание рассматриваемого примера базы данных

Перед тем, как перейти к выполнению поставленных задач, необходимо знать, с какой базой данных предстоит работать.

Компания Adventure Works Bicycles, Inc. является вымышленным оптовым продавцом велосипедов. Она производит и продаёт велосипеды, а также продаёт одежду и аксессуары к велосипедам розничным торговцам по всей стране.

Сотрудники компании выполняют типичные деловые операции, такие как:

- продажи;
- обработка и выполнение заказов;
- управление запасами;
- управление персоналом;
- бюджетирование.

Таблицы в базе данных подразделяются по категориям (схемам):

- dbo – системные базы данных, предназначенные для администрирования;
 - HumanResources – информация о сотрудниках компании;
 - Person – обобщённые таблицы для описания человека (сотрудника, покупателя, поставщика);
 - Production – описание продаваемых компанией товаров;
 - Purchasing – приобретаемые у поставщиков детали для велосипедов и прочие товары;
 - Sales – информация о продажах, продавцах и покупателях.
- Полноценное описание всего содержимого рассматриваемой базы данных дано в документе по следующему адресу: <https://dataedo.com/download/AdventureWorks.pdf>.

Шаг 1

Откройте базу данных AdventureWorks2017 и найдите в ней таблицу Sales.CreditCard (таблица с информацией по кредитным картам покупателей компании). Ознакомьтесь со структурой данной таблицы (рисунок 1.3).

SQLQuery2.sql - Wl...J54\username (52) X

```

/***** Скрипт для команды SelectTopNRows из среды SSMS *****/
SELECT TOP (1000) [CreditCardID]
      ,[CardType]
      ,[CardNumber]
      ,[ExpMonth]
      ,[ExpYear]
      ,[ModifiedDate]
FROM [AdventureWorks2017].[Sales].[CreditCard]

```

100 %

Результаты Сообщения

	CreditCardID	CardType	CardNumber	ExpMonth	ExpYear	ModifiedDate
1	1	SuperiorCard	33332664695310	11	2006	2013-07-29 00:00:00.000
2	2	Distinguish	55552127249722	8	2005	2013-12-05 00:00:00.000
3	3	ColonialVoice	77778344838353	7	2005	2014-01-14 00:00:00.000
4	4	ColonialVoice	77774915718248	7	2006	2013-05-20 00:00:00.000
5	5	Vista	11114404600042	4	2005	2013-02-01 00:00:00.000
6	6	Distinguish	55557132036181	9	2006	2014-04-10 00:00:00.000
7	7	Distinguish	55553635401028	6	2007	2013-02-01 00:00:00.000
8	8	SuperiorCard	33336081193101	7	2007	2013-06-30 00:00:00.000
9	9	Distinguish	55553465625901	2	2005	2013-09-23 00:00:00.000
10	10	SuperiorCard	33332126386493	8	2008	2011-08-31 00:00:00.000
11	11	SuperiorCard	33335352517363	10	2008	2014-05-04 00:00:00.000
12	12	SuperiorCard	33334316194519	4	2008	2012-05-30 00:00:00.000
13	13	Vista	11119775847802	11	2005	2014-03-01 00:00:00.000
14	14	Distinguish	55553287727410	10	2006	2013-11-19 00:00:00.000
15	15	SuperiorCard	33336866065599	11	2008	2013-01-29 00:00:00.000
16	16	Vista	11111985451507	8	2006	2013-12-30 00:00:00.000
17	17	ColonialVoice	77771220960729	8	2008	2014-01-16 00:00:00.000

Рисунок 1.3 – Таблица Sales.CreditCard

Шаг 2

Доступ к базе данных в рамках организации может быть произведён удалённо, в том числе по незащищённому каналу связи. В рамках данной работы рассматриваются два случая:

- 1) вся база данных расположена на одном сервере;
- 2) база данных распределена по различным серверам.

Таблица, приведённая на шаге 1, содержит информацию о типе кредитных карт покупателей, датах их окончания и номерах – данная информация является конфиденциальной и должна быть максимально защищена вне зависимости от внешних обстоятельств.

В первом случае необходимо шифрование базы данных по частям, - для шифрования только значимой информации, потому что один сервер был составлен без значительных запасных производственных ресурсов и не сможет справиться с шифрованием всей базы данных.

Во втором случае возможно шифрование всей базы данных, потому что каждый из множества серверов содержит в себе достаточный набор производственных ресурсов для шифрования своей части базы данных.

Шаг 3

В первом случае требуется шифрование отдельных таблиц базы данных. Необходимо использование общей схемы шифрования и отбор отдельных таблиц, подлежащих зашифровыванию.

Во втором случае допустимо использование схемы прозрачного шифрования данных (TDE) для зашифровывания всей базы данных.

Что касательно критической информации (таблица Sales.CreditCard, выбранная на шаге 1), в обоих случаях данные о кредитных картах покупателей критически уязвимы, и их необходимо защитить от максимально большего числа угроз, включая ренегатство со стороны администраторов баз данных. Для данных о кредитных картах целесообразно использовать механизм постоянного шифрования (AE).

Шаг 4

Часть 1. TDE

В рамках TDE зашифровывается целиком вся база данных. Шифрование базируется либо на внешнем ключе (с помощью ЕКМ), либо с использованием сертификата от сервера. В рамках данной лабораторной работы рассматривается использование сертификата.

Для выполнения TDE создайте новый запрос, нажав на соответствующую кнопку на панели инструментов, расположенной в верхней части окна SSMS. Введите следующий запрос (в качестве пароля, вводимого на третьей строке, используйте случайную строку символов длиной от 8; в качестве поясняющей информации к сертификату укажите свою фамилию):

USE master; - Используем схему «мастер» – действие происходит на уровне мета данных сервера

GO

CREATE MASTER KEY ENCRYPTION BY PASSWORD =
'<UseStrongPasswordHere>'; -- Создаётся мастер-ключ базы данных на основе пароля

GO

CREATE CERTIFICATE MyServerCert WITH SUBJECT = 'My DEK Certificate'; --
Создаётся сертификат, подтверждающий ключевую информацию

GO

USE AdventureWorks2017; -- Используем схему конкретной базы данных – действие происходит на уровне самих данных

GO

CREATE DATABASE ENCRYPTION KEY -- Создаём ключ шифрования базы данных (не обозначен в колесе иерархий шифрования, т.к. существует только в рамках TDE); данный ключ является симметричным.

WITH ALGORITHM = AES_128 -- Определяем алгоритм шифрования

ENCRYPTION BY SERVER CERTIFICATE MyServerCert; -- Определяем подтверждающий сертификат

GO

ALTER DATABASE AdventureWorks2017 -- Изменяем базу данных

SET ENCRYPTION ON; -- Включаем режим шифрования согласно заданным параметрам (с помощью ключа DEK, созданного ранее)

GO

В рамках данного запроса последовательно выполняется следующее:

I. На уровне мета данных сервера (системной базы данных master):

1) создаётся мастер-ключ базы данных (DMK), защищающийся с помощью пароля;

- 2) создаётся сертификат.
- II. На уровне непосредственной базы данных:
 - 3) создаётся симметричный ключ шифрования базы данных (Database Encryption Key) – ключ, используемый только в рамках TDE для шифрования всей базы данных. При создании ключа определяется алгоритм шифрования и сертификат, зашифровывающий этот ключ;
 - 4) необходимая база данных зашифровывается с помощью этого ключа.

Возможно также использование асимметричного ключа вместо сертификата для шифрования DEK. Для этого, соответственно, необходимо заменить код создания сертификата на код создания асимметричного ключа, и также при создании DEK указать, каков будет защищающий его ключ. Создание асимметричного ключа возможно с помощью конструкции `CREATE ASYMMETRIC KEY`. Ключ может быть либо создан с нуля, либо экспортирован из внешнего источника (файла или поставщика ключей). По умолчанию ключ, как и сертификат, зашифровывается с помощью мастер-ключа базы данных, но может быть также зашифрован и паролем. Простейший пример конструкции, которая создаст с нуля пару асимметричных ключей на основе алгоритма RSA с размером 2048 бит и зашифрует их мастер-ключом базы данных:

```
CREATE ASYMMETRIC KEY MyAssymKey  
WITH ALGORITHM = RSA_2048;
```

Чтобы успешно зашифровать DEK с помощью асимметричного ключа, создаваемый асимметричный ключ так или иначе должен находиться в поставщике расширенного управления ключами.

Прозрачное шифрование данных называется таковым, поскольку со стороны пользователя узнать о его существовании сложно. Чтобы проверить, запущен ли режим шифрования, повторите команду включения режима шифрования базы данных (`ALTER DATABASE AdventureWorks2017 SET ENCRYPTION ON`; в рамках базы данных: `USE AdventureWorks2017`;) и убедитесь в том, что выполнение запроса возвращает ошибку, в которой утверждается о том, что шифрование уже запущено (рисунок 1.4).

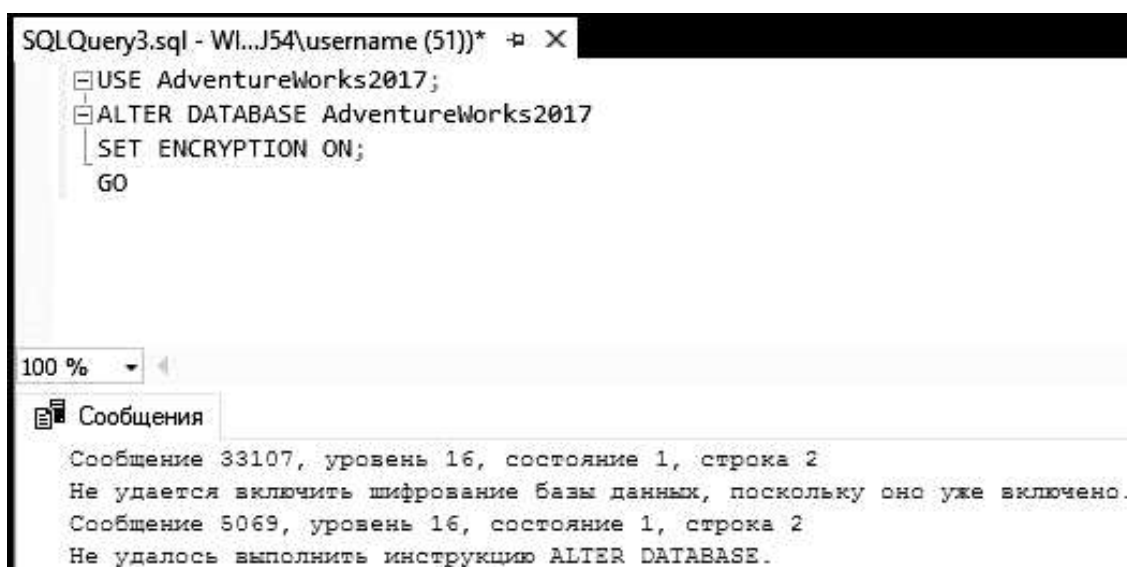


Рисунок 1.4 – Шифрование базы данных включено

Часть 2. Always Encrypted

В качестве примера рассмотрим столбец `CardNumber` в рамках рассмотренной ранее таблицы `Sales.CreditCard`.

Always Encrypted реализуется с использованием мастера («визарда»). Откройте мастер, выбрав в контекстном меню (таблицы или столбца) пункт «Зашифровать столбцы». В первую очередь в мастере выбирается столбец и параметры шифрования (рисунок 1.5). На вкладке «Выбор столбца» определяются зашифровываемые столбцы, тип шифрования и ключ шифрования (имеющийся либо новый).

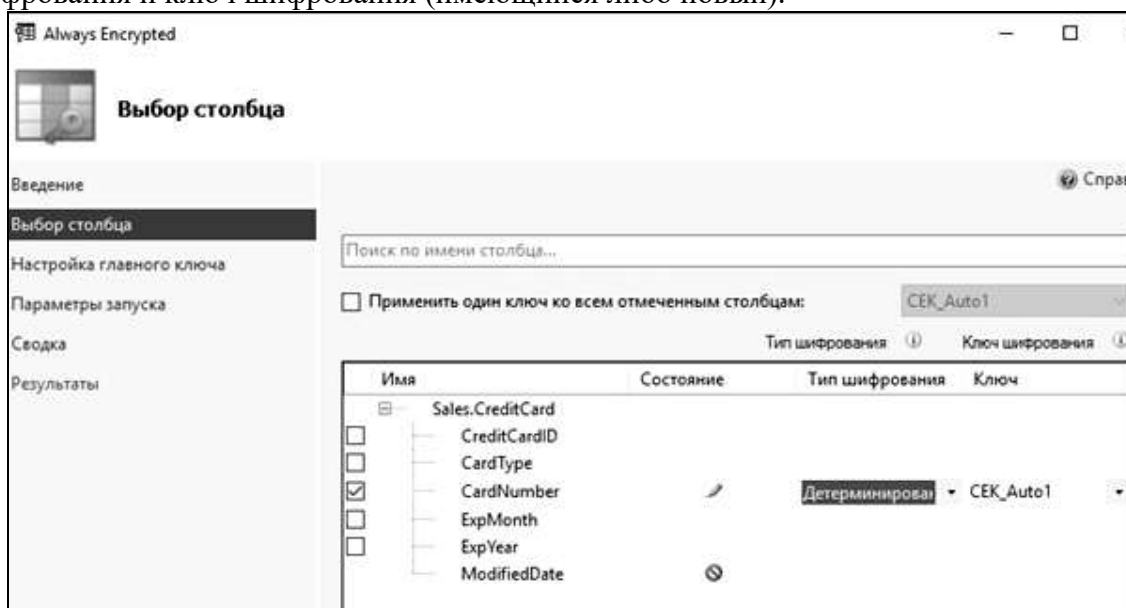


Рисунок 1.5 – Выбор столбца

Если был использован новый ключ шифрования, необходимо указать, к какому мастер-ключу он имеет отношение (рисунок 1.6) и каков источник этого ключа. Так как главный ключ ещё не был создан, возможен выбор только автоматического создания. Поскольку в рамках данной лабораторной работы рассматривается одна машина, в качестве поставщика хранилища оставьте хранилище сертификатов Windows.

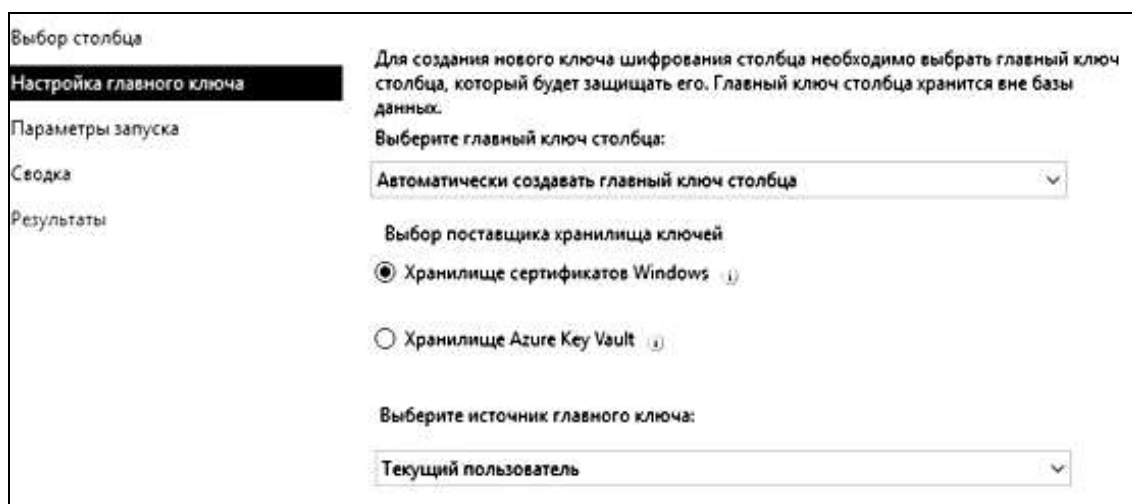


Рисунок 1.6 – Настройка главного ключа

На вкладке «Параметры запуска» определяется, когда будет запущен скрипт шифрования. В рамках реального предприятия возможно отложить процедуру шифрования до момента наименьшей нагрузки на сервер, в рамках же лабораторной работы оставьте немедленное исполнение.

На вкладке «Сводка» приводится для ознакомления итоговый набор параметров, собранный мастером.

После завершения работы мастера становится видно, что указанный столбец (столбцы) зашифрован (рисунок 1.7).



Рисунок 1.7 – Зашифрованный столбец

В таком виде данные хранятся в базе данных, и так их будут видеть все, кроме тех, кто подключится к базе данных с особым параметром и с использованием соответствующих ключей из хранилища.

Проверим считывание зашифрованных данных. Для этого необходимо вновь подключиться к СУБД, при этом указать в дополнительных параметрах параметр column encryption setting=enabled (рисунок 1.8).



Рисунок 1.8 – Указание параметра соединения

Далее выполните запрос на чтение к таблице, содержащей зашифрованную информацию (в контекстном меню таблицы «Выбрать первые 1000 строк»). SSMS сообщит о том, что для обработки данных, находящихся в зашифрованном столбце, необходимо включить определение параметров (рис. 1.9). Подтвердите включение. Убедитесь, что данные успешно расшифрованы.

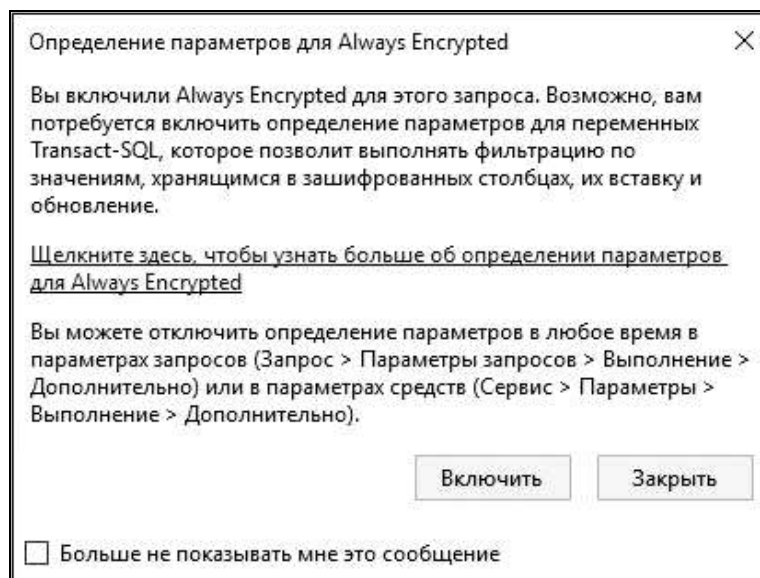


Рисунок 1.9 – Определение параметров для Always Encrypted

В определённый момент – например, при изменении структуры таблицы – может возникнуть необходимость выключить режим шифрования АЕ для каких-либо столбцов. Выполняется отмена режима с помощью того же мастера, с помощью которого шифрование включалось. Необходимо на вкладке «Выбор столбца» у зашифрованного столбца указать в пункте «Тип шифрования» – «Открытый текст» (рисунок 1.10). Выполните процедуру отмены режима шифрования.

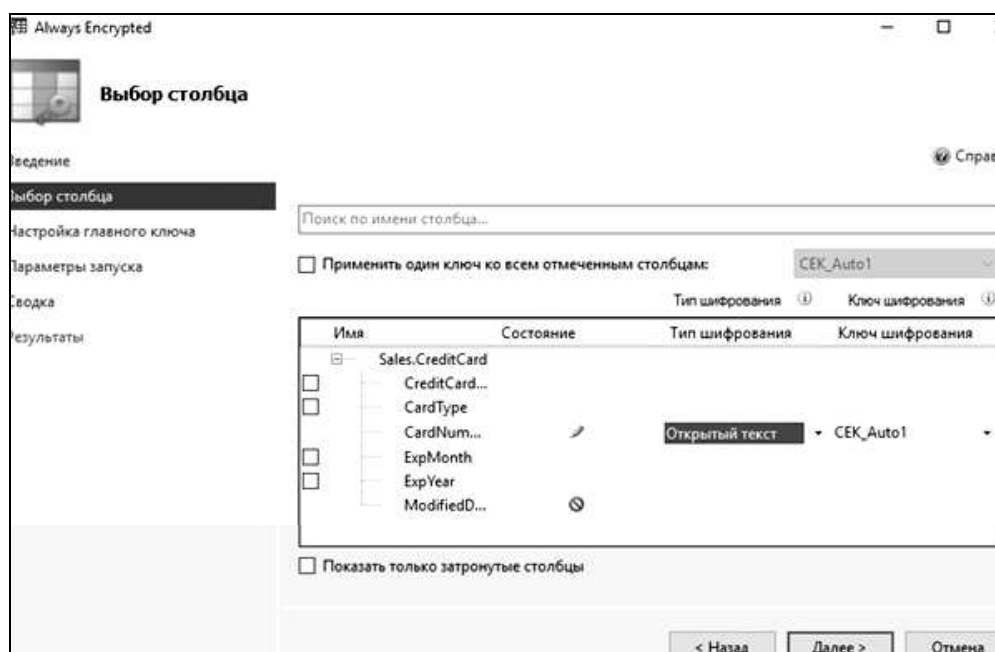


Рисунок 1.10 – Расшифровывание столбца

Ключ расшифрования ключевой информации (главный ключ столбца) хранится либо удалённо в хранилище Azure, либо локально в хранилище сертификатов Windows, что определяется при включении режима шифрования. Локально размещённый ключ можно взять из хранилища сертификатов и перенести в другую машину. Чтобы открыть это хранилище, введите в окне запуска (WIN+R) certmgr.msc. Ключ будет храниться в папке Личное и будет назван Always Encrypted Auto Certificate (рисунок 1.11).

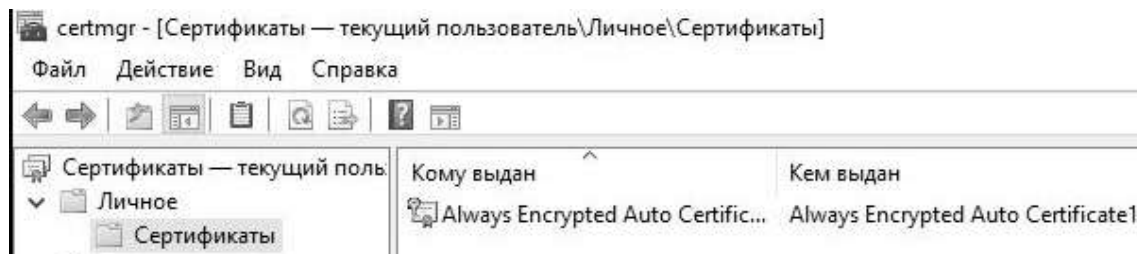


Рисунок 1.11 – Ключ Always Encrypted в хранилище сертификатов Windows

Шаг 5

Определим объект маскирования. Пусть, в совокупности с другими данными, необходимо обеспечить конфиденциальность номеров кредитных карт покупателей в процессе работы с данными теми сотрудниками, которым не нужно знать весь номер кредитной карты, но нужно знать первые четыре цифры этой кредитной карты для подтверждения покупки.

Шаг 6

Необходимо маскировать данные о номерах кредитных карт покупателей. Эти данные расположены в таблице Sales.CreditCard, столбце Card-Number. Снова запустите SSMS и зайдите в СУБД от имени текущей учётной записи Windows, откройте эту таблицу и ознакомьтесь с её структурой (рисунок 1.12).

Имя столбца	Тип данных	Разрешить ...
CreditCardID	int	☐
CardType	nvarchar(50)	☐
CardNumber	nvarchar(25)	☐
ExpMonth	tinyint	☐
ExpYear	smallint	☐
ModifiedDate	datetime	☐

Рисунок 1.12 – Структура таблицы Sales.CreditCard

Шаг 7

Необходимо оставить открытыми первые четыре символа номера и закрыть остальные (десять). Для этого необходимо воспользоваться функцией маскирования partial с параметрами 4, «XXXXXXXXXX» 0.

Шаг 8

Динамическое маскирование реализуется с помощью Transact-SQL. Выполните следующий запрос (в рамках БД AdventureWorks2017):

```
ALTER TABLE Sales.CreditCard
ALTER COLUMN CardNumber ADD MASKED WITH (FUNCTION = 'partial(4,
"XXXXXXXXXX", 0)');
```

Если запрос завершается ошибкой, то это может быть связано с шифрованием, выполненным на предыдущих шагах. Отмените его перед выполнением маскирования.

Просмотрите значения в таблице Sales.CreditCard (с помощью пункта «Выбрать первые 1000 строк» в контекстном меню таблицы или с помощью запроса SELECT). Убедитесь в том, что номер карты отображается незамаскированным (рисунок 1.13).

Результаты		Сообщения				
	CreditCardID	CardType	CardNumber	ExpMonth	ExpYear	ModifiedDate
1	1	SuperiorCard	33332664695310	11	2006	2013-07-29 00:00:00.000
2	2	Distinguish	55552127249722	8	2005	2013-12-05 00:00:00.000
3	3	ColonialVoice	77778344838353	7	2005	2014-01-14 00:00:00.000
4	4	ColonialVoice	77774915718248	7	2006	2013-05-20 00:00:00.000
5	5	Vista	11114404600042	4	2005	2013-02-01 00:00:00.000
6	6	Distinguish	55557132036181	9	2006	2014-04-10 00:00:00.000
7	7	Distinguish	55553635401028	6	2007	2013-02-01 00:00:00.000
8	8	SuperiorCard	33336081193101	7	2007	2013-06-30 00:00:00.000
9	9	Distinguish	55553465625901	2	2005	2013-09-23 00:00:00.000
10	10	SuperiorCard	33332126386493	8	2008	2011-08-31 00:00:00.000
11	11	SuperiorCard	33335352517363	10	2008	2014-05-04 00:00:00.000
12	12	SuperiorCard	33334316194519	4	2008	2012-05-30 00:00:00.000
13	13	Vista	11119775847802	11	2005	2014-03-01 00:00:00.000
14	14	Distinguish	55553287727410	10	2006	2013-11-19 00:00:00.000
15	15	SuperiorCard	33336866065599	11	2008	2013-01-29 00:00:00.000
16	16	Vista	11111985451507	8	2006	2013-12-30 00:00:00.000

Рисунок 1.13 – Незамаскированные данные

Это происходит потому, что у вашей текущей учётной записи (sa или вход с помощью аутентификации Windows) есть полные права на просмотр данных, включая право на просмотр незамаскированных данных. Чтобы проверить, замаскированы ли данные, просмотрите их от имени другого пользователя. Для этого можете создать временного пользователя без имени входа, выдать ему право SELECT на рассматриваемую таблицу и выполнить этот запрос от его имени. Или же создать полноценную пару имя входа-пользователь, выдать пользователю право SELECT, зайти по созданным именам входа и выполнить этот запрос. Выполнив это, убедитесь, что данные действительно замаскированы (рисунок 1.14).

100 %

Результаты Сообщения

	CreditCardID	CardType	CardNumber	ExpMonth	ExpYear	ModifiedDate
1	1	SuperiorCard	3333XXXXXXXXXX	11	2006	2013-07-29 00:00:00.000
2	2	Distinguish	5555XXXXXXXXXX	8	2005	2013-12-05 00:00:00.000
3	3	ColonialVoice	7777XXXXXXXXXX	7	2005	2014-01-14 00:00:00.000
4	4	ColonialVoice	7777XXXXXXXXXX	7	2006	2013-05-20 00:00:00.000
5	5	Vista	1111XXXXXXXXXX	4	2005	2013-02-01 00:00:00.000
6	6	Distinguish	5555XXXXXXXXXX	9	2006	2014-04-10 00:00:00.000
7	7	Distinguish	5555XXXXXXXXXX	6	2007	2013-02-01 00:00:00.000
8	8	SuperiorCard	3333XXXXXXXXXX	7	2007	2013-06-30 00:00:00.000
9	9	Distinguish	5555XXXXXXXXXX	2	2005	2013-09-23 00:00:00.000
10	10	SuperiorCard	3333XXXXXXXXXX	8	2008	2011-08-31 00:00:00.000
11	11	SuperiorCard	3333XXXXXXXXXX	10	2008	2014-05-04 00:00:00.000
12	12	SuperiorCard	3333XXXXXXXXXX	4	2008	2012-05-30 00:00:00.000
13	13	Vista	1111XXXXXXXXXX	11	2005	2014-03-01 00:00:00.000
14	14	Distinguish	5555XXXXXXXXXX	10	2006	2013-11-19 00:00:00.000
15	15	SuperiorCard	3333XXXXXXXXXX	11	2008	2013-01-29 00:00:00.000
16	16	Vista	1111XXXXXXXXXX	8	2006	2013-12-30 00:00:00.000

Рисунок 1.14 – Замаскированные данные

Не все пользователи должны видеть замаскированное значение. Право на просмотр немаскированных значений выдаётся пользователю или роли и распространяется на всю базу данных целиком. Чтобы дать разрешение на просмотр немаскированных данных, используется команда UNMASK:

```
GRANT UNMASK to [имя_пользователя];
```

Аналогичным образом, с помощью команды REVOKE разрешение отменяется.

Выдайте созданному ранее пользователю право на просмотр немаскированных значений. Убедитесь, просмотрев таблицу с маской от его имени, что значения действительно отображаются немаскированными.

Изменение бизнес-процессов в компании, или, возможно, необходимость добавить новые связи в таблицу, могут привести к необходимости изменить или удалить маскирование. Для изменения функции маскирования следует воспользоваться командой изменения столбца с ключевым словом MASKED и указанием функции маскирования (практически аналогично добавлению маски, только без слова ADD):

```
ALTER COLUMN [имя_столбца] MASKED WITH (FUNCTION = '[функция]');
```

Для того же, чтобы удалить маску, необходимо выполнить следующую команду:

```
ALTER COLUMN [имя_столбца] DROP MASKED
```

Проверьте на выбранном столбце ещё какую-нибудь маскирующую функцию (например, random([0, 999999])). Удалите маску со столбца.

Контрольные вопросы

- 1) Что такое шифрование и как оно реализуется?
- 2) Опишите симметричную и асимметричную схему шифрования. Для чего используется открытый ключ, для чего – закрытый?

- 3) Какие есть механизмы шифрования в рамках общей схемы шифрования SQL Server?
- 4) Опишите связь между сертификатом и асимметричным ключом и направления их использования.
- 5) Опишите процесс шифрования по прозрачному шифрованию данных (TDE).
- 6) Для чего предназначены и как связаны мастер-ключи (сервисный и базы данных) (SMK и DMK)?
- 7) Что такое и для чего предназначено постоянное шифрование (Always Encrypted)?
- 8) Какими ключами оперирует постоянное шифрование и для чего эти ключи предназначены?
- 9) Что такое маскирование?
- 10) В чём различия между статическим и динамическим маскированием?
- 11) Для чего чаще всего используется статическое маскирование? Для чего динамическое?
- 12) Какие функции динамического маскирования есть в SQL Server?
- 13) Что позволяет динамическое маскирование (в рамках прав доступа) в SQL Server? В чём его уязвимость относительно этого?
- 14) Каким образом создаётся, изменяется и удаляется динамическая маска в SQL Server?

Задание

Выполните *Ход работы* согласно нижеприведённым вариантам заданий. Составьте отчёт по выполненной работе и защитите его.

Задание по TDE (прозрачному шифрованию данных)

Выполните шифрование всей базы данных AdventureWorks2017, используя для шифрования DEK самоподписанный сертификат. В таблице 1.1 для каждого варианта указан алгоритм симметричного шифрования, который должен быть использован для DEK.

Таблица 1.1 – Варианты задания по TDE

Номер варианта	Алгоритм шифрования DEK
1	AES_256
2	AES_192
3	AES_128

Задание по АЕ (постоянному шифрованию данных)

Ознакомьтесь с описанной ситуацией и зашифруйте столбец с помощью режима АЕ согласно ней. Варианты заданий:

- 1) При написании обзоров на товары компании Adventure Works покупатели оставляли своё имя и электронную почту. Контактные данные обзорщиков необходимо скрывать от всеобщего доступа и защищать.
- 2) Велосипеды компании не так просты, как кажутся. В их составе множество различных мелких деталей. Некоторые из них изготавливаются по секретным разработкам компании, добавляя их продукции особенности для повышения конкурентоспособности. Не стоило бы про них знать конкурентам.

3) Согласно ФЗ «О персональных данных», персональными данными являются «любые сведения, относящиеся к прямо или косвенно определённом или определяемому физическому лицу (субъекту персональных данных), которые предоставляются другому физическому или юридическому лицу либо лицам». Пожалуй, более определяющих данных, нежели номер паспорта, у физического лица быть не может. Такие данные, естественно, подлежат защите. Правда, в рамках компании, поскольку она расположена в Америке, используется несколько другой номер – social security number, который является частным случаем более общего понятия – national identification number.

4) Каждый сотрудник имеет право хранить размер своей заработной платы в секрете. Бездушная автоматизированная машина, обрабатывающая цифры его зарплаты, эта информация нужна. Вполне возможно, она также нужна и кассиру. Но в открытом доступе она точно не нужна (Примечание: в этом варианте задания необходимо перед шифрованием удалить ограничение на столбец).

5) На основании того, что человек приобретает, легко определить его интересы. Этим могут воспользоваться не только легитимные организации по желанию самих пользователей, но и спамеры. Поскольку данные самих пользователей защищаются в других таблицах, необходимо скрыть сопоставление пользователя с покупателем, скрыв его приобретения.

6) Несмотря на то, что пароли пользователей хранятся в хешированном виде, их всё равно необходимо хранить в зашифрованной форме, потому что по большому количеству хешей злоумышленник может почерпнуть полезную для себя информацию, например, определить функцию хеширования.

7) От того, сколько продавец продаёт товаров, зависит в том числе и его оплата. Соответственно, такая информация должна быть конфиденциальна. Однако данные о самих продажах нужны многим отделам, в том числе они выкладываются в открытый доступ в рамках ежегодного отчёта в совокупной форме. Таким образом, необходимо скрыть продажи конкретных продавцов.

8) Никто не хочет обрабатывать «выгодные предложения, от которых просто невозможно отказаться», к тому же на своём рабочем телефоне, который нельзя просто так отключить или проигнорировать. Или телефоне, который покупатель дал компании, чьей обязанностью является защищать пользовательские данные. Некрасиво получится, если номера таких телефонов будут в открытом доступе.

9) По каждому продукту организации имеется минимум одна техническая документация. В списке таких документов имеются в том числе и документы ДСП, например, инструкция по производству определённых деталей. Поскольку в составе продукции компании имеются собственные засекреченные разработки, такие документы следовало бы закрыть от общего доступа.

10) Далеко не все пользователи, разбрасывающие свои электронные почтовые адреса направо и налево, готовы к спаму. Компания, конечно, уважает своих пользователей и рассылает им только действительно важную информацию. Но будут ли настолько добры злоумышленники, продающие спамерам украденные почтовые адреса? Сомнительно.

Задание по маскированию

Проанализируйте ситуацию, данную в вашем варианте, и определите столбцы (столбец), требующие маскирования, и режим маскирования. Примените выбранное маскирование к столбцам. Укажите, кому следует иметь доступ к незамаскированным значениям. Объясните выбор столбцов и субъектов, имеющих доступ к незамаскированным значениям. Варианты задания:

1) При написании обзоров на товары компании Adventure Works покупатели оставляли своё имя и электронную почту. Целью сбора этих электронных адресов явля-

ется оповещение о новых товарах и акциях. Чтобы использовать их в других целях, например, статистических, необходимо эти адреса обезличить.

2) Данные о сотрудниках нужны практически всему начальственному составу организации и практически всему отделу кадров. Если многие данные о сотрудниках действительно нужны для работы, то номер паспорта в бизнес-процессе нужен далеко не всем. Правда, в рамках компании, поскольку она расположена в Америке, используется несколько другой номер – social security number, который является частным случаем более общего понятия – national identification number.

3) С помощью, в том числе, номера отслеживания состояния доставки товара многие перевозчики осуществляют аутентификацию заказчика. Знание этого номера может позволить злоумышленнику перехватить какой-либо заказ, заявив себя как представитель покупателя.

4) Заработные платы сотрудников являются их личной тайной. Время от времени все экономические данные в базе передаются отделу аналитики, в том числе точные цифры заработных плат. Однако знать, какому человеку какие заработные платы соответствуют, им нет нужды.

5) На основании электронных адресов в организации осуществляется подтверждение аутентификации в системе – как для пользователей, так и для сотрудников. При подтверждении, однако, нет никакой нужды подтверждающему знать полный адрес.

Дополнительное задание по шифрованию

Покажите на примере своего варианта, или любого другого шифрования в рамках SQL Server, что до шифрования данные возможно считать из файла базы данных, а после – невозможно. При этом не разрешается использовать никакие прикладные к СУБД программы-интерфейсы (SQL Server Management Studio является таковой).

Дополнительное задание по маскированию

Проанализируйте структуру базы AdventureWorks2017 на предмет возможных применений и определите ещё как минимум 5 таблиц, к которым возможно осмысленное применение маскирования. Укажите режим маскирования, столбцы, к которым следует применять маскирование, и категорию пользователей, которые должны иметь доступ к незамаскированным значениям. Сделанный выбор обоснуйте.

ЛАБОРАТОРНАЯ РАБОТА № 2

Разделение прав доступа

Целью работы является изучение принципов работы разделения прав доступа в базах данных на примере СУБД Microsoft SQL Server.

Теоретический материал

Разделение прав доступа – способ разграничения субъектам доступа к объектам базы данных.

Обычный пользователь в SQL Server имеет имя входа и имя пользователя. По имени входа пользователь осуществляет вход в систему, имя пользователя определяется на уровне всего экземпляра SQL Server. Имя пользователя определяется для каждой базы данных отдельно, на основе имени пользователя осуществляется непосредственное разграничение доступа к данным. Один пользователь может иметь имя пользователя в нескольких базах данных одновременно, но в каждой отдельной базе данных у пользователя имеется только одно имя пользователя. Аналогично именам на уровне сервера пользователи могут быть членами роли сервера, и на уровне базы данных пользователи могут быть членами роли базы данных.

Виды имён входа разделяются по способу входа: аутентификация SQL Server, Windows или Active Directory.

При создании имени пользователя определяется, каким образом (по какому имени входа) у него осуществляется доступ к базе данных. Имя пользователя может быть создано и без имени входа; пользователь, естественно, не может зайти в базу данных под этим именем, но ему можно выдавать разрешения.

Список объектов, к которым может иметь доступ субъект, довольно обширен. По областям объекты могут принадлежать уровню сервера, базы данных или схемы. На уровне сервера объекты общие (роль сервера, имя входа, база данных и пр.), на уровне базы данных объекты общие для всей базы данных (криптографические данные, служба, пользователь, роль приложения, схема и пр.), на уровне схемы располагаются наиболее детализированные объекты (тип схемы, таблица, представление, процедура, внешняя таблица и пр.).

В SQL Server имеется набор предопределённых ролей и имён. На уровне сервера имеется набор ролей для различных администраторов:

- sysadmin – полный доступ;
- serveradmin – параметры конфигурации и выключение сервера;
- securityadmin – распределение прав доступа, сброс паролей (может назначать разрешения, поэтому почти эквивалентен sysadmin);
- processadmin – завершение процессов в сервере;
- setupadmin – добавление и удаление связанных серверов (при работе в SSMS необходимо также быть в роли sysadmin);
- bulkadmin – инструкция BULK INSERT (импорт базы данных или таблицы в виде файла);
- diskadmin – управление файлами;
- dbcreator – создание, удаление, изменение, восстановление баз данных.

Также имеется роль public – наиболее общая роль, которая не предоставляет особых привилегий и к которой в обязательном порядке принадлежат все имена входа. Предназначена она для выдачи прав доступа, которые должны иметь все пользователи. Изменять права доступа ролей сервера можно только для роли public и пользовательских ролей.

Имеется одно предопределённое имя входа: sa. Пользователь sa (системный администратор) имеет доступ ко всем субъектам и ни в чём не ограничен, однако само имя входа можно отключить. При установке экземпляра SQL Server создаётся пользователь с аутентификацией Windows, также наделённый всеми правами, под которым был создан этот экземпляр.

На уровне базы данных (БД) также имеется роль public. Эта роль по умолчанию имеет множество прав доступа, которые необходимы для самых базовых процессов в базе данных и не представляют угрозы безопасности. Права доступа этой роли также можно менять. Все имена входа по умолчанию принадлежат к этой роли. Имеются также и некоторые другие предопределённые роли, название которых, как правило, говорит само за себя (db_owner – полный доступ, db_securityadmin – разделение прав доступа, db_datareader – полное чтение всех пользовательских таблиц, db_denydatareader – полный запрет на чтение всех пользовательских таблиц, и пр.).

Имеются следующие предопределённые имена пользователя в каждой базе данных: dbo, guest, sys и INFORMATION_SCHEMA. Пользователь dbo имеет доступ ко всем объектам в базе данных и не может быть в них ограничен. Под этим именем подключается набор пользователей, в число которых входят различные администраторы, пользователи роли sysadmin и имени входа sa, а также владелец базы данных. Пользователь guest (гость) имеет набор разрешений, которые наследуются пользователями, имеющими доступ (CONNECT) к базе данных. Пользователи sys и INFORMATION_SCHEMA являются системными и операции с ними недоступны. Всех этих пользователей нельзя удалить (хотя у guest можно отменить разрешение на CONNECT, что не позволит под ним зайти). Также, при использовании некоторых сертификатов, для их использования создаются системные пользователи, имя которых обрамлено хэш-символами (##).

Для создания имени входа используется инструкция CREATE LOGIN. При создании определяются имя пользователя и пароль, могут также задаваться и параметры вроде необходимости смены пароля при первом входе, ограниченном времени пароля, применения стандартных политик и пр. Чтобы создать имя входа с авторизацией в SQL Server, следует указать имя пользователя и пароль:

```
CREATE LOGIN <имя> WITH PASSWORD = '<пароль >';
```

а чтобы создать имя входа с авторизацией в Windows, следует указать домен и имя пользователя:

```
CREATE LOGIN [<домен>\<имя>] FROM WINDOWS;
```

Для создания имени пользователя используется инструкция CREATE USER. Чаще всего создаются пользователи с привязкой к имени входа, для чего оно указывается:

```
CREATE USER <имя пользователя> FOR LOGIN <имя входа>;
```

Иногда, впрочем, необходимо создать пользователя без входа, для чего вместо FOR LOGIN <имя входа> указывается WITHOUT LOGIN. Это используется, как правило, для выдачи разрешений перед предоставлением доступа. Чтобы впоследствии сопоставить такого пользователя с именем входа, используется инструкция ALTER USER <имя пользователя> с указанием нужного имени входа (WITH LOGIN <имя входа>). Также не имеют возможности входа пользователи, созданные на основе сертификатов и асимметричных ключей. Такие пользователи создаются для подписи соответствующих криптографических модулей.

Чтобы создать и удалить роль, используются инструкции CREATE и DROP. Их синтаксис таков:

CREATE SERVER ROLE <имя роли> '<имя входа владельца роли>' -- создание роли уровня сервера (если не указано имя входа владельца, им станет исполнитель инструкции)

DROP SERVER ROLE <имя роли> -- удаление роли уровня сервера (возможно только если роль не имеет членов)

CREATE ROLE <имя роли> '<имя пользователя владельца роли>' -- создание роли уровня базы данных (при исполнении инструкции убедитесь, что находитесь в рамках нужной базы данных, что определяется в окне SSMS на самой нижней из верхних панели слева)

DROP ROLE <имя роли> -- удаление роли уровня базы данных (аналогично)

Чтобы сопоставить имя с ролью, необходимо использовать инструкцию ALTER. Синтаксис:

ALTER ROLE <имя роли> ADD MEMBER <имя пользователя>; -- включение имени пользователя в роль базы данных

ALTER SERVER ROLE <имя роли> ADD MEMBER <имя входа>; -- включение имени входа в роль сервера

ALTER ROLE <имя роли> DROP MEMBER <имя пользователя>; -- удаление имени пользователя из роли базы данных

ALTER SERVER ROLE <имя роли> DROP MEMBER <имя входа>; -- удаление имени входа из роли сервера

Также этой инструкцией можно изменить имя роли:

ALTER (SERVER) ROLE <имя роли> WITH NAME = <новое имя>;

Помимо использованных запросов, многие описанные объекты (роли, имена входа, имена пользователей) можно создать и настроить с помощью графического интерфейса SSMS. Происходит это в рамках папки «Безопасность», расположенной в корне для объектов уровня сервера и в корне базы данных для объектов уровня базы данных (рисунок 2.1).

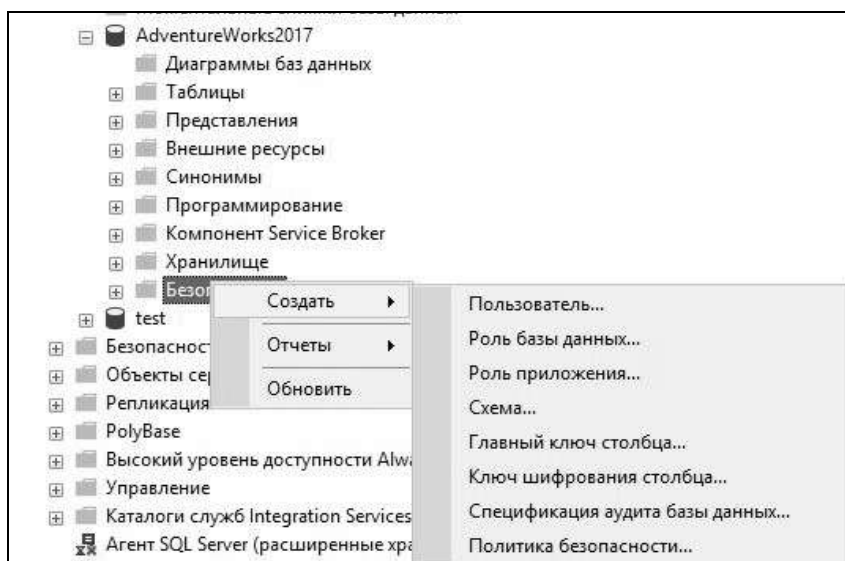


Рисунок 2.1 – Создание субъектов безопасности

С каждым объектом в SQL Server связан набор разрешений, которые могут быть выданы субъектам. Всего в SQL Server имеется 230 видов разрешений (многие из них специфицированы для определённых объектов). Разрешения предоставляются и отнимаются командами GRANT, REVOKE и DENY. GRANT выдаёт разрешение, REVOKE отнимает выданное напрямую данному субъекту разрешение, DENY запрещает этому субъекту выполнение указанного права вне зависимости от прочих разрешений. Невозможно изменить разрешения системным именам (sys, sa, INFORMATION_SCHEMA и dbo), владельцу сущности и пользователю, исполняющему процесс изменения прав доступа (самому себе).

Основной список разрешений, с которым осуществляется работа при разделении прав доступа, это разрешения пользователям на SELECT, INSERT, UPDATE и DELETE определённых таблиц, представлений и столбцов в них. Также следует отметить следующие разрешения: CONTROL, дающее полный доступ к объекту; EXECUTE, позволяющее запускать функции; ALTER (ANY), позволяющее изменять конкретный объект (все объекты указанного типа); CONNECT, позволяющее осуществлять подключение (отмена этого разрешения вообще запретит пользователю подключаться, то есть практически отберёт все права без их отъёма на уровне самой СУБД); и CREATE, позволяющее создавать объекты указанного типа.

Синтаксис этих команд специфичен для различных видов объектов. Простейший вариант выглядит следующим образом:

```
GRANT [список разрешений] ON [объект] TO [субъект];
DENY [список разрешений] ON [объект] TO [субъект];
REVOKE [список разрешений] ON [объект] FROM [субъект];
```

Обработка прав доступа с ролями аналогична обработке прав доступа с именами.

Также для некоторых объектов (в том числе таблиц) возможна настройка прав доступа в графическом интерфейсе SSMS. Для этого следует открыть свойства нужного объекта, вкладку «Разрешения» и с помощью кнопки «Найти...» определить нужный субъект, которому далее указать необходимые права (рисунок 2.2). В рамках таблицы, для некоторых разрешений, где это применимо, возможно указание прав доступа к конкретным столбцам (рисунок 2.3).

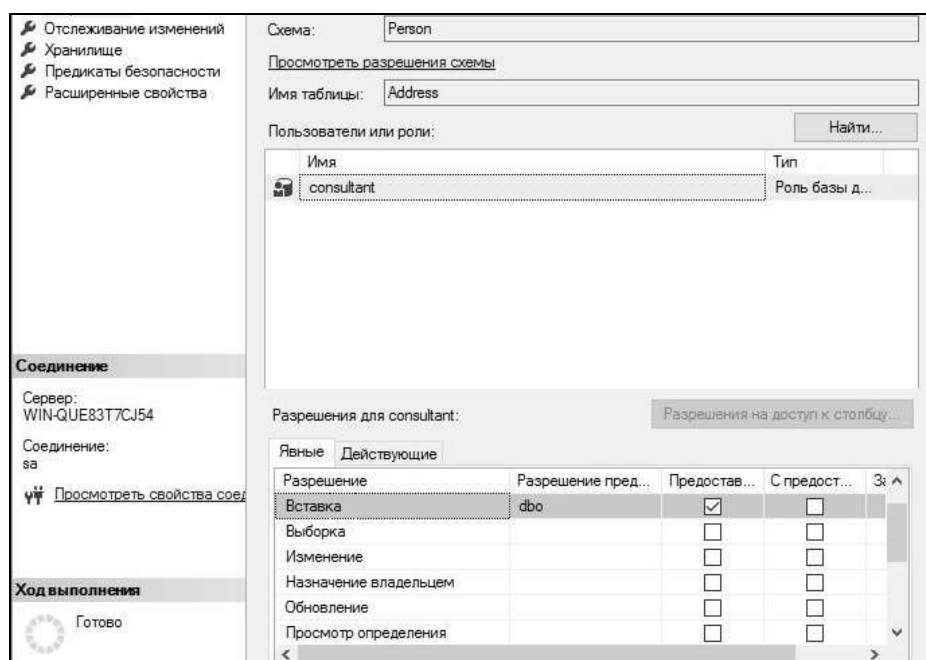


Рисунок 2.2 – Задание прав доступа субъекту

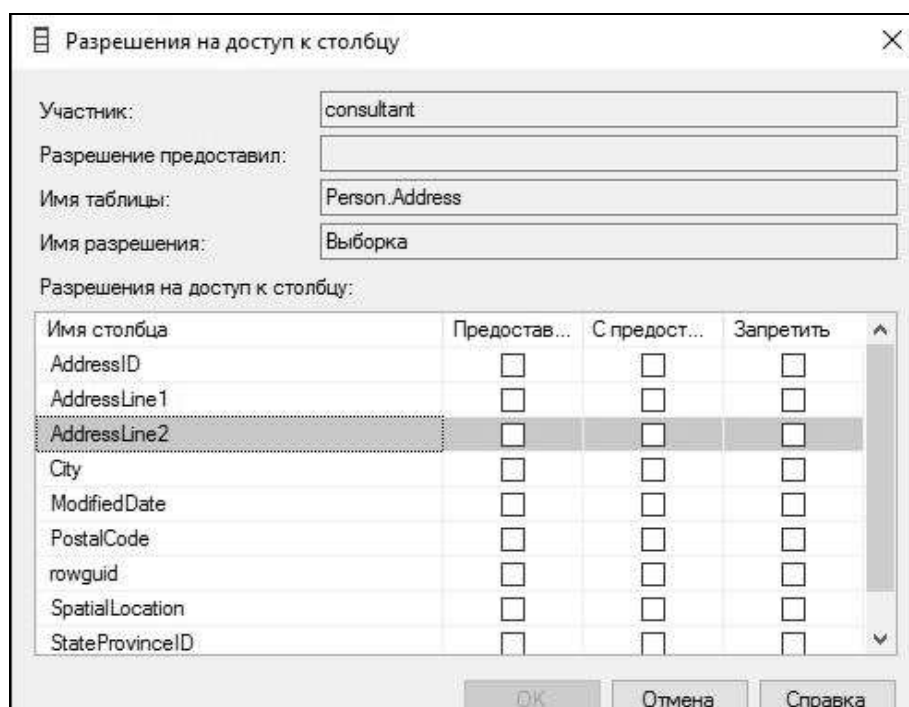


Рисунок 2.3 – Задание прав доступа субъекту на уровне столбцов

Ход работы

В процессе работы будет выполнена задача соблюдения конфиденциальности данных за счёт разделения прав доступа. Перед выполнением работы проверьте, что у вас в разделе «Безопасность» в свойствах сервера в качестве «Серверной проверки подлинности» стоит «Проверка подлинности SQL Server и Windows». Если нет, смените и перезапустите службы в «Диспетчере конфигурации SQL сервера». Также проверьте в «Диспетчере конфигурации SQL сервера» -> «Сетевая конфигурация», включены ли именованные каналы и TCP/IP протокол. Если нет, включите и перезапустите службы в «Диспетчере конфигурации SQL сервера». Если выдает ошибку с описанием, касающемся именованных каналов, то при подключении под созданным именем входа в параметрах подключения смените протокол на TCP/IP (при условии, что вы его уже включили в диспетчере конфигурации).

Основным средством соблюдения конфиденциальности является разграничение прав доступа. Этот инструмент буквально реализует требование конфиденциальности, а именно разрешает доступ тем, кто должен его иметь, и запрещает всем остальным. Система разделения прав доступа в SQL Server весьма гибка. В её рамках возможно разделение прав на самых различных уровнях: от возможности создавать базы данных и настраивать безопасность сервера до доступа к конкретным строкам (защита на уровне строк рассмотрена в следующей лабораторной). На практике, чем крупнее организация, тем более узкие и точные выдаются права доступа каждому сотруднику, и тем лучше выполняется правило минимальных прав доступа. В рамках SQL Server возможно достаточно точное разделение прав за счёт наличия 230 различных видов разрешений на множество субъектов, от полного доступа до возможности выполнения конкретной операции. Однако всё-таки основным защищаемым объектом является сама информация, то есть содержимое таблиц в базах данных.

Рассматриваемая задача разделяется на следующие шаги (в рамках одной пользовательской БД):

- 1) определение списка всех пользователей, которые должны работать с базой данных, и их прав доступа (к бизнес-сущностям);

- 2) определение списка ролей согласно всем уникальным случаям разделения прав;
- 3) создание (или сопоставление с predetermined) необходимых ролей и наделение их требуемыми правами доступа;
- 4) создание пользователей и сопоставление их с ролями.

Шаг 1

Для примера рассмотрим одного пользователя: консультанта. Консультант – это специалист по работе с клиентами, задачами которого являются демонстрация клиенту товара, помощь в выборе товара, сбор отзывов от клиентов. Соответственно, консультанту необходим доступ к таблицам, в которых содержится базовая информация о товаре, его местонахождении, стоимости, а также документация к товару.

Шаг 2

В рамках нашего примера используется только один пользователь, поэтому роль будет только одна. На уровне сервера консультанту особых привилегий не требуется, поэтому он будет членом только роли public. На уровне базы данных создадим новую роль и назовём эту роль consultant.

Шаг 3

Для создания роли можно воспользоваться следующей инструкцией:

CREATE ROLE consultant;

Либо же воспользоваться графическим интерфейсом, выбрав в контекстном меню любой из папок: «Безопасность», «Роли», «Роли базы данных» пункт «Создать» -> «Роль базы данных», и в появившемся окне указать имя роли (рисунок 2.4).

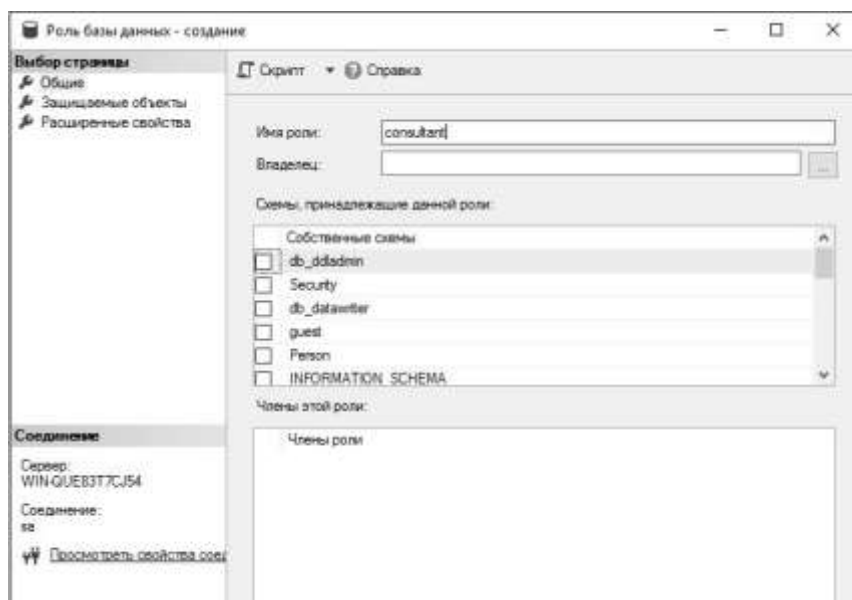


Рисунок 2.4 – Создание роли базы данных

Владельцем роли по умолчанию станет её создатель, т. е., скорее всего (если вы зашли в СУБД под именем входа sa или через аутентификацию Windows), пользователь dbo.

С учётом необходимых прав, определённых на первом шаге, консультанту нужны будут следующие таблицы со следующими правами (таблица 2.1).

Таблица 2.1 – Определение необходимых прав доступа

Таблица	SELECT	INSERT	UPDATE	DELETE
Production.Product	+	-	-	-
Production.Document	+	-	-	-
Production.ProductCategory	+	-	-	-
Production.ProductDescription	+	-	-	-
Production.ProductPhoto	+	-	-	-
Production.ProductReview	+	+	+	+

Таблицы консультанту необходимы только на чтение, кроме таблицы с оформлением обзора от клиента. Клиент может обозревать продукт в магазине (без онлайн доступа), поэтому консультант будет вносить данные в базу за него. Клиент также может изменить свой обзор и отказаться от него, поэтому права будут нужны на все базовые действия.

Далее необходимо наделить роль правами. Сделать это можно как с помощью графического интерфейса (в свойствах каждой таблицы), так и с помощью команд. Покажем команду на примере таблицы с обзорами пользователей:

```
GRANT SELECT, INSERT, UPDATE, DELETE ON Production.ProductReview TO consultant;
```

Наделите роль консультанта заданными правами.

Шаг 4

Когда все права определены, можно создавать пользователей и давать им доступ к базе данных. Создадим имя входа с аутентификацией SQL Server. Для создания имени входа воспользуйтесь инструкцией:

```
CREATE LOGIN username WITH PASSWORD = '12345';
```

Если выдаётся ошибка о слишком простом пароле, добавьте к паролю прописные, строчные английские буквы и знаки препинания, а также учтите минимальную длину пароля в 8 символов.

Пользователя также можно создать с помощью графического интерфейса. Для этого, из корневой папки «Безопасность», выберите «Создать» -> «Имя входа» и определите нужные параметры имени входа (рисунок 2.5).

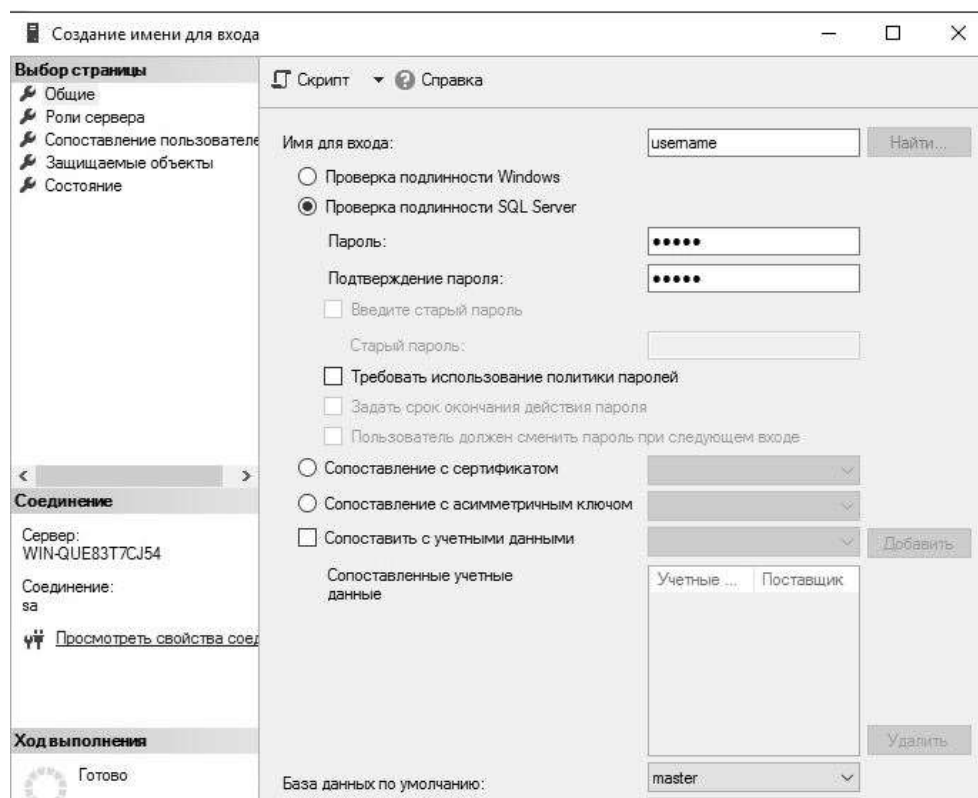


Рисунок 2.5 – Создание имени входа

Далее необходимо создать имя пользователя в рамках базы данных AdventureWorks2017. Реализуется это аналогично созданию имени входа, также возможно создание с помощью графического интерфейса (рисунок 2.6) или TRANSACT-SQL. Инструкция выглядит следующим образом (убедитесь, что находитесь на уровне базы данных AdventureWorks2017):

```
CREATE USER username FOR LOGIN username;
```

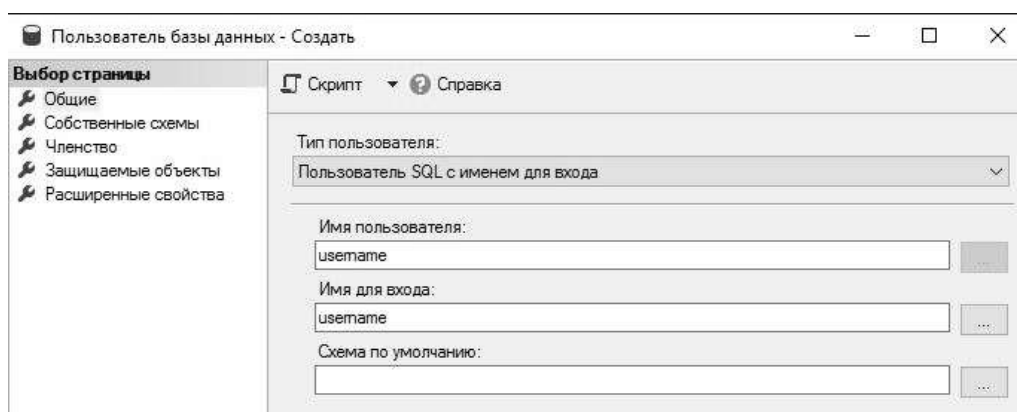


Рисунок 2.6 – Создание имени пользователя

Контрольные вопросы

- 1) Опишите систему имён и ролей в SQL Server.
- 2) Назовите несколько predefined ролей сервера и опишите их права.
- 3) Назовите несколько predefined ролей базы данных и опишите их права.

- 4) Назовите все предопределённые имена входа и имена пользователя и опишите их права.
- 5) Как пользователь, не имеющий никаких явно выделенных ему прав в рамках базы данных и не имеющий явного членства в ролях, может осуществлять простейшие операции, такие как подключение к базе данных, чтение списка баз данных и пр.?
- 6) Какими командами создаются, изменяются и удаляются имена и роли? Какими командами выдаются, отзываются и запрещаются права?
- 7) Опишите различие между отзывом права и запретом на право.

Задание

Первое задание

Создайте имя входа и присвойте ему предопределённую роль сервера согласно указанным в варианте правам (таблица 2.2). Создайте другое имя входа и имя пользователя в рамках базы данных AdventureWorks2017, сопоставьте их и присвойте имени пользователя предопределённую роль согласно указанным в варианте правам (список ролей можно посмотреть в папке «Безопасность» -> «Роли» -> «Роли базы данных»).

Таблица 2.2 – Варианты к первому заданию

Номер варианта	Права роли сервера	Права роли БД
1	Доступ без ограничений	Строгий запрет на чтение
2	Работа с файлами	Выполнение команд DDL
3	Настройка и отключение сервера	Работа с данными
4	Работа с базами данных	Управление правами на удалённый доступ
5	Доступ без ограничений	Строгий запрет на изменение
6	Работа с правами доступа	Чтение данных
7	Настройка и отключение сервера	Управление разрешениями
8	Инструкция BULK INSERT	Владелец базы данных
9	Работа с базами данных	Создание резервной копии
10	Остановка процессов	Выполнение команд DDL

Второе задание

На основании заданной в варианте должности создайте роль уровня базы данных и пару имя входа/имя пользователя, определите круг данных, к которым должен выделяться доступ, и на основании этого круга выберите минимум 5 таблиц (либо обоснуйте, почему меньшего числа достаточно) и права доступа (рассмотрите только 4 права: SELECT, INSERT, UPDATE, DELETE) к ним и выдайте определённые права доступа созданной роли. Составьте таблицу согласно определённым правам (аналогично Таблице 2.1). Вкратце (1-2 предложения на таблицу) объясните выбор таблиц и прав доступа к ней. Сопоставьте имя пользователя с ролью. Продемонстрируйте работу преподавателю.

Внимание: не опускайтесь на уровень ниже таблицы. Со строками работа будет осуществляться в рамках следующей лабораторной работы.

Варианты должностей:

- 1) бухгалтер;
- 2) ведущий собеседование по найму на работу;
- 3) сотрудник отдела кадров;
- 4) продавец;
- 5) покупатель на сайте организации (незарегистрированный);

- 6) покупатель на сайте организации (зарегистрированный, права отдельно от незарегистрированного);
- 7) дизайнер онлайн магазина;
- 8) разработчик новой продукции;
- 9) кассир (выдаёт зарплату сотрудникам);
- 10) производитель продукции.

Дополнительное задание

В рамках своего варианта, согласно выполненному второму заданию, рассмотрите все столбцы выбранных вами таблиц. Уточните права к каждому из столбцов. Обоснуйте свой выбор для каждого столбца.

ЛАБОРАТОРНАЯ РАБОТА № 3

Защита на уровне строк

Целью работы является изучение принципов работы защиты на уровне строк (Row-Level Security) на примере СУБД Microsoft SQL Server.

Теоретический материал

Защита на уровне строк – способ разграничения прав доступа к содержимому таблицы на уровне строк (разграничение права идёт построчно) в зависимости от характеристик пользователя, который выполняет запрос, или некоторых иных условий. Основная причина ввода такого рода разграничений – облегчение процесса программирования базы данных: ранее процесс разграничения прав доступа в рамках одной таблицы (одного набора таблиц) осуществлялся с помощью использования представлений, функций и триггеров (что является ресурсозатратным и трудномасштабируемым), или на основе логики приложения (что является небезопасным, так как злоумышленник может обойти логику приложения и воспользоваться всеми правами, которыми обладает приложение). Защита на уровне строк действует на уровне базы данных и применяется для всех без исключения соединений, что позволяет уменьшить пространство для атаки. Механизм защиты определяется для таблицы.

В SQL Server защита на уровне строк осуществляется на основании предикатов безопасности. Реализуется данная защита с помощью средств TRANSACT-SQL. Существуют два вида предикатов: фильтр и блокировка.

Фильтр скрытно от пользователя (прозрачно) осуществляет фильтрацию строк, оставляя в результате запроса только те строки, которые удовлетворяют условиям фильтра. Блокировка запрещает выполнение определённых операций при выполнении условия, и если пользователь наткнётся на блок, то ему будет возвращена ошибка.

Предназначение фильтра заключается в скрытии данных при их чтении, и затрагивает он, соответственно, все операции с чтением: SELECT, UPDATE, DELETE (нельзя обновлять и удалять скрытые строки). Однако пользователь может так обновить доступную ему строку, что на неё начнут действовать условия фильтра и он потеряет к ней доступ. Предназначение блокировки состоит в запрете определённых изменений, и затрагивает она, соответственно, все операции с изменениями: INSERT, UPDATE, DELETE.

У блокировки есть два формата: BEFORE и AFTER. AFTER используется с INSERT и UPDATE, BEFORE – с UPDATE и DELETE. Предикаты с AFTER запрещают пользователю добавлять и изменять значения, попадающие под условие (нарушающие) предиката. Предикаты с BEFORE запрещают пользователю обновлять и удалять значения, которые на данный момент попадают под условие предиката. Особенность работы блока AFTER UPDATE: блок позволяет работать с уже нарушившими условие столбцами. Например, блок запрещает выставлять работнику зарплату выше ста тысяч. Но такой блок разрешит изменять адрес работника, чья зарплата уже превышает сто тысяч. Разделение работы блока UPDATE на BEFORE и AFTER накладывает некоторые ограничения на доступный функционал: например, запретить пользователю с помощью блока изменять значение столбца на превышающее текущее (учитывает и текущее значение, и будущее) нельзя. Подобный функционал рекомендуется реализовывать по старинке триггерами.

При создании политики безопасности являются отключёнными. Отключённые политики никак не влияют на процедуры запросов к таблицам базы данных. Однако возможно создать только одну пару предикат-операция (например, на таблицу может быть лишь один предикат фильтра и по одному предикату блока на каждую форму блока: AFTER INSERT, AFTER UPDATE, BEFORE UPDATE и BEFORE DELETE), причём

не имеет значения, включена ли политика, вызывающая эти предикаты, или отключена. Также невозможно менять связанные с предикатом столбцы базы данных (не затронутые менять разрешено). Предикаты действуют на всех пользователей, на которых заданы действовать, включая владельца базы данных, системного администратора и прочих системных ролей (хотя владелец базы данных всегда может изменить или удалить эти предикаты; что, впрочем, можно аудировать).

Функционально предикаты полностью эквиваленты тому, что доступно при использовании проверок WHERE. Предикаты основываются на исполнении заранее созданной функции. Предикаты могут задействовать внешние таблицы и/или функции при проверке своих условий. При применении предикатов не рекомендуется осуществлять преобразования типов данных, использовать рекурсии и присоединять чрезмерно много внешних таблиц, чтобы обеспечить приемлемую скорость работы предикатов и обойти возможные ошибки (преобразований и бесконечных рекурсий).

Определяется функционал защиты на уровне строк через ключевое слово SECURITY POLICY. Через его CREATE, ALTER и DROP происходит создание, изменение и удаление политики соответственно. SECURITY POLICY определяет, какие предикаты используются в рамках конкретной таблицы. Простейший пример создания защиты на уровне строк:

```
CREATE SECURITY POLICY Security.SalesFilter -- Создаём политику с заданием её названия в схеме Security
    ADD FILTER PREDICATE Security.fn_securitypredicate(AppUserId) -- Добавляем предикат-фильтр на основании указанной функции, передавая ей название проверяемого столбца AppUserId
    ON dbo.Sales, - Определяем таблицу, на которую задаётся предикат
    ADD BLOCK PREDICATE Security.fn_securitypredicate(AppUserId) -- Добавляем предикат-блок на основании той же функции
    ON dbo.Sales AFTER INSERT -- Указываем таблицу и режим оперирования блока
    ADD BLOCK PREDICATE Security.fn_securitypredicate(AppUserId)
    ON dbo.Sales AFTER UPDATE -- Аналогично для изменения
    WITH (STATE = ON); -- Включаем политику при создании
```

Данный пример подразумевает, помимо существования таблицы Sales и схемы Security, наличие созданной функции fn_securitypredicate в рамках схемы Security, которая принимает на вход столбец и осуществляет проверку на основании содержимого ячеек этого столбца (столбцов). Пример создания такой функции:

```
CREATE FUNCTION Security.fn_securitypredicate(@AppUserId int) -- Создаётся функция с указанием названия и столбца с целыми значениями
    RETURNS TABLE -- Функция возвращает таблицу
    WITH SCHEMABINDING -- С привязкой схемы: это означает, что для всех запросов, применяемых в рамках фильтра, пользователю не нужно иметь права доступа к таковым запросам
    AS
    RETURN SELECT 1 AS fn_securitypredicate_result -- возвращает 1 для каждой строки
    WHERE -- Строки фильтруются следующим образом
    DATABASE_PRINCIPAL_ID() = DATABASE_PRINCIPAL_ID('AppUser') -- ID текущего пользователя на уровне базы данных сравнивается с заданным названием: логика проверки в том, что проверяется совпадение имени текущего пользователя с име-
```

нем, используемым приложением, и таким образом только через приложение можно получить доступ к данной таблице

AND CAST(SESSION_CONTEXT(N'UserId') AS int) = @AppUserId; -- Из сессионного контекста по ключевому слову UserId берётся значение и сравнивается со значениями столбца, название которого определяется при вызове функции; если строка в указанном столбце совпадает со значением контекста (по ключу данного контекста очевидно, что оно будет содержать ID пользователя в приложении), фильтр примет такую строку, и для этой строки будет возвращена единица.

Такая функция подразумевает задание сессионного контекста под ключевым словом UserId, иначе фильтр не вернёт ни одной строки. В целом предикат основывает фильтрацию на том, что доступ имеют только пользователи определённого приложения к тем строкам, в которых указан ID этого пользователя в приложении; таковые пользователи не могут просматривать и изменять такие данные (за что отвечает фильтр) и могут вставлять только данные, в которых в столбце имени пользователя будет указано их имя. Блок имеет смысл только для AFTER, потому что BEFORE уже было убрано из доступа с помощью фильтра.

Если в организации не используется приложение, или же приложение не подразумевает использование сессионного контекста и работает вокруг пользовательской системы самой СУБД, необходимо использовать иные средства определения того, какие строки какой пользователь может просматривать. В таком случае зачастую наиболее подходящим способом будет работа вокруг ролей и сопоставление прав доступа пользователя с его принадлежностью к той или иной роли, а также, если пользовательские данные хранятся в СУБД, возможно сопоставление имени пользователя в базе данных с именем текущего пользователя и фильтрация данных на основании этого сопоставления.

В случае работы с ролями основной функцией будет IS_MEMBER, определяющая членство текущего пользователя в указанной роли, и сопоставление этой роли с данными, соответствующими этой роли. Пример функции для работы вокруг ролей (исходные данные: таблица с данными о студентах, в которой есть столбец с названием факультета):

```
CREATE FUNCTION Security.fn_facultypredicate(@faculty AS sysname)
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN SELECT 1 AS fn_securitypredicate_result
WHERE
(@faculty == 'PK' AND IS_MEMBER('RK_decanate')) OR -- Деканат PK считывает данные только студентов с PK
(@faculty == 'PT' AND IS_MEMBER('RT_decanate')) OR
(@faculty == 'ФБ' AND IS_MEMBER('FS_decanate'))
```

В случае работы вокруг сопоставления имени пользователя в базе данных с текущим, или иного сопоставления, работающего с именем текущего пользователя, основной функцией будет USER_NAME(), возвращающей имя текущего пользователя (в базе данных) в виде строки данных. Пример функции для работы вокруг имени текущего пользователя (исходные данные: таблица продаж, в которой имеется столбец с именем пользователя):

```
CREATE FUNCTION Security.fn_securitypredicate(@SalesRep AS sysname)
RETURNS TABLE
WITH SCHEMABINDING
```

```

AS
RETURN SELECT 1 AS fn_securitypredicate_result
WHERE @SalesRep = USER_NAME() OR USER_NAME() = 'Manager'; -- продав-
цы имеют доступ только к своим продажам, а менеджер (пользователь с именем пользо-
вателя Manager) может считывать их все

```

Однако далеко не всегда у разработчиков разделения прав доступа на уровне строк есть возможность изменять структуру таблиц и добавлять туда определяющие столбцы, и им приходится ограничивать права доступа в одной таблице на основании данных, хранящихся в другой. Например,

```

CREATE FUNCTION Security.foo(@bid as sysname)
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN SELECT 1 AS res
WHERE @bid = (SELECT BusinessEntityID
FROM Person.PersonPhone
WHERE PhoneNumberTypeID = 3 AND BusinessEntityID = @bid);

```

Данная функция предназначена для фильтрации таблицы Person.Person, на основании столбца BusinessEntityID (в качестве переменной @bid), и фильтр по этой функции пропустит только данные о тех людях, у которых тип номера телефона является третьим по идентификатору. В таблице Person.Person при этом нет никаких данных о типе номера телефона, эти данные берутся из таблицы Person.PersonPhone, а сопоставление этих таблиц происходит с помощью встроенного запроса (запроса, находящегося в коде функции в скобках).

Возможно также, что данные понадобится взять из более чем одной внешней таблицы, в таком случае в рамках встроенного запроса эти таблицы объединяются с помощью JOIN на основании общего для них столбца. Например,

```

SELECT BusinessEntityID
FROM      HumanResources.Department      JOIN      HumanRe-
sources.EmployeeDepartmentHistory
ON        HumanResources.Department.DepartmentID      =      HumanRe-
sources.EmployeeDepartmentHistory.DepartmentID

```

объединяет таблицы HumanResources.Department и HumanResources.EmployeeDepartmentHistory на основании идентификатора отдела (DepartmentID), что позволяет, например, определить, в каком отделе находится сейчас конкретный работник. В таблице HumanResources.EmployeeDepartmentHistory хранятся данные о смене отдела работниками, но из данных об отделе там содержится только внешний ключ – идентификатор отдела, в то время как в таблице HumanResources.Department эти идентификаторы сопоставляются с параметрами отдела, например, с его названием.

Ход работы

В процессе работы будет выполнена задача соблюдения конфиденциальности данных с помощью разделения прав доступа к данным на уровне строк.

Часто случается так, что к одной и той же таблице базы данных необходимо иметь доступ нескольким лицам, или даже нескольким группам лиц. Однако не каждый из них должен иметь полный доступ ко всей такой таблице. Наиболее простой пример

такой таблицы – таблица продаж: сотруднику следует видеть (и обрабатывать) только те продажи, которые были выполнены им самим, но знать о продажах других сотрудников ему не следует. Подобные примеры можно найти в большинстве предметных областей: деканат имеет доступ к данным только тех студентов, которые числятся на его факультете; начальнику отдела необходимы данные только о тех сотрудниках, которые принадлежат его отделу, и пр. Ранее подобный функционал реализовывался с помощью представлений (каждому лицу/роли представлялось своё представление, которое с помощью какого-либо условия обрабатывало одну или несколько исходных таблиц) и функций по внесению изменений в исходные таблицы при изменении этих представлений, а также за счёт триггеров (функций, запускающихся автоматически при выполнении определённых действий, например, запросов к таблицам). Сейчас же есть возможность непосредственного указания условия на доступ к строкам с помощью соответствующего функционала – защиты на уровне строк.

Рассматриваемая задача разделяется на следующие шаги:

- 1) определение таблиц общего доступа в базе данных;
- 2) определение круга лиц и/или ролей, которые должны иметь доступ к определённой таблице;
- 3) определение прав, которые должны иметь определённые лица и/или роли;
- 4) настройка определённых прав с помощью защиты на уровне строк.

Шаг 1

Откройте базу данных AdventureWorks2017 и найдите в ней таблицу Sales.SalesOrderHeader. Ознакомьтесь со структурой данной таблицы (рисунок 3.1).

	Имя столбца	Тип данных	Разрешить ...
▶	SalesOrderID	int	☐
	RevisionNumber	tinyint	☐
	OrderDate	datetime	☐
	DueDate	datetime	☐
	ShipDate	datetime	☒
	Status	tinyint	☐
	OnlineOrderFlag	Flag:bit	☐
	SalesOrderNumber		☐
	PurchaseOrderNumber	OrderNumber:nvar...	☒
	AccountNumber	AccountNumber:n...	☒
	CustomerID	int	☐
	SalesPersonID	int	☒
	TerritoryID	int	☒
	BillToAddressID	int	☐
	ShipToAddressID	int	☐
	ShipMethodID	int	☐
	CreditCardID	int	☒
	CreditCardApprovalCode	varchar(15)	☒
	CurrencyRateID	int	☒
	SubTotal	money	☐
	TaxAmt	money	☐
	Freight	money	☐
	TotalDue		☐
	Comment	nvarchar(128)	☒
	rowguid	uniqueidentifier	☐
	ModifiedDate	datetime	☐
			☐

Рисунок 3.1 – Таблица Sales.SalesOrderHeader

Шаг 2

Обычно для поддержки защиты на уровне строк используется один из трёх методов:

- 1) создаётся отдельная таблица, сопоставляющая имена пользователей в базе данных с их идентификаторами в базе данных;
- 2) в рамках необходимой таблицы вводится дополнительный столбец, в котором записывается, какой пользователь имеет право доступа к строке (или какая роль имеет право доступа);
- 3) процедура реализуется через внешнее приложение, которое передаёт СУБД через сессионный контекст необходимую информацию (идентификатор пользователя), который сопоставляется с данными в таблице.

Возможны и другие варианты, однако указанные являются наиболее простыми и понятными. Чаще всего встречается последний вариант (в подавляющем большинстве организаций роли и имена входа в СУБД используются на уровне администрирования СУБД, а для доступа к самим данным используется один пользователь – имя входа для приложения, в сессионном контексте которого определяется пользователь приложения), поэтому в ходе работы продемонстрируем именно его.

Создайте имя входа в СУБД: в контекстном меню папки «Безопасность» на уровне соединения выберите «Создать» -> «Вход...». Назовите его app (рисунок 3.2). Создайте пользователя в базе данных AdventureWorks2017: в контекстном меню папки «Безопасность» на уровне базы данных выберите «Создать» -> «Пользователь...». Назовите его своими инициалами ФИО и сопоставьте с именем входа app (рисунок 3.3). Выдайте пользователю все стандартные права доступа (Выборка, Вставка, Обновление, Удаление, которые соответствуют правам на выполнение запросов SELECT, INSERT, UPDATE, DELETE соответственно) на таблицу Sales.SalesOrderHeader, либо через меню «Свойства» и пункт «Разрешения» нужной таблицы (рисунок 3.4), либо через свойства самого пользователя, либо с помощью запроса.

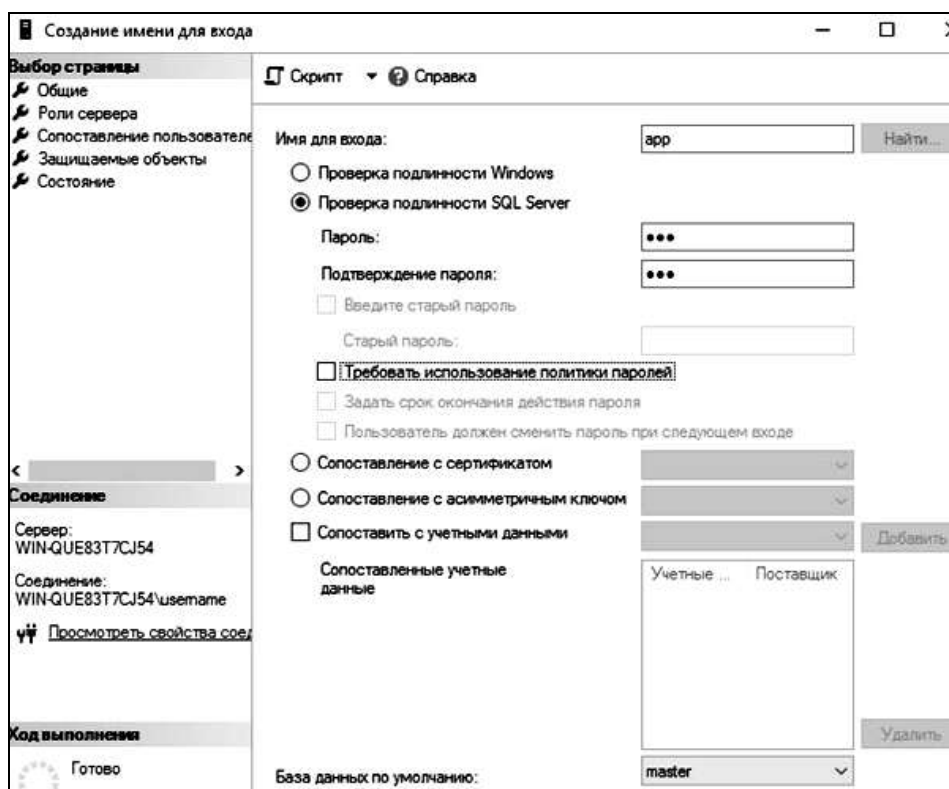


Рисунок 3.2 – Создание имени входа

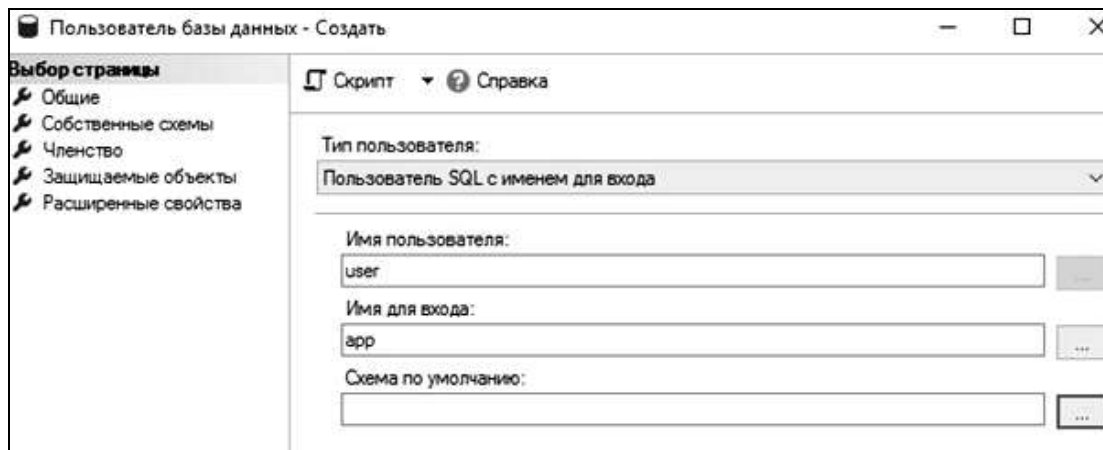


Рисунок 3.3 – Создание пользователя

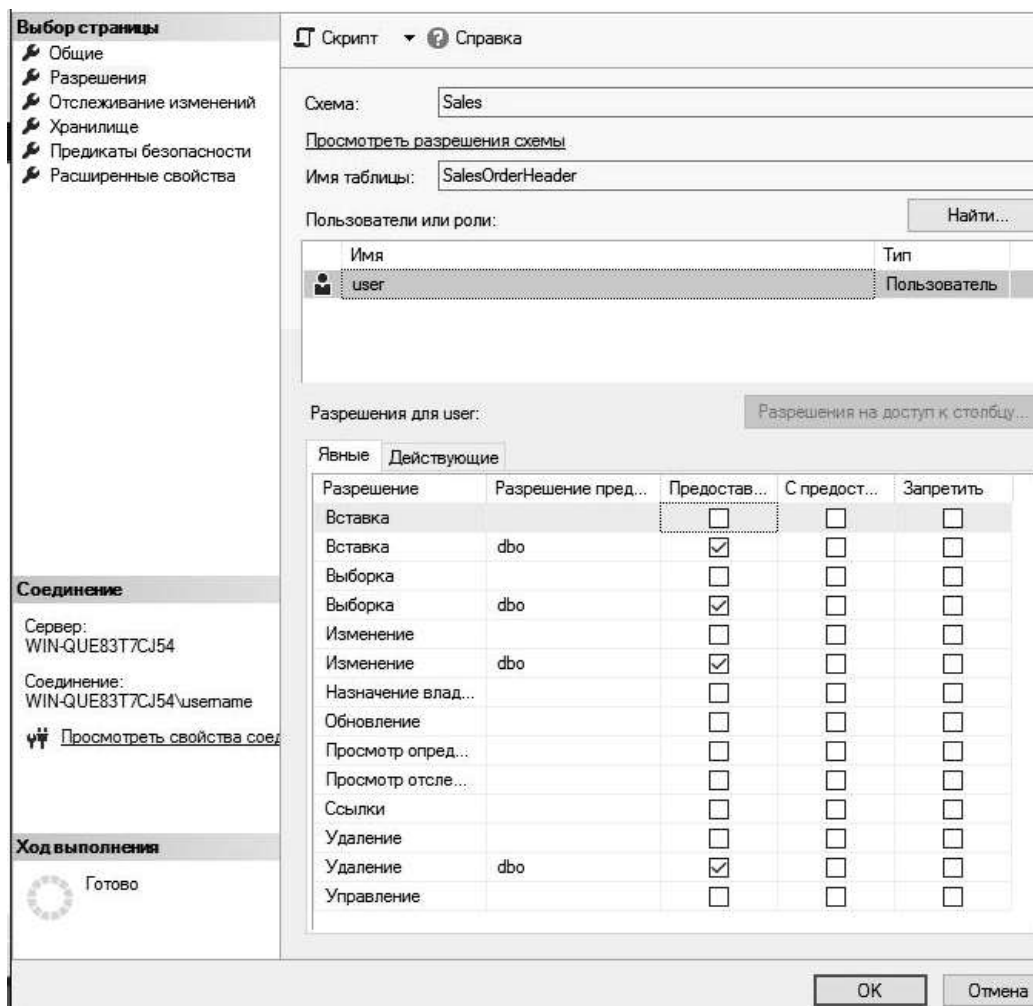


Рисунок 3.4 – Выдача пользователю прав

С точки зрения СУБД, по построенной нами логике работы, пользователь, использующий внешнее приложение, совпадает с тем пользователем, идентификатор которого приложение представляет в СУБД через сессионный контекст (SESSION_CONTEXT). Его идентификатор является целым числом. Посчитаем, что у этого пользователя идентификатор 275.

Шаг 3

Поскольку был выбран третий вариант, права, которые выдаются пользователям, определяются на основании их идентификатора, который передаётся приложением через сессионный контекст. Предназначением выбранной нами таблицы является сбор всей основной связующей информации по конкретным продажам, с указанием такового идентификатора. Соответственно, разграничением доступа на уровне строк будет фильтрация всех тех строк, идентификатор (SalesPersonID) в которых совпадает с идентификатором текущего пользователя, а также запрет на добавление таких строк, или изменение имеющихся строк на такие, или удаление таких строк, которые в столбце SalesPersonID содержат идентификатор, отличный от идентификатора текущего пользователя.

Примечание: хотя блок, казалось бы, не имеет значения, так как со стороны приложения подобный функционал должен реализовываться без участия пользователя (выдаваемые пользователю на просмотр таблицы не должны содержать ключевой информации, поскольку в ней у пользователя отсутствует нужда; она нужна для связи таблиц, а таковой функционал вполне реализуем автоматизированными средствами), однако это создаёт дополнительный слой защиты как от возможных случайных ошибок со стороны пользователей и/или приложения, так и со стороны возможного злоупотребления отсутствием таковой проверки злоумышленником.

Шаг 4

Создайте новый запрос в рамках базы данных AdventureWorks2017. Введите следующий код в запрос (на место [ВАШЕ_ФИО] введите имя созданного ранее пользователя):

```
CREATE FUNCTION Security.spred(@SalesPersonId int) -- Создаётся функция с указанием названия и столбца с целыми значениями в качестве параметра; назвать функцию можно как угодно
```

```
RETURNS TABLE -- Функция возвращает таблицу
```

```
WITH SCHEMABINDING -- С привязкой схемы: это означает, что для всех запросов, применяемых в рамках фильтра, пользователю не нужно иметь права доступа к таковым запросам
```

```
AS
```

```
RETURN SELECT 1 AS result -- возвращает 1 для каждой строки; назвать возвращаемое значение можно как угодно
```

```
WHERE -- Строки фильтруются следующим образом:
```

```
DATABASE_PRINCIPAL_ID() = DATABASE_PRINCIPAL_ID('[ВАШЕ_ФИО]')
```

```
-- ID текущего пользователя на уровне базы данных сравнивается с заданным названием: логика проверки в том, что проверяется совпадение имени текущего пользователя с именем, используемым приложением, и таким образом только через приложение можно получить доступ к данной таблице
```

```
AND CAST(SESSION_CONTEXT(N'UserId') AS int) = @SalesPersonId; -- Из сессионного контекста по ключевому слову UserId берётся значение и сравнивается со значениями столбца, название которого определяется при вызове функции; если строка в указанном столбце совпадает со значением контекста (по ключу данного контекста очевидно, что оно будет содержать ID пользователя в приложении), фильтр примет такую строку.
```

Выполните запрос. Этот запрос создаёт схему Security и в рамках этой схемы определяет функцию безопасности. Создайте и выполните также следующий запрос:

```

CREATE SECURITY POLICY Security.SalesOrderHeader -- Создаём политику с
заданием её названия в схеме Security
    ADD FILTER PREDICATE Security.spred(SalesPersonId) -- Добавляем предикат-
фильтр на основании указанной функции, передавая ей столбец SalesPersonId из табли-
цы Sales.SalesOrderHeader
    ON Sales.SalesOrderHeader, -- Определяем таблицу, на которую задаётся предикат
кат
    ADD BLOCK PREDICATE Security.spred (SalesPersonId) -- Добавляем предикат-
блок на основании той же функции
    ON Sales.SalesOrderHeader AFTER INSERT -- Указываем таблицу и режим опе-
рирования блока
    ADD BLOCK PREDICATE Security.spred(SalesPersonId)
    ON Sales.SalesOrderHeader AFTER UPDATE -- Аналогично для изменения
WITH (STATE = ON); -- Включаем политику при создании

```

Этот запрос создаёт и запускает политику безопасности, которая осуществляет фильтрацию и блокировку на основании определённой ранее функции безопасности.

Для проверки того, что права доступа на уровне строк успешно разделены, выполните любые запросы SELECT, INSERT, UPDATE, DELETE от имени входа app. Сначала присоединитесь к СУБД по заданному имени входа при создании пароля, либо выполните запрос от имени пользователя:

```

REVERT
EXECUTE AS USER = '[ВАШЕ_ФИО]';

```

Затем задайте сессионный контекст, равный идентификатору пользователя:

```
EXEC sp_set_session_context @key=N'UserId', @value=275;
```

Убедитесь, что запрос SELECT * FROM Sales.SalesOrderHeader возвращает только те строки, в которых значение столбца SalesPersonID совпадает с указанным в рамках сессионного контекста, и что остальные запросы с участием этого столбца выполняются только в том случае, если значение соответствующего столбца совпадает с идентификатором пользователя. На рисунке 3.5 видно, что в результате выполнения операции обновления затронуты не все строки, а только 450 (всего в таблице 31465 строк), то есть только те, в которых SalesPersonID равен числу 275.



Рисунок 3.5 – Результат выполнения запроса

Обратите внимание, что если сейчас попробовать взаимодействовать с этой таблицей от имени суперпользователя (под которым был выполнен изначальный вход), то ничего не получится, так как политика безопасности действует на всех пользователей

без исключения. Однако пользователи с правами на работу с политиками безопасности могут их включать и отключать, таким образом, за ними остаётся полноценный функционал.

Контрольные вопросы

- 1) Что есть защита на уровне строк? В чём её предназначение?
- 2) Какие два вида предикатов защиты на уровне строк существуют в SQL Server? Как они работают?
- 3) Какие операции затрагивает каждый из видов предикатов?
- 4) Какие есть форматы у блока? В чём их смысл? С какими операциями они используются?
- 5) Какая команда включает и отключает предикаты? На чём (каком функционале) основаны предикаты?

Задание

В рамках данной лабораторной работы задание является продолжением задания предыдущей лабораторной работы. Используйте тот же вариант, что и в предыдущей лабораторной работе.

Определите для как минимум одной (наиболее подходящей по смыслу в рамках варианта) определённой во втором задании предыдущей лабораторной работы таблицы разделение прав доступа на уровне строк. Напишите защиту на уровне строк согласно разделению, указанному в варианте этой лабораторной работы. Для задания указанного параметра обязательно воспользуйтесь пользовательским контекстом или иным свойством пользователя (имя пользователя, членство в ролях); защита на уровне строк должна определять разным пользователям (с разным контекстом) различное содержимое таблицы. Указывая различные контексты, убедитесь в работоспособности защиты на уровне строк.

Варианты задания для организации защиты на уровне строк:

- 1) учёт данных только сотрудников определённого отдела (параметр: название отдела);
- 2) работа только с сотрудниками на испытательном сроке (сотрудником на испытательном сроке считается сотрудник, у которого указан код бизнес-сущности, но который при этом является кандидатом);
- 3) работа в рамках своего отдела (параметр: название отдела);
- 4) только данные о себе (параметр: код сотрудника);
- 5) документация по продукции (параметр: язык документа);
- 6) только данные о себе (параметр: код покупателя);
- 7) предложения только своего региона (параметр: название региона (локация));
- 8) работа с определённым типом велосипедов (параметр: название категории велосипеда);
- 9) работа в рамках своего отдела (параметр: название отдела);
- 10) работа с определённым типом велосипедов (параметр: название категории велосипеда).

Список вариантов предыдущей лабораторной работы (для удобства):

- 1) бухгалтер;
- 2) ведущий собеседование по найму на работу;
- 3) сотрудник отдела кадров;
- 4) продавец;
- 5) покупатель на сайте организации (незарегистрированный);

- 6) покупатель на сайте организации (зарегистрированный, права отдельно от незарегистрированного);
- 7) дизайнер онлайн магазина;
- 8) разработчик новой продукции;
- 9) кассир (выдаёт зарплату сотрудникам);
- 10) производитель продукции.

Подсказка: можете обратиться к коду, расположенному в дополнительном задании, для примера, совместно с примерами, уже приведёнными в рамках текста лабораторной работы.

Дополнительное задание

Рассмотрите приведённый пример кода для разделения доступа на уровне строк. Данный код записан в файле, расположенном на виртуальной машине, по адресу C:/RLS_code.txt. В нём распределяются права доступа к таблице Sales.SalesOrderHeader для пяти ролей: покупатель, продавец, перевозчик, поддержка покупателей и обработчик онлайн заказов (возможно, автоматизированный). Подразумеваемый спектр прав приводится далее.

- Покупатель – считывание информации по своему заказу, отмена заказа;
- Продавец – работа со своими заказами, создание заказов от покупателей в режиме оффлайн;
- Перевозчик – считывание информации по товару, который необходимо ему перевезти, подтверждение в получении или отказе в получении товара;
- Поддержка покупателей – обработка заказов, которые были отклонены или отменены;
- Обработчик онлайн заказов – создание и обновление заказов в режиме онлайн (по сути, посредник между онлайн покупателем и БД).

Никто не может удалять заказы, так как заказы, которые были по тем или иным причинам не выполнены, остаются в БД в статусе отменённых или отклонённых.

Определите в коде защиты на уровне строк все случаи некорректной выдачи прав доступа. Данный код является синтаксически корректным (если предполагать, что существуют все объекты, на который он ссылается, включая схему Security и указанные роли, а также существование в БД AdventureWorks2017), поэтому в нём нет ошибок написания, только недоработки распределения доступа. Все найденные недоработки обоснуйте. Исправьте этот код, установив верные права доступа. В недоработки не включается недостаток или превышение ролевого распределения (все роли, приведённые в коде, не должны меняться). Создайте окружение для распределения доступа (роли и пользователи для них) с использованием кода из вышеуказанного файла. Продемонстрируйте на нескольких примерах разницу между приведённым разделением доступа и исправленным. **Подсказка:** всего в коде предусмотрено 4 недоработки разделения прав доступа.

Код распределяющих функций и самого предиката также расположен в вышеуказанном файле (C:/RLS_code.txt).

ЛАБОРАТОРНАЯ РАБОТА № 4

Протокол уровня защищённых сокетов (SSL protocol)

Цель работы – изучение принципов работы протокола уровня защищённых сокетов (Secure Socket Layers, SSL) на примере СУБД Microsoft SQL Server.

Теоретический материал

Протокол уровня защищённых сокетов (SSL) – криптографический протокол, предназначенный для создания конфиденциального канала связи. Протокол был создан компанией Netscape Communications и применён в конце девяностых годов в рамках веб-браузера Netscape Navigator для создания защищённого HTTP-соединения (HTTPS). На данный момент доказано, что этот протокол является уязвимым для атак. Для устранения уязвимостей был разработан протокол TLS (Transport Layer Security, защита на транспортном уровне), который сейчас находится в практическом использовании. Часто название SSL упоминается в публичных источниках информации как устоявшееся для защищённых соединений, хотя за этим названием стоит реализация TLS. Подавляющее большинство современных браузеров поддерживают протокол TLS как минимум версии 1.0. TLS в целом основан на тех же принципах, что и SSL, предназначен абсолютно для тех же целей, что и SSL, и является просто «работой над ошибками» SSL.

Протокол SSL/TLS основан на двух основных процедурах: протокол «рукопожатия» (handshake) и протокол записи. В ходе первого протокола сервер и клиент устанавливают соединение, проверяют валидность (иногда друг друга, иногда только сервера, изредка сохраняется полная анонимность) и настраивают сеансовый ключ. В ходе протокола записи идёт передача сообщений. Протокол записи достаточно прост: на основе определённого сеансового ключа (который является симметричным) идёт шифрование данных, передача их другой стороне и расшифрование данных при получении.

Протокол «рукопожатия», в общем виде, включает в себя следующие основные шаги:

- 1) клиент подключается к серверу с запросом на защищённое соединение, предоставляя список поддерживаемых алгоритмов шифрования;
- 2) сервер выбирает алгоритм из списка (наиболее надёжный и при этом доступный на сервере) и отправляет решение клиенту, вместе с сертификатом для аутентификации себя;
- 3) клиент проверяет валидность полученного сертификата (путём обращения к указанному в сертификате центру сертификации и сверкой центра сертификации со списком доверенных центров);
- 4) клиент и сервер создают общий сеансовый ключ (на основе, например, протокола Диффи-Хеллмана).

Соответственно, для создания SSL-соединения необходимы удостоверяющий сертификат сервера на сервере и наличие сертификата удостоверяющего центра сервера в списке доверенных центров сертификации у клиента.

Ход работы

В процессе работы будет выполнена задача соблюдения конфиденциальности данных в процессе передачи данных по открытым каналам связи с задействованием протокола SSL.

Обеспечение защищённого канала связи – непростая задача. Технически защищённый канал связи экономически оправдан только в рамках внутренней сети организации, но за её пределами проводить отдельную защищённую линию связи крайне сложно.

Для защиты открытых соединений, проходящих через общую сеть, от перехвата используются криптографические протоколы, превращающие открытый текст в зашифрованный и обеспечивающие уверенность в том, что соединение происходит с доверенным сервером. Одним из таких протоколов является SSL (TSL).

Рассматриваемая задача разделяется на следующие шаги:

- 1) получение сертификата, отвечающего требованиям;
- 2) введение сертификата в окружение сервера и включение режима шифрования;
- 3) проверка настроенного режима.

Шаг 1

SQL Server реализует протокол SSL с помощью сертификатов асимметричного шифрования. При установке сертификата на сервере необходимо, чтобы сертификат удовлетворял следующим требованиям:

- время сертификата валидно (системное время компьютера находится между значениями времени валидности сертификата);
- сертификат выдан субъекту, название которого полностью совпадает с полным доменным именем сервера или именем компьютера-сервера;
- использование ключа сертификата включает в себя аутентификацию сервера;
- ключ допускает обмен и экспорт (создан с параметром AT_KEYEXCHANGE);
- в сертификате имеется закрытый ключ.

Данные требования не включают в себя указание вида сертификата – возможно использование любого из них. В рамках данной лабораторной работы будет создан и использован самоподписанный сертификат, однако следует понимать, что такой сертификат допустимо использовать только тогда, когда нет необходимости в установлении доверенного соединения с сервером, а это только два случая: тестирование сервера и использование протокола в доверенной сети (внутренняя сеть организации, например).

Создать сертификат возможно встроенными средствами Microsoft Windows. Ранее для этого использовалось приложение MakeCert, однако начиная с Windows 10 (Windows Server 2016) функционал по генерации сертификатов входит в расширенную командную строку Windows PowerShell.

Для начала работы запустите Windows PowerShell, обязательно от имени администратора системы. Введите три следующие команды:

```
New-SelfSignedCertificate -DnsName WIN-8PB5C78B03Q -KeySpec KeyExchange  
$Password = ConvertTo-SecureString -String "12345" -Force -AsPlainText  
Export-PfxCertificate -Cert cert:\LocalMachine\My\[Отпечаток_сертификата] -  
FilePath [Путь_для_сохранения] -Password $Password
```

Первая команда создаёт самоподписанный сертификат без закрытого ключа и со спецификацией ключа на обмен (остальные требования к сертификату SQL Server уже удовлетворены за счёт настроек по умолчанию) с именем виртуальной машины, вторая записывает в переменную Password пароль 12345 в формате защищённой строки (Secure String), третья экспортирует созданный сертификат по указанному пути (не забудьте указать название сертификата и расширение .pfx) с созданием закрытого ключа, защищённого заданным паролем.

Отпечаток сертификата будет выведен в консоль (под словом Thumbprint), однако, чтобы не переписывать его весь, можно открыть созданный сертификат в хранилище сертификатов и скопировать отпечаток из его состава. Чтобы открыть хранилище, с помощью окна запуска Win+R запустите mmc. Откроется окно оснастка, добавьте оснастку

«Сертификаты», при определении хранилища выберите «Учётная запись компьютера», далее «Локальный компьютер». Откроется хранилище сертификатов. В хранилище найдите созданный сертификат – он будет в папке «Промежуточные корневые центры сертификации», подпапке «Сертификаты». Откройте его (двойным кликом ЛКМ) и на вкладке «Состав» найдите параметр «Отпечаток» (рисунок 4.1), который можно скопировать. Необходимо будет удалить пробелы из отпечатка после вставки, но эта процедура гарантирует, что вы не ошибётесь при переписывании отпечатка из консоли вручную.

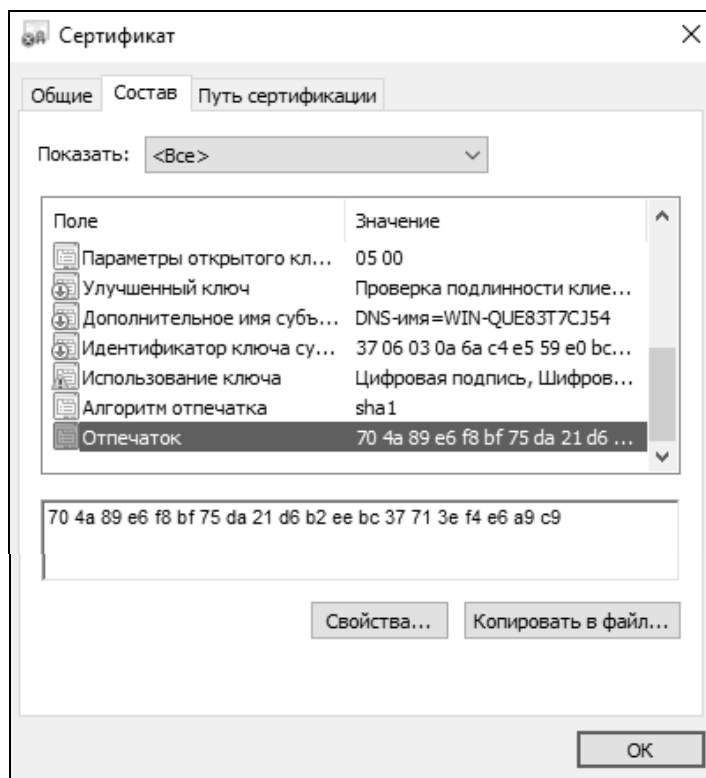


Рисунок 4.1 – Отпечаток сертификата

Шаг 2

Итак, у нас есть сертификат с закрытым ключом. Теперь необходимо импортировать его в хранилище. Для этого просто его запустите, и откроется Мастер импорта сертификатов (рисунок 4.2).

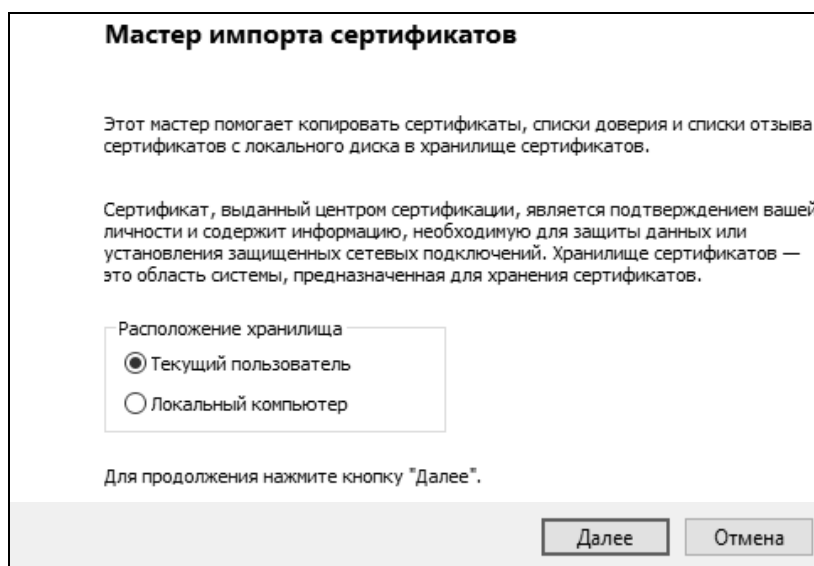


Рисунок 4.2 – Мастер импорта сертификатов

В первом окне Мастера укажите в качестве хранилища локальный компьютер. Второе окно оставьте без изменений. В третьем окне введите пароль, указанный при определении переменной пароля (12345 в примере) (рисунок 4.3).

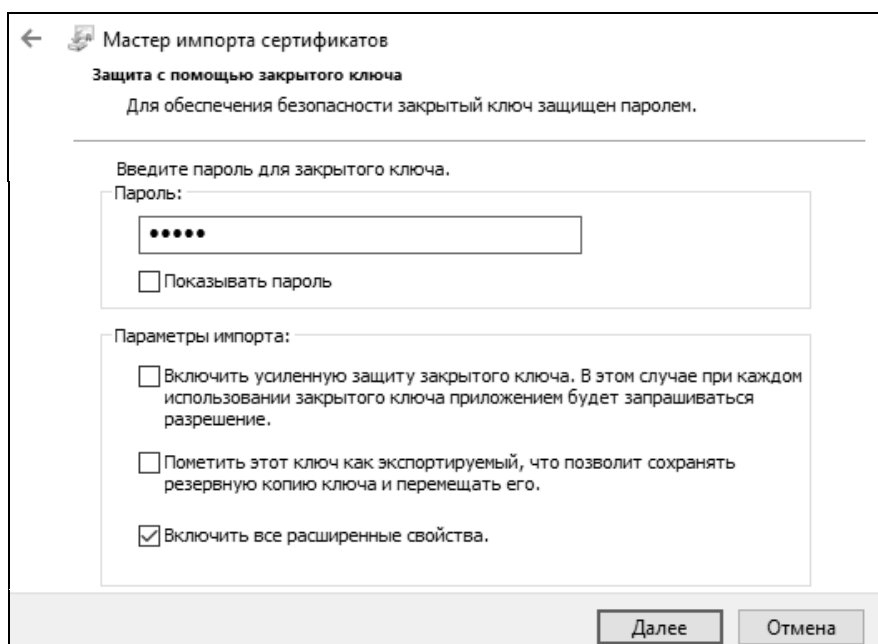


Рисунок 4.3 – Ввод пароля

Остальные параметры в третьем окне оставьте без изменений.

В четвёртом окне Мастер предложит указать папку (хранилище), куда будет положен сертификат. По умолчанию сертификат будет положен в папку «Личное» – именно там он и должен находиться.

Теперь необходимо выдать доступ службе Microsoft SQL Server к этому сертификату. Для этого, находясь в окне хранилища сертификатов, в контекстном меню этого сертификата (который должен быть в папке «Личное») выберите последовательно пункты «Все задачи» -> «Управление закрытыми ключами». В открывшемся окне укажите пользователя NT Service/MSSQL\$SQLSERVER и определите права на полный доступ. Если пункта «Управление закрытыми ключами» нет, значит, ключ находится в хранилище текущего пользователя, и его необходимо поместить в хранилище локального компьютера.

Чтобы сертификат мог быть определён клиентом как допустимый, необходимо положить сертификат удостоверяющего центра (в нашем случае сертификат самоподписанный, следовательно, это будет тот же самый сертификат) в хранилище «Доверенные корневые центры сертификации». Созданный нами изначально сертификат, который находится в папке «Промежуточные центры сертификации», содержащий только открытый ключ, как раз является таким сертификатом. Необходимо перенести этот сертификат в папку доверенных центров в окне хранилища сертификатов.

Теперь в хранилище есть два созданных сертификата – убедитесь, что у сертификата в папке доверенных центров нет закрытого ключа, а в личной папке – есть, что отображается на вкладке «Общие» в окне сертификата (рисунок 4.4 и рисунок 4.5).

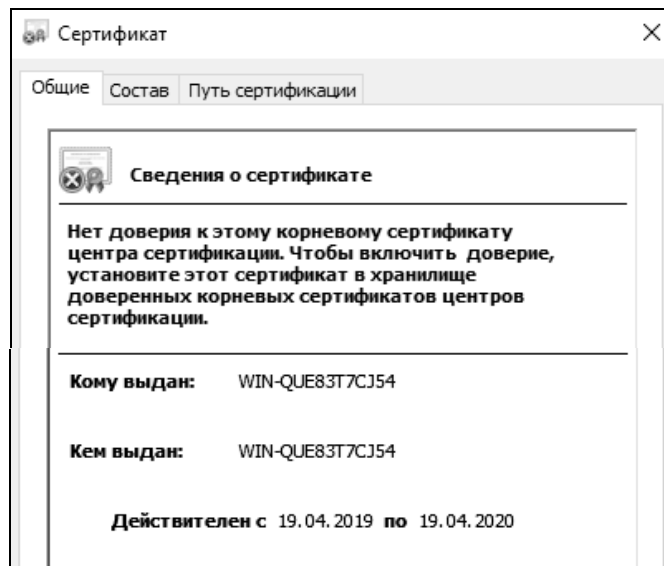


Рисунок 4.4 – Сертификат без закрытого ключа

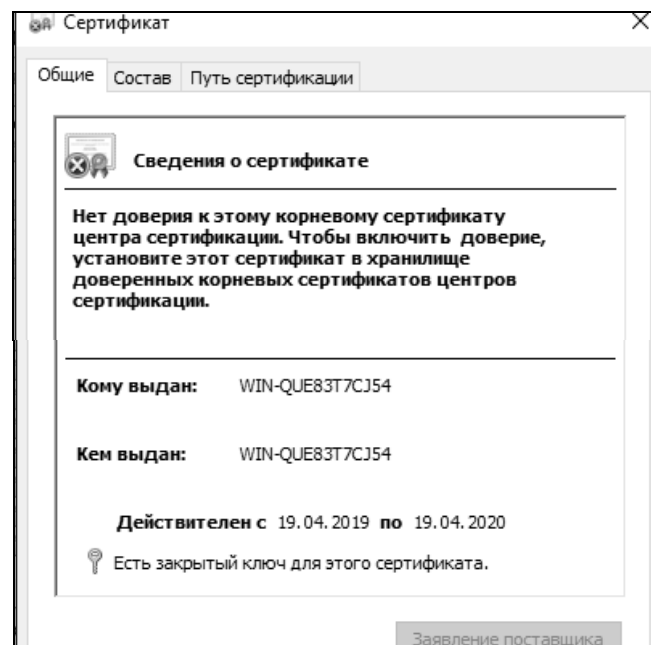


Рисунок 4.5 – Сертификат с закрытым ключом

Итак, нужные сертификаты добавлены в хранилище. Теперь необходимо указать SQL Server, какой именно сертификат следует использовать для шифрования соединений. Чтобы это сделать, откройте Диспетчер конфигурации SQL Server и с помощью контекстного меню войдите в свойства вкладки «Протоколы для MSSQLSERVER» (рисунок 4.6). В окне свойств на вкладке «Флаги» включите «Принудительное шифрование», а на вкладке «Сертификат» – укажите добавленный сертификат. Если сертификат не отображается для выбора, где-то была допущена ошибка в создании и/или добавлении.

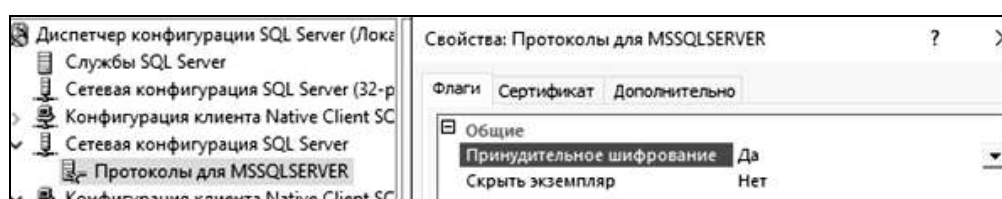


Рисунок 4.6 – Свойства протоколов

Шаг 3

Чтобы проверить соединение, необходимо от хоста (основной машины) или другой виртуальной машины, с настроенным соединением с виртуальной машиной-сервером, присоединиться к серверу с помощью SSMS, или иному приложению, с помощью которого такое соединение возможно. Для организации сетевого соединения между виртуальной и основной машинами необходимо установить тип соединения в настройках виртуальной машины на «Сетевой мост» либо «NAT».

В данной лабораторной работе покажем выполнение соединения с базой на примере SSMS.

В первую очередь необходимо добавить сертификат доверенного центра сертификации сертификата сервера (поскольку это самоподписанный сертификат, это опять тот же самый созданный нами сертификат) в хранилище сертификатов на стороне клиента. Для этого следует экспортировать сертификат с сервера на клиент и добавить его в хранилище. Для этого достаточно сертификата с открытым ключом (без закрытого), но можно взять и сертификат с закрытым ключом. Добавляется сертификат в хранилище так же, как и на стороне сервера, однако обратите внимание, что сертификат необходимо добавить в папку доверенных сертификатов (рисунок 4.7).

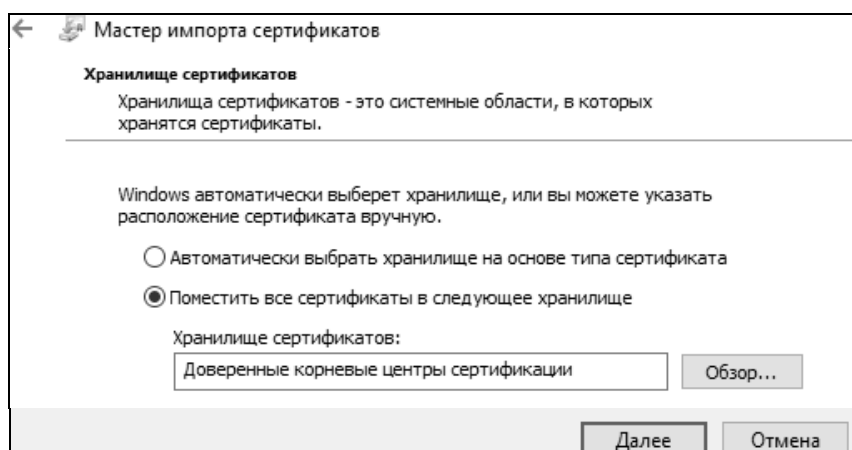


Рисунок 4.7 – Определение хранилища для сертификата

Теперь необходимо соединиться с сервером. Для этого, при подключении к базе данных, в качестве имени сервера используйте «tcp:WIN-QUE83T7CJ54», что означает подключение к машине с указанным именем по протоколу TCP (рисунок 4.8). Поскольку клиент не добавлен в список пользователей СУБД, воспользуйтесь учётной записью sa (пароли указаны в файле «Пароли» на рабочем столе сервера) или иной другой учётной записью с входом по логину/паролю.

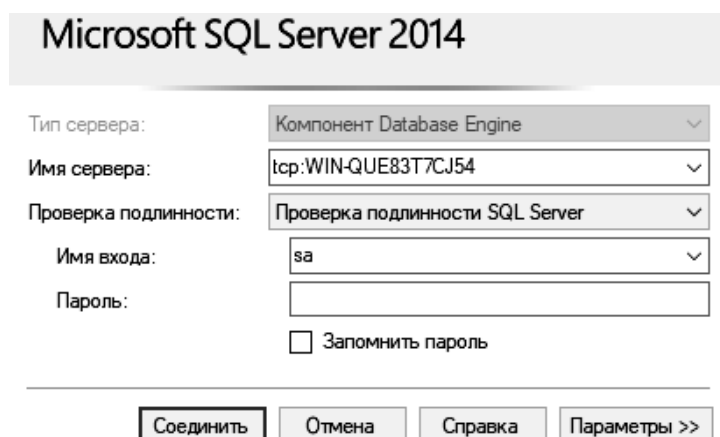


Рисунок 4.8 – Соединение с сервером

В зависимости от настроек на сервере, клиент может подключаться как с защищённым соединением, так и без него. Указанный ранее параметр «Принудительное шифрование» уже настроил обязательное шифрование со стороны сервера. Чтобы со стороны клиента использовать именно зашифрованное соединение, следует открыть «Параметры >>» в окне соединения и на вкладке «Свойства соединения» отметить пункт «Шифровать соединение» (рисунок 4.9). Убедитесь, что удаётся подключиться к серверу с использованием SSL.

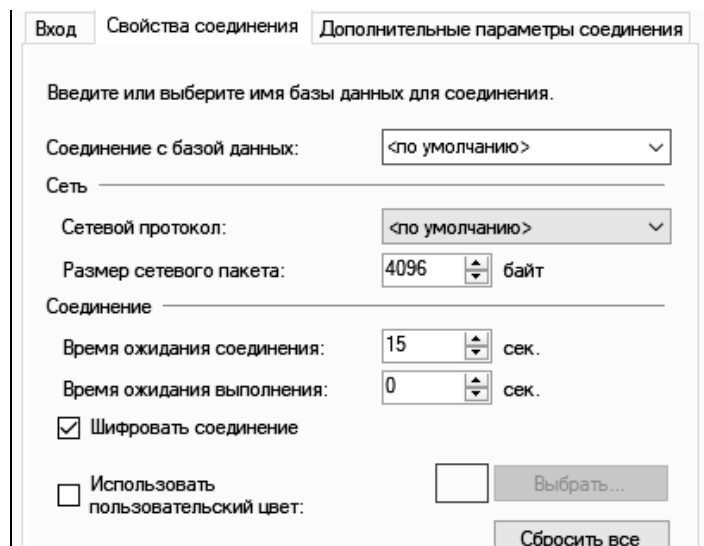


Рисунок 4.9 – Принудительное применение шифрования со стороны клиента

Контрольные вопросы

- 1) Как расшифровывается SSL и TLS? Что такое SSL и в чём принципиальное отличие от TLS?
- 2) Какие протоколы (фазы) используются в процессе реализации SSL? Кто участвует в SSL-соединении?
- 3) Опишите шаги первого протокола (фазы) в SSL.
- 4) Что необходимо для создания SSL-соединения?
- 5) Какие существуют виды сертификатов? Чем они отличаются друг от друга? Для чего используются?
- 6) Какие требования предъявляются к сертификатам в SQL Server?

Задание

Выполните «Ход работы» на виртуальной машине. Составьте отчёт по проделанной работе и продемонстрируйте отчёт преподавателю.

Дополнительное задание

Используя любой анализатор трафика (сниффер), например, «Wireshark», продемонстрируйте, что без использования SSL-соединения данные передаются в открытой форме, а с ним они зашифрованы. Учтите, что могут передаваться данные, зашифрованные в рамках первой лабораторной работы, поэтому передавайте незашифрованные данные или отмените режим шифрования.

ЛАБОРАТОРНАЯ РАБОТА № 5

Оценка уязвимостей и классификация данных

Целью работы является изучение принципов работы автоматизированных оценщиков уязвимостей и классификаторов данных на примере соответствующих инструментов в SQL Server.

Теоретический материал

Инструмент оценки уязвимостей (Vulnerability Assessment) в SQL Server основывается на наборе предопределённых правил. Правило представляет собой рекомендацию от корпорации Microsoft по соблюдению информационной безопасности. Структурно правило состоит из следующих компонентов:

- номер (ID);
- название;
- уровень угрозы (риска) (высокий, средний, низкий);
- статус (проверка пройдена или провалена);
- описание правила;
- влияние угрозы в правиле;
- запрос – обычный запрос на чтение (SELECT) к базе данных, по результатам которого и определяется, соблюдено ли правило или нет;
- рекомендация от Microsoft к правилу;
- фактический результат запроса;
- решение;
- скрипт решения (если доступен).

При выполнении проверки определяется, какие правила из всего имеющегося перечня соблюдаются. У правил существует понятие базового показателя – значения, которое считается нормой; нарушением правила будет отклонение от такого значения. Примером правила, которое основано на базовом показателе, является правило с ID VA1281, которое заключается в проверке того, что все членства пользователей в пользовательских ролях учтены в этом базовом показателе – такое правило позволяет динамически отслеживать изменение состава ролей в базе данных.

Всего таких проверок 56 на обычные базы данных. Также присутствует оценка уязвимостей на системные базы данных, на каждую из которых прописан уникальный набор проверок (кроме tempdb, на которую проверок нет), хотя многие правила повторяются.

Средства классификации данных предназначены для обозначения уязвимой, критичной и прочей важной информации в рамках баз данных. Такие средства чаще всего используются в совокупности с другими средствами, предлагая применить те или иные меры на классифицированные так или иначе столбцы. Правда, эти средства в MS SQL Server таких предложений не делают. Как и средства поиска уязвимостей, классификация данных предназначена больше для начинающего администратора безопасности, но также полезна в рамках больших баз данных, когда необходимо быстро определить потенциально уязвимые столбцы, а времени на ознакомление со всем бизнес-процессом и, соответственно, ручного определения таких данных не хватает. Также это средство, как и оценка уязвимостей, очень полезно, когда окружение (структура баз данных) меняется динамически и довольно быстро. В целом, они помогают легко построить основу, по которой можно быстро принять решение о принятии базовых мер по информационной безопасности данных и сформировать итоговый отчёт по выполненным мерам.

Одно из правил оценки уязвимостей, а именно правило VA1285, предлагает идентифицировать все столбцы с конфиденциальной информацией. Это напрямую связывает

оценку уязвимостей с классификацией данных. Обнаружение и классификация данных (Data discovery and classification) – инструмент, предназначенный для пометки уязвимых столбцов для более удобной работы с такими столбцами. SQL Server позволяет автоматически определить столбцы с уязвимыми данными и затем в ручном режиме подтвердить/отказать и дополнить построенную классификацию. Столбцы помечаются различными метками (персональные данные, кредитная карта, финансы, аутентификационная информация и пр.). После определения уязвимых столбцов и их меток инструмент сформирует отчёт о классификации данных, с использованием круговых диаграмм и с перечислением уязвимых данных.

Ход работы

В процессе работы будет выполнена задача обеспечения безопасности данных путём применения автоматизированных средств оценки уязвимостей и классификации данных.

В различных окружениях существуют разнообразные средства автоматизации. В рамках СУБД и обеспечения информационной безопасности также существуют средства, позволяющие так или иначе облегчить процесс защиты данных. Такие средства помогают познакомить пользователя с имеющимися в СУБД средствами защиты, обнаружить типичные уязвимости, присутствующие в установке по умолчанию, указать на неверные способы выдачи прав доступа, определить содержащие потенциально конфиденциальную информацию столбцы, и прочее. Однако никакое средство автоматизации поиска уязвимостей не обнаружит все реально существующие уязвимости, находящиеся в их окружении; каждое из них ограничено тем кругом поиска, на который их настроили создатели. И совершенно точно такие средства не могут самостоятельно, без обращения к внешним источникам, обнаруживать программные уязвимости. Однако в рамках большого количества крупных баз данных они позволят осуществлять быструю проверку на применение самых основных мер безопасности в каждой из них, а также помогут обнаружить те уязвимости, которые можно обнаружить автоматизированным путём, но сложно обнаружить человеку, например, некоторые преувеличения разделения прав доступа.

Рассматриваемая задача разделяется на следующие шаги:

- 1) запуск автоматизированного средства оценки уязвимостей;
- 2) оценка полученного списка проблем и определение мер по устранению;
- 3) принятие определённых мер;
- 4) составление списка информации критичной важности.

Шаг 1

Оценка уязвимостей в SQL Server происходит на уровне базы данных. Чтобы запустить оценку уязвимостей, следует в контекстном меню базы данных выбрать «Задачи» -> «Оценка уязвимостей» -> «Проверить на наличие уязвимостей...». Откроется окно выбора места сохранения результатов оценки, можно оставить значение по умолчанию. После выбора этого места будет произведена оценка, и затем откроется окно результатов оценки (рисунок 5.1).

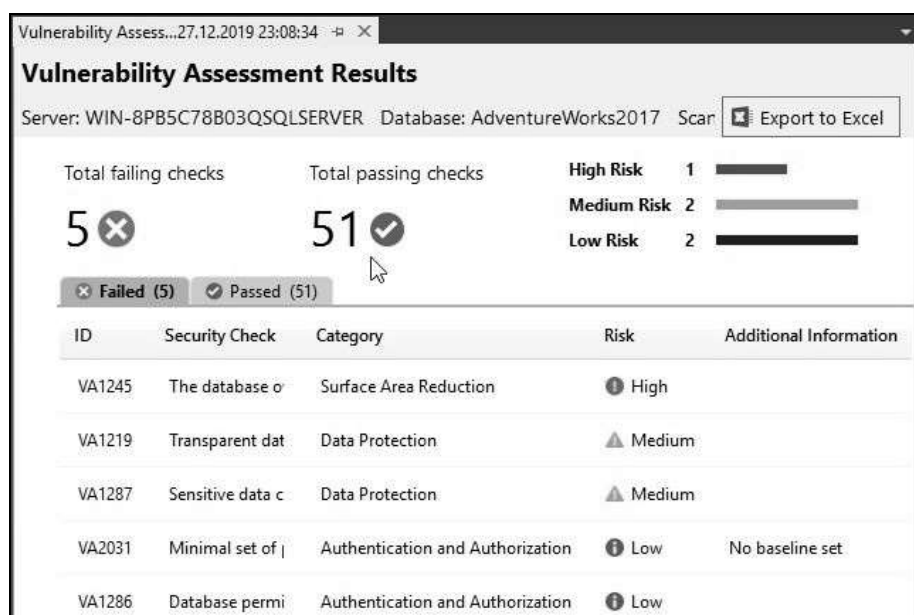


Рисунок 5.1 – Результат оценки

Шаг 2

Некоторые из правил будут провалены (failed), многие же останутся выполненными (passed). Точный список зависит от того, что и как было выполнено в предыдущих лабораторных работах. Обратимся к одному из правил, а именно правилу с ID VA1286 (рисунок 5.2).



Рисунок 5.2 – Описание правила

После прочтения правила, становится понятно, о чём оно, что проверяет, какую проблему затрагивает и какое решение предлагает. В случае этого конкретного правила проверяется, что разрешения на объекты и столбцы не выдаются непосредственно пользователям. Оно не несёт большого риска. Такой случай считается небезопасным не сам по себе, а потому что в реальных организациях обязанности конкретных людей меняются постоянно; соответственно, меняются и их права доступа в СУБД. Чтобы более цен-

трализованно распределять права, рекомендуется выдавать доступ ролям, а не конкретным людям, за чем, в числе других, следит это правило. Рекомендуется отозвать эти права, для чего уже подготовлен скрипт в пункте Remedation Script. Скрипт можно открыть в окне запроса нажатием соответствующей кнопки снизу. Также можно принять текущее состояние как основу, нажав на кнопку Approve as Baseline сверху. Потребуется подтвердить своё решение, прежде чем базовый показатель будет принят. Чтобы увидеть изменения, следует выполнить оценку уязвимостей по новой. Впоследствии у этого правила будет приписка в столбце Additional information о том, что к нему задан пользовательский базовый показатель.

Шаг 3

Рассмотрев таким образом все правила, примите меры к решению проблем. Если Вы соглашаетесь с предлагаемым решением, Вы его одобряете, принимая предлагаемые изменения или же SQL-запросы, или же реализуете самостоятельно.

Шаг 4

Одно из правил наверняка провалено, а именно правило с ID VA1287. Это правило строится вокруг классификации данных, и его суть проста – критически важные данные должны быть классифицированы. В рамках оценки уязвимостей будет предложен набор автоматически выбранных (на основании названия) столбцов. Чтобы открыть средство классификации данных в отдельном окне, следует выбрать его в контекстном меню нужной базы данных ("Задачи" -> "Обнаружение и классификация данных" -> "Классификация данных..."). Откроется окно классификации (рисунок 5.3).

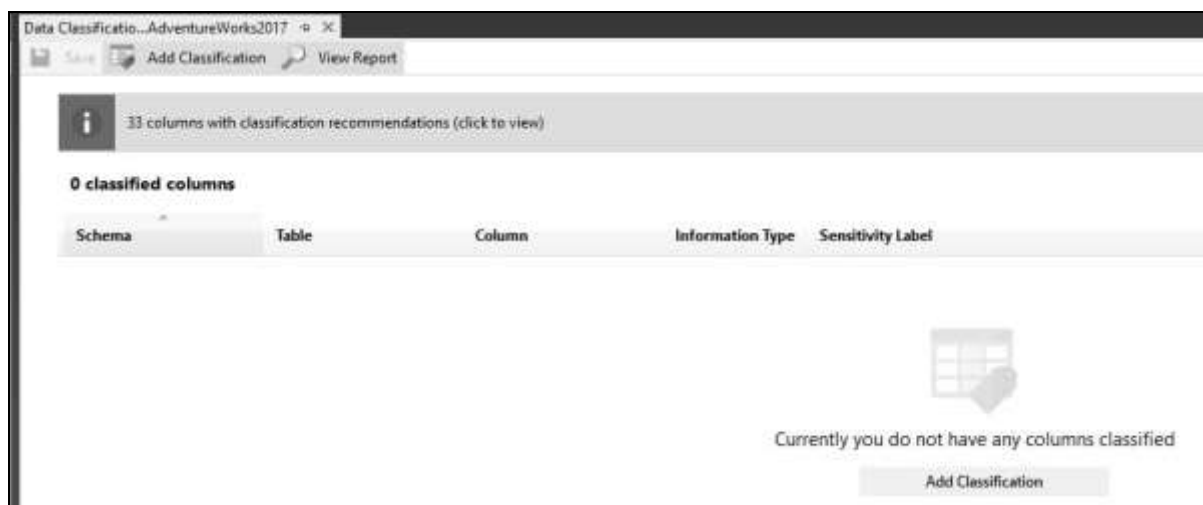


Рисунок 5.3 – Классификация данных

Изначально классифицированных данных нет, однако СУБД предлагает автоматически найденные столбцы, которых в рамках БД AdventureWorks2017 оказывается 33. Просмотрите предложенную классификацию, кликнув по окну с сообщением (рисунок 5.4).

33 columns with classification recommendations (click to minimize)

Accept selected recommendations

☐	Schema	Table	Column	Information Type	Sensitivity Label
☐	dbo	ErrorLog	UserName	Credentials	Confidential
☐	HumanResources	Employee	BirthDate	Date Of Birth	Confidential - GDPR
☐	HumanResources	Employee	NationalIDNumber	National ID	Confidential - GDPR
☐	Person	Address	AddressLine1	Contact Info	Confidential
☐	Person	Address	AddressLine2	Contact Info	Confidential
☐	Person	Address	City	Contact Info	Confidential
☐	Person	Address	PostalCode	Contact Info	Confidential
☐	Person	EmailAddress	EmailAddress	Contact Info	Confidential
☐	Person	Password	PasswordHash	Credentials	Confidential

Рисунок 5.4 – Предложенная классификация

Каждая классификация состоит из следующих частей: схема, таблица, столбец, тип информации, метка уязвимости. Тип информации относится к тому, чем является информация (дата рождения, персональные данные, контактные данные, финансовые данные и т.д.), меткой уязвимости определяется, насколько информация критична (публичная, общая, конфиденциальная, персональные данные (GDPR – General Data Protection Regulation, постановление Европейского союза, регулирующее защиту персональных данных), высоко конфиденциальная, высоко конфиденциальные ПД и без категории). На основании сгенерированной классификации можно поменять тип информации и метку уязвимости, удалить лишние классификации, а также добавить новые с помощью меню справа (рисунок 5.5).

Add Classification

Data Classification

Schema
HumanResources

Table
EmployeePayHistory

Column
Rate

Information Type
Financial

Sensitivity Label
Confidential - GDPR

Рисунок 5.5 – Добавление классификации

После завершения работы над классификацией необходимо сохранить изменения. Далее производится генерация отчёта, в котором будет отмечено на диаграммах, сколько было сделано классификаций и по каким категориям они распределены, а также представлена таблица с классификациями (рисунок 5.6).

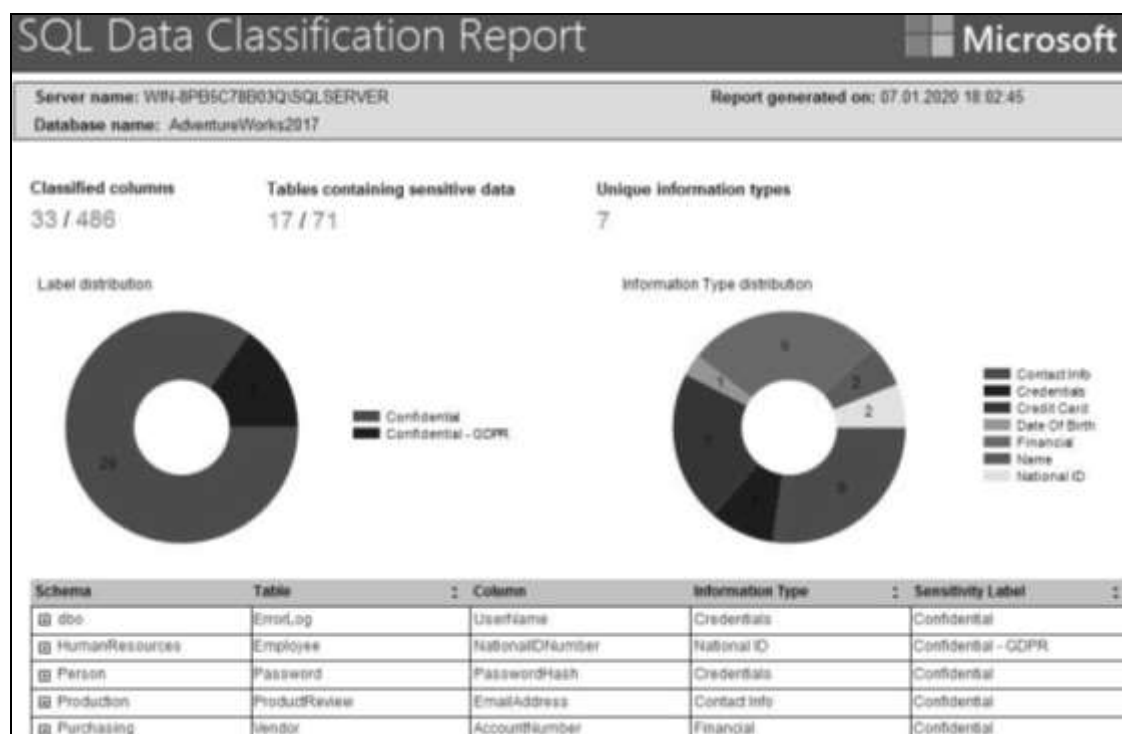


Рисунок 5.6 – Отчёт по классификации данных

Контрольные вопросы

- 1) Дайте определение автоматизированной оценке уязвимостей.
- 2) Каковы основные недостатки автоматизированной оценки уязвимостей? Каковы её достоинства?
- 3) Когда наиболее эффективно использовать автоматизированную оценку уязвимостей?
- 4) На чём основываются правила в оценке уязвимостей SQL Server?
- 5) Что такое базовый показатель (baseline) в оценке уязвимостей SQL Server?
- 6) Что такое и для чего нужна классификация данных?

Задание

Выполните «Ход работы». Также проведите оценку уязвимостей и классификацию данных в базе данных и заданных рамках согласно варианту. Учитывайте, что не все меры и не вся классификация, предоставленные автоматизированными средствами, могут соответствовать описанной в варианте ситуации. Составьте отчёт по проделанной работе, оформите пункт отчёта с ходом работы в таком виде, в каком бы вы предоставили его своему вышестоящему начальству о проделанной начальной работе по обеспечению безопасности в базе данных. Необходимо указать, какие меры были приняты, в чём они заключались, от чего (от каких уязвимостей) они защитили и обозначить критичные данные в базе данных с их классификацией, а также сделать краткий вывод. Также следует проанализировать выполненные правила и составить список проблем, которые уже были учтены. В отчёте используйте автоматически генерируемые отчёты инструментов оценки уязвимостей и классификации данных. Не выходите за рамки средств оценки уязвимостей и классификации данных.

Дополнительное задание

Прделайте то же задание, только с системной базой данных master и без классификации данных.

ЛАБОРАТОРНАЯ РАБОТА №6

Защита от SQL-инъекций

Целью работы является изучение принципов реализации атак вида «SQL-инъекция» и методов защиты от атак такого вида.

Теоретический материал

SQL-инъекция – атака, которая заключается во внедрении вредоносного кода через пользовательский интерфейс, который в процессе обработки внутри приложения будет преобразован в SQL запрос. Уязвимости вида "SQL-инъекция" не относятся к уязвимостям самих СУБД и происходят на уровне приложения. Поскольку СУБД обрабатывает любой синтаксически верный запрос, защита на уровне СУБД от внедрения SQL кода невозможна (возможна с проверкой на «подозрительный» код, однако наверняка отличить обычный код от внедрённого в рамках СУБД нереально). Однако, поскольку в результате SQL-инъекций, происходит непосредственная утечка информации из СУБД, необходимо рассмотреть её в рамках этого курса лабораторных работ.

В ходе реализации SQL-инъекции злоумышленник вводит в окно пользовательского запроса (это может быть окно приложения, окно ввода данных в веб-сайте, http-запрос к веб-сайту, и т.п.) замаскированный от приложения SQL код, который впоследствии интерпретируется приложением не как переменная в запросе, как предполагалось его разработчиками, а как код, потому что происходит конкатенация этих пользовательских данных с заранее определённой строкой кода SQL запроса. Таким образом злоумышленник может внедрять самые разнообразные запросы в код, получая необходимую ему информацию.

Для лучшего понимания принципа работы SQL-инъекций приведём пример. Допустим, проверка пароля пользователя происходит посредством поиска в таблице Password совпадения введённых пользователем имени и пароля с содержимым столбцов username и password соответственно; если соответствующая запись в таблице есть, то человек проходит аутентификацию (примечание: на практике подобная проверка должна осуществляться принципиально другими, более безопасными, методами, для чего используются односторонняя функция хеширования и соль). Для этого составляется запрос вида:

```
SELECT * FROM Password WHERE username = '[user]' and password = '[pass]';
```

Приложение вместо [user] подставляет введённое пользователем имя, а вместо [pass] – введённый пользователем пароль.

Злоумышленник, каким-либо образом, например, методом проб и ошибок, определил структуру этого запроса и решил вместо указания пароля ввести, например:

```
1' or username = 'admin
```

В результате чего получится следующий запрос:

```
SELECT * FROM Password WHERE username = " and password = '1' or username = 'admin';
```

В результате него будет возвращена запись с именем пользователя admin. Если в системе такой пользователь имеется и является администратором, злоумышленник получит доступ к функционалу администратора в приложении. Если же злоумышленник

хочет получить полный список пользователей, ему достаточно указать всегда истинное выражение, например:

1' or 1=1; -- ';

Сочетание символов -- (два дефиса) означает комментарий, соответственно, символы ' ; , которые добавляются приложением в конце запроса и завершают его, будут проигнорированы, вместо них останется только ; , указанный злоумышленником для составления синтаксически верного запроса.

Существует множество различных видов SQL-инъекций и множество различных техник их применения. Применяемые техники не стоят на месте и развиваются вместе с развитием средств защиты. Перечислим основные и самые распространенные виды SQL-инъекций:

- внедрение в строковые параметры операторов SQL – основной и самый распространенный вид инъекции, простой пример которой описан выше: злоумышленник внедряет в код свои операторы и получает полноценный ответ на свой запрос;

- Error-based SQL-injections (инъекции, основанные на выводе информации об ошибках СУБД) - в данном случае атакующий заменяет или добавляет в уязвимый параметр синтаксически неправильное выражение, после чего веб-сервер производит парсинг (отображение) HTTP-ответа в поиске ошибок СУБД, в которых содержалась бы заранее известная инъекция (последовательность символов), которая будет сконкатенирована с интересующей атакующего информацией. Данный метод может применяться только в тех случаях, когда веб-приложение по каким-то причинам (чаще всего в целях отладки) раскрывает ошибки СУБД;

- Stacked queries SQL injection (инъекция, основанная на последовательном выполнении SQL-запросов) – при использовании данной техники атакующий добавляет в уязвимый параметр HTTP-запроса точку с запятой, после чего следом размещает внедряемый SQL-запрос. Этот прием используется для внедрения SQL-команд, отличных от SELECT, например, для манипуляции данными (записи, обновления, удаления). Данная техника потенциально может дать возможность злоумышленнику возможность чтения или записи из файловой системы, а также выполнения команд в ОС;

- Boolean-based blind SQL injection (слепая SQL-инъекция) – вид инъекции, в которой злоумышленник получает в качестве ответа от приложения не результат запроса, а бинарный ответ (да или нет). Соответственно этому внедряемый им код определяется так, чтобы ответ был получен не на тот вопрос, который подразумевается разработчиками приложения, а на вопрос, который ставит злоумышленник. Этот вид инъекции разделяется в зависимости от того, возвращается ли приложением сообщение об ошибке или нет (из сообщения об ошибке можно получить множество ценной для злоумышленника информации);

- Time-based blind SQL injection (инъекция, основанная на времени ответа СУБД) – злоумышленник получает в качестве ответа только время выполнения запроса; результат запроса же всегда одинаков (например, что запрос был обработан).

Основные техники защиты от инъекций включают в себя три вида правил:

- 1) обнаружение инъекции в рамках приложения при построении кода – набор техник, используемых при программировании приложения, которые будут подробнее рассмотрены далее;

- 2) минимизация прав доступа пользователя – даже если злоумышленнику удастся внедрить свой код, СУБД не позволит ему исполнить больше, чем пользователь, от имени которого он выступает. Зачастую приложение выступает от имени одного фактического пользователя СУБД, который может всё или почти всё, а разделение прав доступа происходит на уровне приложения. Такой подход развязывает злоумышленникам руки при использовании SQL-инъекций. Техники внедрения инъекций включают в себя

в том числе обход техник защиты от них, однако при всём желании одними только SQL-инъекциями преодолеть доступные пользователю права злоумышленник не сможет;

3) использование автоматизированных средств обнаружения инъекций (WAF – web application firewall) – средства, обнаруживающие "подозрительный" код и призывающие внимание администратора в случае обнаружения ими такового.

Основными программными методами защиты являются:

- фильтрация или экранирование спецсимволов, кавычек и escape-последовательностей – превращение в рамках пользовательских данных символов, интерпретируемых как служебные, в символы, интерпретируемые как текст; её можно осуществить, например, полным запретом на выполнение запроса при наличии таких символов, или экранированием спецсимволов (оно выполняется с помощью другого служебного спецсимвола – чаще всего это обратный слэш «\») – чтобы символ интерпретировался именно как текст);

- фильтрация по типу данных и типизация данных – данные, полученные от пользователя, ни в коем случае нельзя напрямую вставлять в SQL-запрос, так как он может допустить ошибку во время ввода данных, или оказаться злоумышленником, пытающимся реализовать атаку на веб-приложение. Следовательно, полученные данные необходимо в обязательном порядке приводить к тому типу, который ожидается базой данных. Также, в случае если заранее известно, данные какого формата должен вводить пользователь, следует ограничивать этот параметр до этих рамок – например, если известно, что пароль не может быть длиннее 10 символов и не должен содержать определённых символов, следует при превышении длины или наличии запрещённых символов в пароле отклонять запрос;

- ограничение возвращаемой информации – пользователь должен получать только абсолютный минимум необходимой информации в качестве ответа, и никогда не должен получать необработанного ответа от СУБД. Отсюда следует обязательный отказ от возвращения сообщения об ошибке в той форме, в которой его предоставляет СУБД, а также невозврат вообще каких бы то ни было данных при подозрении в том, что происходит попытка атаки;

- применение параметризованных (подготовленных) запросов (prepared statements) – многие СУБД (включая SQL Server) и языки программирования поддерживают использование заранее подготовленных процедур при программировании, которые, учитывая свойства СУБД, обрабатывают передаваемый пользователем текст и интерпретируют его исключительно как параметр, но не как код. Корректность данного метода напрямую зависит от разработчика кода параметризованных запросов. Для языка программирования C#, например, существуют коллекции SqlCommandBuilder, SqlDataAdapter и SqlParameter.

- учёт размера буфера данных – более актуально при использовании низкоуровневого программирования или процедур в СУБД. Необходимо проверять передаваемые пользователем данные на предмет того, чтобы они не превысили определённый при создании процедуры допустимый размер переменных; если это произойдёт, лишние данные будут усечены и добавлены в другую часть кода, которая не предназначена для этой переменной.

Главный принцип при программировании веб-приложений звучит следующим образом – никогда и ни на каком этапе обработки параметров нельзя доверять данным, передаваемым от пользователя.

В качестве заключения к теоретической части добавим, что защита от SQL-инъекций, как и всякий процесс защиты, должен быть комплексным и многоуровневым. Не существует ни одного универсального средства, которое бы разом закрыло проблему, при этом всегда необходимо придерживаться концепции многослойной обороны – ис-

пользовать несколько средств защиты, каждое из которых сможет если не предотвратить угрозу, то задержать злоумышленника.

Ход работы

В рамках данной лабораторной работы будут рассмотрены четыре наиболее распространенных вида SQL-инъекций: внедрение в строковые параметры, инъекция, основанная на выводе текста ошибки, инъекция, модифицирующая данные и слепая инъекция.

Применение инъекций будет рассмотрено на примере отдельной виртуальной машины с Microsoft SQL Server. На ней установлены несколько веб-приложений, написанных на языке программирования PHP, которые могут быть запущены с помощью локального веб-сервера IIS. Помимо этого, в данной виртуальной машине располагаются несколько баз данных с разными предметными областями.

В рамках данной лабораторной работы также будут рассмотрены методы защиты от вышеприведенных SQL-инъекций: типизация данных, экранирование значений и подготовленные выражения.

Для начала работы скачайте и запустите образ виртуальной машины по инструкции из введения. После успешного импорта конфигурации запустите виртуальную машину, и войдите под предложенной учетной записью (пароль от учетной записи содержится в скачанном архиве). После окончательной загрузки рабочего стола запустите браузер и перейдите по адресу localhost. После загрузки на экране появится визор (рисунок 6.1), в котором будут отражены все файлы, хранящиеся в каталоге данного сайта (каталог можно изменить в настройках IIS).

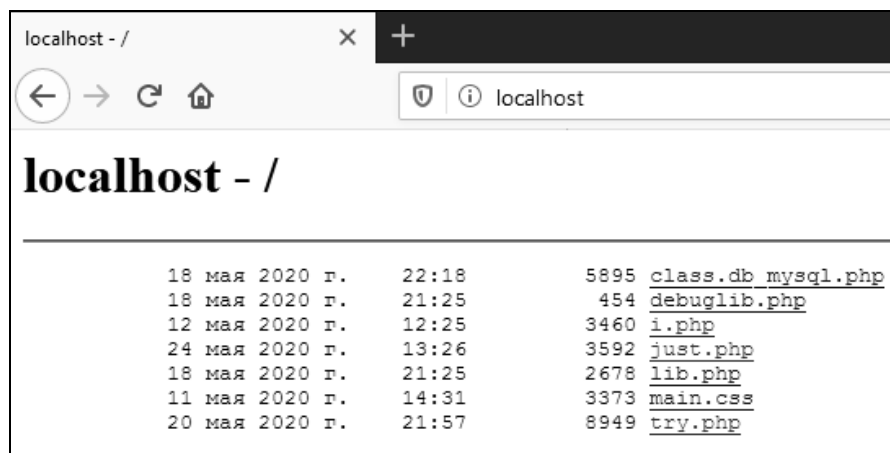


Рисунок 6.1 – Пример визора локального сервера

В визоре перейдите по ссылке с названием injections.php. Данное веб-приложение предназначено для демонстрации работы SQL-инъекций.

Внедрение в строковые параметры

Данный тип инъекций является самым простым, вследствие чего получил свое распространение среди злоумышленников. Атака основывается на внедрении в исполняемый SQL-запрос сторонних параметров или выражений, позволяющих злоумышленнику получить необходимую ему информацию. Как правило, запросы такого типа содержат в себе логические операторы OR или AND, либо же оператор объединения двух запросов UNION.

Рассмотрим пример. После запуска файла injections.php пользователь увидит главное окно веб-приложения (рисунок 6.2).

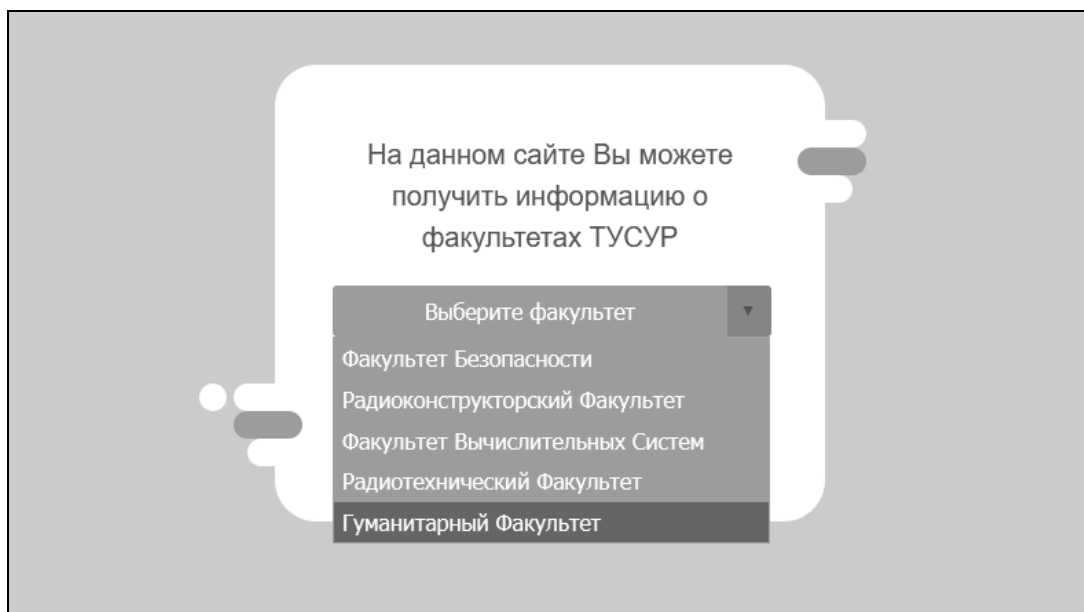


Рисунок 6.2 – Главное окно веб-приложения

На рисунке видно, что у пользователя нет возможности ввести какие-либо данные, так как выпадающее меню выбора реализовано с помощью тега `select`. Следовательно, SQL-инъекцию придется внедрять через URL-строку. Прежде всего необходимо разобраться, какие же параметры передаются СУБД и из каких элементов состоит SQL-запрос. Для этого достаточно выбрать любой из факультетов и нажать кнопку «Получить данные», после чего взглянуть на URL-строку (рисунок 6.3).

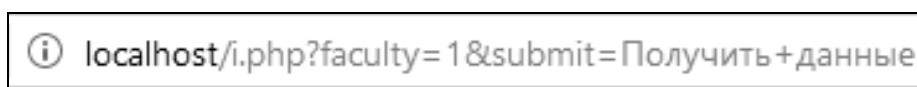


Рисунок 6.3 – URL-строка веб-приложения

Напоминаем, что GET (или же POST) параметрами называются те параметры, которые передаются после вопросительного знака. Разница же между GET и POST параметрами заключается в том, что параметры, переданные через метод GET, показываются в URL-строке, в отличие от параметров, переданных через метод POST. В нашем случае GET-параметрами будут поле `faculty`, которое приняло значение 1 и поле `submit` (кнопка подтверждения), принявшая строковое значение «Получить данные». Следовательно, из данных параметров составляется следующий запрос `SELECT * FROM Faculty Where Faculty ID = 1` (параметр `submit` в запросе не используется и, по сути, является триггером для запуска PHP скрипта). Так как в коде SQL-инъекции мы будем добавлять GET-параметры, то вставлять код инъекции необходимо между полем `faculty` и полем `submit`.

Перед началом атаки не помешало бы проверить, а защищен ли данный сайт от самых простых инъекций, и выводятся ли тут ошибки СУБД. Сделать это можно, вставив простейшую SQL-инъекцию, содержащую оператор AND и сравнение, которое изначально является неверным, а именно `AND 1=""`. В данном случае мы намеренно пишем неправильный запрос, для того, чтобы СУБД сгенерировала исключение и выдала ошибку. В итоге содержимое URL-строки будет выглядеть следующим образом (рисунок 6.4).



Рисунок 6.4 – Содержимое URL-строки после внедрения инъекции

Теперь посмотрим, что выдало веб-приложение после внедрения инъекции (рисунок 6.5).

```
Array ( [0] => Array ( [0] => 42000 [SQLSTATE] => 42000 [1] => 1038 [2] => [Microsoft][ODBC Driver 17 for SQL Server][SQL Server]Имя объекта или столбца отсутствует или пусто. Убедитесь, что каждый из столбцов в инструкции SELECT INTO имеет имя. Для других инструкций проверьте наличие пустых имен-псевдонимов. Не допускаются псевдонимы, определенные как [] или "". Измените псевдоним на допустимое имя. [message] => [Microsoft][ODBC Driver 17 for SQL Server][SQL Server]Имя объекта или столбца отсутствует или пусто. Убедитесь, что каждый из столбцов в инструкции SELECT INTO имеет имя. Для других инструкций проверьте наличие пустых имен-псевдонимов. Не допускаются псевдонимы, определенные как [] или "". Измените псевдоним на допустимое имя. ))
```

Рисунок 6.5 – Результат внедрения инъекции

На рисунке видно, что СУБД выдала текст ошибки. Это означает, что разработчики данного веб-приложения никаким образом не обработали вывод ошибок. Используя данную информацию, злоумышленник также может нанести ущерб. Теперь, зная, что веб-приложение не обладает должным уровнем защиты, мы можем внедрять более серьезные инъекции, например, содержащие оператор UNION, но для начала попытаемся вывести всю информацию из открытой таблицы. Для этого можно использовать логический оператор OR и всегда истинное условие 1=1. Так как одно из условий всегда будет истинно, то СУБД выведет всю информацию из таблицы. Код инъекции и результат ее внедрения приведены на рисунке 6.6 и рисунке 6.7.

localhost/i.php?faculty=2 OR 1=1&submit=Получить+данные

Рисунок 6.6 – Код внедряемой SQL-инъекции



Номер факультета	Название факультета	Декан факультета	Номер телефона
1	Факультет Безопасности	Кручинин Д.В.	88005553535
2	Радиоконструкторский Факультет	Озеркин Д.В.	89524561231
3	Факультет Вычислительных Систем	Черкашин М.В.	89601796574
4	Радиотехнический Факультет	Попова К.Ю.	89511795275
5	Гуманитарный Факультет	Иванов И.И.	555

Рисунок 6.7 – Результат внедрения SQL-инъекции

Теперь попытаемся внедрить SQL-инъекцию с оператором UNION. Данный оператор служит для объединения двух SELECT (как правило из разных таблиц) и требует того, чтобы типы данных в выбираемых полях таблиц и количество этих полей совпадали. Имена остальных таблиц и их полей вам необходимо узнать самостоятельно (сделать это можно с помощью системных таблиц INFORMATION_SCHEMA.TABLES и INFORMATION_SCHEMA.COLUMNS используя UNION).

В итоге SQL-инъекция с UNION будет выглядеть следующим образом:

SELECT * FROM Faculty WHERE [Faculty ID] =1 UNION SELECT 0, [First Name], Address, [Passport Number] FROM Student.

Теперь разберем данную инъекцию по частям.

SELECT * FROM Faculty WHERE [Faculty ID] =1 – стандартный для данного веб-приложения SQL-запрос.

UNION SELECT 0, [First Name], Address, [Passport Number] FROM Student – как уже было сказано выше, количество выбираемых полей таблиц и их типы должны совпадать. Если посмотреть на стандартный вывод веб-приложения, то мы увидим, что выводится 4 столбца, следовательно, во втором SELECT должно быть тоже 4 столбца. Но что делать, если мы, например, знаем имена только 1 или 2 столбцов? Тогда можно искусственно «создать» столбец, заменив его с помощью любой цифры (если тип данных числовой), либо же строки (если тип данных строковый). Так как первым полем таблицы как правило является идентификатор, то в данном случае он был заменен на 0 (тип integer) и ошибки не произошло. Остальные же поля в обеих таблицах являются строковыми. Результат выполнения данной инъекции приведен на рисунке 6.8.

Номер факультета	Название факультета	Декан факультета	Номер телефона
2	Радиоконструкторский Факультет	Озеркин Д.В.	89524561231
0	Адам	Учебная 11	324564
0	Александра	Лыткина 18	124656
0	Антон	Ленина 23	125426
0	Валерия	Мокрушина 20	346546
0	Ильвир	Нахимова 19	798658

Рисунок 6.8 – Результат внедрения SQL-инъекции с UNION

Однако следует упомянуть, что при всей простоте исполнения инъекции такого вида, сама по себе она срабатывает довольно-таки редко, так как для защиты от данного вида инъекций необходимо лишь приводить данные к ожидаемому типу.

Error based SQL-injections

Данный вид инъекций основывается на выводе текста ошибки СУБД. В предыдущем пункте уже был приведен пример, когда СУБД вывела в окно веб-приложения подробную информацию об ошибке. Однако, в тексте той ошибки не содержалась какая-либо конфиденциальная информация. Но некоторую конфиденциальную информацию, например, версию СУБД или наименование БД с легкостью можно извлечь, используя данный вид инъекций.

Для примера, попытаемся узнать версию СУБД. Версия СУБД хранится в системном параметре под названием @@version. Следовательно, нам необходимо с помощью обычного сравнения заставить СУБД сгенерировать ошибку. Понятно, что переменная @@version имеет строковый тип данных, следовательно, для того, чтобы СУБД сгенерировала исключение необходимо сравнить данную переменную с числом.

В итоге получается инъекция следующего вида:

```
SELECT * FROM Faculty WHERE [Faculty ID]=1 AND @@version=1.
```

В данном случае необходимо использовать логический оператор AND, так как его результат будет 1, если оба условия истинны. Результат внедрения инъекции приведен на рисунке 6.9.

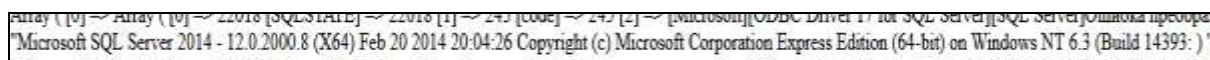


Рисунок 6.9 – Результат внедрения error-based инъекции

На рисунке видно, что текст ошибки содержит информацию о версии СУБД. Таким же образом, можно получить информацию о наименовании БД.

Стоит сказать, что данная инъекция сама по себе не является довольно распространенной и срабатывает только в том случае, если разработчик на этапе сдачи проекта не отключил подробный вывод ошибок, либо же не обработал их вовсе.

Stacked queries SQL-injection (инъекция, основанная на последовательном выполнении запросов)

Данный тип атаки является, пожалуй, одним из самых опасных, так как позволяет злоумышленнику выполнять операции, связанные с модификацией данных (изменять, удалять, добавлять записи в таблицы, либо же сами таблицы). Данная атака может быть реализована далеко не во всех СУБД, так как возможность последовательного выполнения SQL-запросов имеется лишь в некоторых системах, например, PostgreSQL или Microsoft Server SQL.

Особенность последовательного выполнения SQL-запросов заключается в том, что пользователь может написать несколько SQL-запросов, разделив их точкой с запятой, и в случае корректного синтаксиса и логического исполнения, они выполняются последовательно и независимо друг от друга.

Для примера попытаемся изменить какую-либо запись в существующей таблице, а именно добавим новый факультет. Для этого нам понадобится оператор INSERT, синтаксис которого имеет следующий вид

INSERT INTO <имя таблицы>[(<имя столбца>,...)] {VALUES (<значение столбца>,...)} | <выражение запроса>

Внедрять же данный оператор мы будем сразу после основного запроса SELECT. Для начала проверим, какие данные есть в таблице факультетов на данный момент с помощью OR 1=1 (рисунок 6.10).

Номер факультета	Название факультета	Декан факультета	Номер телефона
1	Факультет Безопасности	Кручинин Д.В.	88005553535
2	Радиоконструкторский Факультет	Озеркин Д.В.	89524561231
3	Факультет Вычислительных Систем	Черкашин М.В.	89601796574
4	Радиотехнический Факультет	Попова К.Ю.	89511795275
5	Гуманитарный Факультет	Иванов И.И.	555 Чтобы

Рисунок 6.10 – Исходные данные таблицы факультетов

Как видно на рисунке 6.10, на данный момент в таблице факультетов всего 5 записей. Попробуем добавить новую. Имена столбцов таблицы и их количество вам необходимо узнать самостоятельно (сделать это можно с помощью системных таблиц INFORMATION_SCHEMA.TABLES и INFORMATION_SCHEMA.COLUMNS используя UNION). Итоговый код SQL-инъекции будет выглядеть следующим образом:

SELECT * FROM Faculty WHERE [Faculty ID] =1;INSERT Faculty VALUES (135346, 'Какой-то новый факультет', 'Какой-то новый декан', 'И новый телефон');

Разберем данную SQL-инъекцию по частям.

SELECT * FROM Faculty WHERE [Faculty ID] =1 - стандартный для данного веб-приложения SQL-запрос.

INSERT Faculty VALUES (135346, 'Какой-то новый факультет', 'Какой-то новый декан', 'И новый телефон') – в данном коде просто необходимо написать нужные ключе-

вые слова (INSERT и VALUES), после чего указать в скобках значения, которым они будут равняться. Стоит упомянуть, что количество вставляемых значений должно быть равно количеству полей таблицы, в противном случае СУБД выдаст ошибку. Результат внедрения данной SQL-инъекции приведен на рисунке 6.11.

Номер факультета	Название факультета	Декан факультета	Номер телефона
1	Факультет Безопасности	Кручинин Д.В.	88005553535
2	Радиоконструкторский Факультет	Озеркин Д.В.	89524561231
3	Факультет Вычислительных Систем	Черкашин М.В.	89601796574
4	Радиотехнический Факультет	Попова К.Ю.	89511795275
5	Гуманитарный Факультет	Иванов И.И.	555
135346	Какой-то новый факультет	Какой-то новый декан	И новый телефон

Рисунок 6.11 – Результат внедрения stacked SQL-инъекции

На рисунке 6.11 видно, что записей в таблице факультетов уже 6, а это значит, что SQL-инъекция сработала.

Однако, как уже было сказано выше, данный вид инъекции позволяет не только добавлять, но и удалять и модифицировать данные. Проверим это. Например, изменим название добавленного факультета. Для этого нам понадобится оператор UPDATE, синтаксис которого имеет следующий вид:

```
UPDATE Название_таблицы
SET столбец1 = значение1, столбец2 = значение2, ...
WHERE условие;
```

Итоговый код SQL-инъекции будет выглядеть следующим образом:

```
SELECT * FROM Faculty WHERE [Faculty ID] =1; UPDATE Faculty SET [Faculty Title] = 'Это уже старый факультет' WHERE [Faculty ID] = 135346;
```

Результат выполнения данной SQL-инъекции приведен на рисунке 6.12.

Номер факультета	Название факультета	Декан факультета	Номер телефона
1	Факультет Безопасности	Кручинин Д.В.	88005553535
2	Радиоконструкторский Факультет	Озеркин Д.В.	89524561231
3	Факультет Вычислительных Систем	Черкашин М.В.	89601796574
4	Радиотехнический Факультет	Попова К.Ю.	89511795275
5	Гуманитарный Факультет	Иванов И.И.	555
135346	Это уже старый факультет	Какой-то новый декан	И новый телефон

Рисунок 6.12 – Результат внедрения stacked SQL-инъекции

Таким же образом можно использовать оператор DELETE для удаления данных из таблицы или же удаления самих таблиц. Попробуйте самостоятельно удалить запись из какой-либо таблицы.

Однако, стоит сказать, что исполнение данного вида инъекций требует наличия у злоумышленника прав на создание, удаление, редактирование таблиц. Следовательно, такая инъекция может быть внедрена только в случае ошибки администратора при назначении прав учетным записям пользователей.

Boolean-based blind SQL-injections

Данный вид SQL-инъекций по структуре и стилю исполнения схож с последовательно выполняемыми инъекциями, однако, имеет свою отличительную особенность. Она заключается в том, что после успешного внедрения инъекции злоумышленник не получает визуального результата отработки инъекции. В данном случае инъекция либо работает, либо нет, и именно поэтому она получила название Boolean-based.

Особенностью данного вида инъекций является возможность выполнения системных процедур, что открывает перед злоумышленником большие возможности. Так как хранимые процедуры можно отправить на исполнение в любой момент времени (достаточно лишь знать их название), то злоумышленник может воспользоваться любой процедурой из списка системных.

Для примера попробуем выполнить команду ping, которая является системной командой для ОС Windows и хранимой процедурой для Microsoft SQL Server. Синтаксис вызова системных процедур выглядит следующим образом:

```
[[ EXEC [ UTE] имя_процедуры [;номер]
[[@имя_параметра=]{значение | @имя_переменной}
[OUTPUT ]][DEFAULT ]][...n]
```

Список системных процедур хранится в таблице INFORMATION.SCHEMA.ROUTINE.

Синтаксис вызова процедуры ping выглядит следующим образом:

```
Exec master..xp_cmdshell "ping IP_адрес"
```

В итоге SQL-инъекция будет выглядеть следующим образом:

```
SELECT * FROM Faculty WHERE [Faculty ID] =1;exec master..xp_cmdshell "ping IP_адрес"
```

Злоумышленник, например, может регулярно пытаться «пинговать» недоступный IP. А так как у данной команды есть тайм-аут, который в обязательном порядке должен пройти, прежде чем пользователь получит сообщение о том, что данный адрес недоступен, то СУБД будет раз за разом обращаться по недоступному адресу, что вызовет отказ системы. Результат работы продемонстрирован на рисунке 6.13.

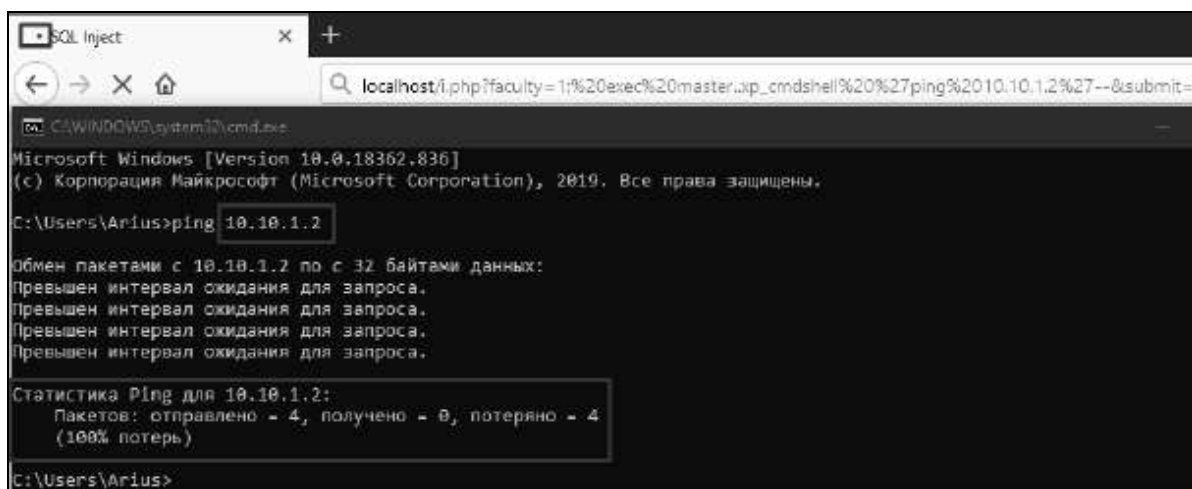


Рисунок 6.13 – Результат внедрения SQL-инъекции

Недостатки у данного вида инъекций абсолютно такие же, как и у последовательно выполняемых инъекций, так как по синтаксису они идентичны.

Методы защиты от атак типа SQL-инъекции

Как уже было сказано, аксиомой веб-разработки является утверждение, что ни в коем случае нельзя доверять как пользователю, так и данным, которые пришли от него. Вследствие этого, программисты, как правило, используют несколько методов защиты, которые внедрены повсеместно и уже зарекомендовали себя.

В ходе данной работы будут рассмотрены следующие методы защиты от атак типа SQL-инъекции:

- 1) типизация данных;
- 2) экранирование строковых значений;
- 3) подготовленные запросы (prepared statements).

Для демонстрации работы данных методов защиты необходимо из визора локального сервера запустить файл с названием `injections_secured.php`. Данное приложение имеет абсолютно такой же интерфейс, как и предыдущее, однако, в коде добавлены методы для защиты от инъекций. Данное приложение написано с использованием объектно-ориентированного подхода, что значительно упрощает для пользователя процедуры обращения к базе данных, фильтрации данных и их получения.

Типизация пользовательских данных

Данный метод защиты является одним из основных и повсеместно применяется в веб-разработке. Суть данного метода состоит в принудительном приведении данных, полученных от пользователя.

Рассмотрим такую ситуацию. При разработке приложения программист предполагает, что в данную форму пользователь будет вводить, например, только числа. Однако, в какой-то момент, пользователь (неважно, намеренно или же по ошибке) вводит в данное поле строку, состоящую из чисел, букв и каких-нибудь знаков. Если программист не предусмотрел такую ситуацию, то в лучшем случае дело обойдется ошибкой, которую сгенерирует СУБД. Пользователь прочтет текст данной ошибки и просто решит, что приложение работает некорректно.

При худшем варианте развития событий злоумышленник выполнит SQL-инъекцию, получив или же модифицировав данные, к которым у него нет санкционированного доступа.

Однако, если же программист будет принудительно приводить полученные данные к тому типу, который ожидается (например, к числовому) и корректно обрабатывать ситуации, когда привести данные к нужному типу не получилось (например, интерпретатор PHP никак не сможет привести строку, состоящую только из букв к числовому типу), то проблем подобного типа можно будет избежать.

В PHP преобразование типов происходит неявно, это значит, что программисту необязательно следить за типом возвращаемого значения. Однако данный механизм может значительно усложнить поиск логической ошибки в коде программы. Поэтому данные функции необходимо использовать с осторожностью.

Для преобразования одного типа в другой программисту всего лишь необходимо перед именем переменной в круглых скобках указать тип, к которому необходимо привести данную переменную. Предположим, что у нас есть набор переменных разного типа (рисунок 6.14).

```
$integer = 10;  
$float = 20.5;  
$string = '10aaa';
```

Рисунок 6.14 – Набор переменных разного типа

Напомним, что переменные в языке PHP обозначаются символом \$. В данном случае название переменных соответствует их типу (но так делать необязательно). Теперь же попытаемся перевести, например, переменную string в целочисленный тип (изначально, это - строка). Выполнить данное преобразование можно с помощью следующего кода:

```
$string = (int)$string;
```

Проверить тип какой-либо переменной в PHP можно с помощью функции `var_dump`.

```
echo var_dump($string);
```

После выполнения данной команды интерпретатор PHP выведет следующее:

```
int(10), где int – тип переменной, 10 – значение переменной.
```

Получается, что интерпретатор просто обрезал символы, которые не смог привести в нужный тип. Если первым символом была буква, то интерпретатор, встретив символ, который он не может преобразовать к нужному типу, даже не будет дальше пытаться выполнить данную операцию. Он просто вернет `int(0)`, то есть переменная будет нужного типа, однако данные будут потеряны.

С этой стороны самым универсальным типом является string, так как к нему можно привести абсолютно любую последовательность символов.

Именно на этом принципе основан метод класса `TCore_db` под названием `clear_param` (рис. 6.15).

```
function clear_param($p, $type = 'str'){
    # code...
    switch ($type) {
        case 'int':    return (int)$p;
        case 'float':  return (float)$p;
        case 'bool':   return (bool)$p;
        case 'str':    return (string)$p;
        case 'string': return (string)$p;
        case 'html':   return preg_replace('/\<script.*?\</script\>/', '', $p);
        case 'clear':  return strip_tags(htmlspecialchars($p));
    }
}
```

Рисунок 6.15 – Код метода `clear-param`

Данный метод вызывается другим методом под названием `get_param`. Последний в свою очередь получает данные, которые передал пользователь. А получает он их из GET или POST (в зависимости от используемого метода) суперглобальных массивов. Попадая в `clear_param`, данные принудительно приводятся к нужному типу.

Добавочные функции `preg_replace` и `strip_tags` нужны для защиты от атак типа XSS, при которых пользователь применяет так называемые «небезопасные HTML-теги».

Откройте с помощью текстового редактора файл `injections_secured.php` (он находится в каталоге сайта). Перейдите к месту, где вызывается метод `get_param` (рисунок 6.16).

```

301         if ($value = $DB->get_param('faculty', false, 'int'))
302         {
303             if (!$value || !is_numeric($value))
304             {
305                 echo 'Записи с таким идентификатором не найдено!';
306             }
307             else
308             {
309                 $DB->get_records('Faculty', ['Faculty ID'=>$value]);
310             }
311         }

```

Рисунок 6.16 – Вызов метода get_param

Если вы просмотрите код, то увидите, что разработчик данного веб-приложения получает пользовательский параметр в переменную \$value. Однако, перед тем как данная переменная будет вставлена в SQL-запрос, который формируется при вызове метода get_records, она подвергается операции приведения к нужному типу, как раз с помощью метода get_param, который в свою очередь вызывает clear_param. Тип, к которому необходимо привести данные, указывается в качестве одного из аргументов метода. Еще раз напомним, что данное действие необходимо производить со всеми пользовательскими данными.

Проверим работоспособность данного метода. Для этого нужно вставить любую UNION – based инъекцию и посмотреть, что выведет приложение. Воспользуемся вышеупомянутой инъекцией, код которой приведен ниже:

UNION SELECT 0, [First Name], Address, [Passport Number] FROM Student

Для удобства и наглядности, SQL запрос, который был отправлен в СУБД, также будет выведен на рисунке.

Результат внедрения данной инъекции приведен на рисунке 6.17.

На данном сайте Вы можете получить информацию о факультетах ТУСУР

Выберите факультет ▼

Получить данные

SELECT * FROM Faculty WHERE [Faculty ID] = 1--

Id	Название факультета	Декан факультета	Номер телефона
1	Факультет Безопасности	Кручинин Д.В.	88005553535

Рисунок 6.17 – Результат внедрения UNION-based инъекции

На рисунке 6.17 видно, что хотя пользователь попытался внедрить в запрос SQL-инъекцию, на экран была выведена только та информация, к которой у пользователя

действительно есть доступ. Такой результат был достигнут простым усечением части SQL-запроса, которая содержит инъекцию. То есть, изначальный запрос

```
SELECT * FROM Faculty WHERE [Faculty ID] =1 UNION SELECT 0, [First Name], Address, [Passport Number] FROM Student
```

после вызова метода `get_param` и `clear_param` превратился в

```
SELECT * FROM Faculty WHERE [Faculty ID] =1
```

Также стоит упомянуть, что двойной дефис в конце запроса (означает комментарий и не обрабатывается СУБД) также является неким дополнительным средством защиты, которое позволяет игнорировать код, идущий после данного оператора.

Экранирование пользовательских параметров

Данный прием также широко распространен в среде веб-программистов и довольно часто используется в веб-приложениях, которые содержат в SQL-запросах строковые параметры.

Предыдущий метод защиты (типизация параметров) в данном случае не подходит по одной простой причине – мы никак не можем усечь пользовательский запрос с помощью приведения к какому-либо типу, так как это вызовет некорректное поведение программы, и пользователь не сможет получить запрашиваемую информацию. Соответственно, для того чтобы пресечь SQL-инъекции, нам необходимо контролировать, в каком месте заканчивается настоящий SQL-запрос пользователя и начинается код самой инъекции.

Для того чтобы использовать какие-либо методы защиты, необходимо понимать, каким образом злоумышленник будет проводить атаку, и что он будет для этого использовать. На самом деле отличий при работе со строковыми параметрами от числовых совсем немного. Первые просто требуют кавычек при использовании WHERE в СУБД. Однако именно на использовании кавычек злоумышленник и может построить свою атаку.

Предположим, что теперь у нас есть приложение, где необходимо выбрать декана факультета, и если в базе данных найдется запись с таким именем, то приложение выведет информацию о данном факультете. То есть в этом приложении мы немного меняем структуру нашего GET-массива, помещая в него не число, а строку (имя декана). На рисунке 6.18 приведен вид главного окна приложения.

The screenshot shows a web application interface. At the top, there is a button labeled "Получить данные". Below the button, there is a text input field containing the SQL query: "SELECT * FROM Faculty WHERE [Faculty Dean] = 'Кручинин Д.В.'--". Below the input field, there is a table with four columns: "Id", "Название факультета", "Декан факультета", and "Номер телефона". The table contains one row of data.

Id	Название факультета	Декан факультета	Номер телефона
1	Факультет Безопасности	Кручинин Д.В.	88005553535

Рисунок 6.18 – Главное окно приложения

Также ниже приведена URL-строка. В ней можно увидеть, какие параметры будут помещены в суперглобальный массив GET. Помимо этого, приведен код исполняемого SQL-запроса (рисунок 6.19).

localhost/try.php?faculty=Кручинин+Д.В.&submit=Получить+данные

Рисунок 6.19 – URL-строка приложения

На рисунке 6.18 видно, что сравниваемое значение (Кручинин Д.В.) помещено в кавычки, так как является строкой. Зная это, злоумышленник может воспользоваться следующей SQL-инъекцией

SELECT * from Faculty WHERE [Faculty Dean] LIKE 'Кручинин Д.В. ' OR 1=1' –

Результат внедрения такой инъекции приведен на рисунке 6.20.

Номер факультета	Название факультета	Декан факультета	Номер телефона
1	Факультет Безопасности	Кручинин Д.В.	88005553535
2	Радиоконструкторский Факультет	Озеркин Д.В.	89524561231
3	Факультет Вычислительных Систем	Черкашин М.В.	89601796574
4	Радиотехнический Факультет	Попова К.Ю.	89511795275
5	Гуманитарный Факультет	Иванов И.И.	555

Рисунок 6.20 – Результат внедрения SQL-инъекции

Т. е., добавляя кавычку и оператор комментирования, злоумышленник получает возможность внедрить те же инъекции, что и в том случае, когда приложение работает с числовыми параметрами.

Помимо этого, необходимо учитывать случаи, когда строка может содержать в себе символы, которые могут повлиять на ход выполнения программы (например, \, ', " и т.д.). Такие символы необходимо экранировать, то есть ставить перед ними обратный слеш (\).

Также одним из методов защиты является добавление программистом в конец SQL-запроса оператора комментирования (--). Данный оператор показывает СУБД, что вся строка, находящаяся после данного оператора, является комментарием и не подлежит обработке.

Для этих целей в классе TCore_db был написан метод под названием mssql_escape, который позволяет экранировать некоторые символы (например, \, ', " и т.д.). Данный метод является аналогом метода mysql_real_escape_string, который широко используется при работе с СУБД MySQL. К сожалению, данный метод недоступен для использования при работе с MSSQL, однако mssql_escape имеет практически схожий функционал. Метод mssql_escape имеет следующий код (рисунок 6.21)

```
protected function mssql_escape($data) {
    if (!isset($data) or empty($data)) return '';
    if (is_numeric($data)) return $data;

    $non_displayables = array(
        '/%0[0-8bcef]/',
        '/%1[0-9a-f]/',
        '/[\x00-\x08]/',
        '/\x0b/',
        '/\x0c/',
        '/[\x0e-\x1f]/'
    );
    foreach ( $non_displayables as $regex )
        $data = preg_replace($regex, '', $data);
    $data = str_replace("'", "''", $data);
    return $data;
}
```

Рисунок 6.21 – Код метода mssql_escape

В данном методе символы, которые необходимо экранировать представлены в виде массива мнемонических кодов. Данные коды необходимы для того, чтобы браузер корректно воспринимал некоторые символы (например, мнемоническим кодом для символа амперсанда будет & и т.д.). При вызове данного метода строка с SQL-запросом исследуется на наличие символов из массива кодов и при нахождении последнего, перед ним ставится дополнительная кавычка и обратный слеш.

Продemonстрируем работоспособность данного метода. Для этого внедрим в SQL-запрос простую инъекцию с логическим оператором OR 1=1. В итоге, исполняемый SQL-запрос будет выглядеть следующим образом:

```
SELECT * FROM Faculty WHERE [Faculty Dean] = 'Кручинин Д.В. 'OR 1=1 '—
```

В данной инъекции злоумышленник ставит кавычку, после чего вставляет SQL-инъекцию, тем самым давая СУБД понять, что данный блок также необходимо обработать.

Однако после того, как данный SQL-запрос будет обработан методом mssql_escape, вторая кавычка будет экранирована, что не позволит злоумышленнику внедрить инъекцию. В итоге код SQL-запроса будет выглядеть следующим образом:

```
SELECT * FROM Faculty WHERE [Faculty Dean] = 'Кручинин Д.В. \' OR 1=1 '—
```

Результат выполнения данного SQL-запроса приведен на рисунке 6.22

```
SELECT * FROM Faculty WHERE [Faculty Dean] = 'Кручинин Д.В. \' OR 1=1 '—
Записи с таким идентификатором не найдено!
```

Рисунок 6.22 – Результат выполнения SQL-запроса

На рисунке 6.22 видно, что веб-приложение выдало ошибку, указывающую на то, что записи, соответствующей данной строке (Кручинин Д.В.) не найдено. Приложение выдало такое сообщение по причине того, что после экранирования кавычки, с которой начинается код SQL-инъекции, СУБД восприняла всю строку (от первой одинарной ка-

вычки до последней) 'Кручинин Д.В. \' OR 1=1 ' как один большой параметр, который был передан пользователем, и тем самым не дала инъекции выполниться. Так как факультета с таким деканом в таблице не найдено, то приложение показывает пользователю, что такой записи в таблице нет.

Подготовленные запросы (prepared statements)

В современных реалиях практически все СУБД поддерживают концепцию подготовленных запросов. Данная технология интересна тем, что позволяет программисту не только обезопасить себя от SQL-инъекций, но и получить выигрыш в производительности.

Такой приём можно описать как некий вид скомпилированного шаблона SQL-запроса, который будет запускаться приложением и настраиваться с помощью входных параметров. Главными преимуществами использования данного метода защиты является то, что параметры подготовленного запроса не нуждаются в экранировании, так как драйвер производит данную операцию автоматически.

Помимо этого, неоспоримым преимуществом запросов данного типа является то, что исполняемый запрос необходимо подготовить всего один раз и затем его можно запускать столько раз, сколько нужно, причем как с теми же, так и с отличающимися параметрами. Когда запрос подготовлен, СУБД его анализирует, компилирует и оптимизирует план выполнения. В случае сложных запросов этот процесс может занимать ощутимое время и заметно замедлить работу приложения, если потребуется много раз выполнять запрос с разными параметрами. При использовании подготовленного запроса СУБД анализирует, компилирует и оптимизирует запрос любой сложности только один раз, а приложение запускает на выполнение уже подготовленный шаблон. Таким образом подготовленные запросы потребляют меньше ресурсов и работают быстрее.

Однако, вышеперечисленные преимущества данного метода защиты являются в некой степени вторичными, так как сам механизм защиты от SQL-инъекций скрыт внутри данного метода.

Как мы уже знаем, для того, чтобы получить информацию, пользователь выбирает некоторые параметры с помощью веб-интерфейса и при нажатии кнопки запрос отправляется на веб-сервер, который уже в свою очередь делегирует данный запрос к СУБД. В обычном случае СУБД составит запрос следующего типа

```
SELECT * FROM Faculty WHERE [Faculty Dean] =1
```

Однако, при использовании концепции подготовленных запросов, итоговый SQL-запрос будет выглядеть следующим образом:

```
SELECT * FROM Faculty WHERE [Faculty Dean] = ?
```

Как мы видим, в конце запроса появился вопросительный знак. Из теоретической информации, которая была приведена выше, можно сделать вывод, что данный SQL-запрос является шаблоном. Всё, кроме вопросительного знака, в данном случае будет являться так называемой статической частью запроса и не будет меняться от запроса к запросу. Всё, что помечено вопросительным знаком, является динамической частью запроса и меняется в зависимости от нужд пользователя. Пользователю лишь необходимо сформировать массив из параметров, которые будут подставляться вместо знака вопроса.

Безопасность данной подстановки обеспечивается тем, что она происходит на стороне СУБД, и злоумышленник, даже при всем желании, просто не имеет возможности вмешаться в данный процесс.

Для работы с данной технологией пользователю необходимо использовать функции языка PHP под названием `sqlsrv_prepare` и `sqlsrv_execute`.

В текстовом редакторе откройте файл под названием `injections_secured.php`. В данном файле найдите метод с именем `query`. В нем фигурируют две вышеприведенные функции.

Функция `sqlsrv_prepare` принимает на вход 3 аргумента – экземпляр соединения с базой данных, SQL-запрос в виде строки и массив аргументов. Данная функция занимается анализом запроса и составлением плана его выполнения (рисунок 6.23).

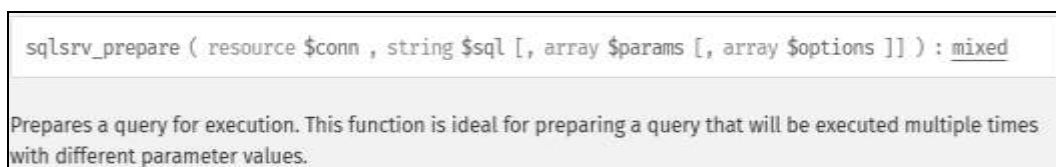


Рисунок 6.23 – Описание функции `sqlsrv_prepare`

Функция `sqlsrv_execute` принимает на вход ресурс, подготовленный функцией `sqlsrv_prepare` (данные функции не будут работать без связи друг с другом), и занимается непосредственным выполнением подготовленного запроса (рисунок 6.24).

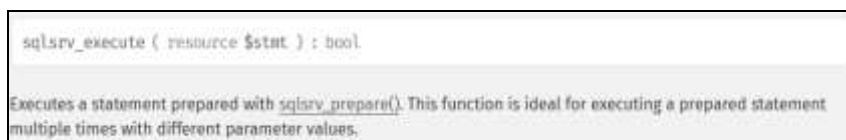


Рисунок 6.24 – Описание функции `sqlsrv_execute`

Проверим работоспособность данного метода защиты. Для начала выполним подготовленный запрос без SQL-инъекции. На рисунке 6.25 будет представлен результат выполнения и код SQL-запроса.

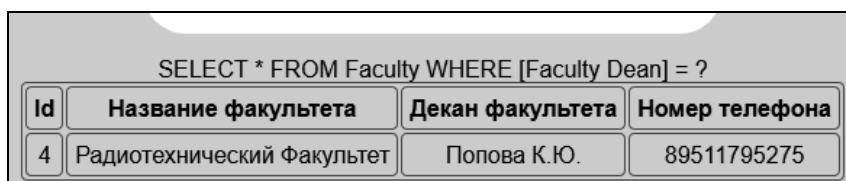


Рисунок 6.25 – Код SQL-запроса и результат его выполнения

Теперь же попытаемся внедрить в данный запрос SQL-инъекцию с оператором `UNION`. Код запроса и результат внедрения инъекции представлены на рисунке 6.26.

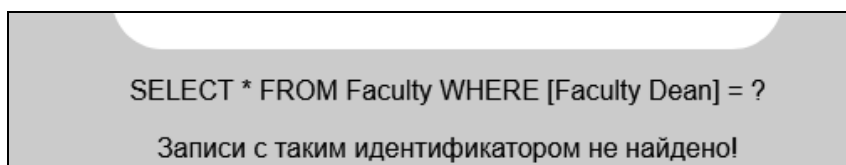


Рисунок 6.26 – Результат внедрения инъекции

В данном случае был выполнен запрос следующего вида:

`SELECT * FROM Faculty WHERE [Faculty Dean] = ? 'UNION SELECT 0, [First Name], Address, [Passport Number] FROM Student –`

Однако использование данного метода защиты привело к тому, что часть запроса, содержащая SQL-инъекцию, просто была проигнорирована. Следовательно, метод защиты работает.

Еще раз хотелось бы отметить то, что использование каждого из вышеприведенных методов по отдельности не является панацеей; их необходимо использовать в совокупности. Защита веб-приложения должна быть комплексной, и, в случае если злоумышленник преодолет один из защитных барьеров, то на его пути должны быть активизированы следующие методы защиты.

Контрольные вопросы

- 1) Дайте определение понятию SQL-инъекции.
- 2) Опишите отличительные признаки, достоинства и недостатки типов инъекций, приведенных в лабораторной работе.
- 3) Каким образом можно защититься от SQL-инъекции?
- 4) Перечислите несколько программных техник, с помощью которых можно ограничить некоторые виды инъекций.
- 5) Чем метод передачи данных GET отличается от POST? Какие данные можно передавать через GET и какие через POST?
- 6) Чего злоумышленник может добиться посредством SQL-инъекции? За какие границы он не может выйти посредством одних только инъекций?
- 7) Приведите конкретный пример SQL-инъекции на следующем запросе: `SELECT * FROM salaries WHERE user_name = '[userdata]'`; – в запросе строка `[userdata]` заменяется на пользовательские данные.
- 8) Для чего в SQL-запрос добавляется оператор комментирования?
- 9) Какие функции языка PHP могут использоваться для защиты от SQL-инъекций? Какие параметры они принимают и что делают?
- 10) *Всегда ли будет срабатывать последовательно выполняемая инъекция удаления той или иной таблицы? Почему?

Задание

Задание по реализации SQL-инъекций

Далее приводятся варианты первого задания:

1 вариант – предметная область «**Колл-центр**». В базе данных колл-центра хранятся группы данные сотрудников (как персональные, так и служебные), линии, которые обслуживаются данными сотрудниками, подробная информация о каждом поступившем звонке. Кроме того, сотрудники собраны в группы, и у каждой из групп есть свой супервайзер. В целом, данное предприятие консультирует людей по телефону по поводу того или иного вопроса. Распространение персональных данных сотрудников колл-центра может повлечь уголовную ответственность.

2 вариант – предметная область «**Аэропорт**». В базе данных аэропорта хранится информация о перевозчиках, предстоящих рейсах, и пассажирах, купивших или забронировавших места на предстоящие рейсы. Распространение персональных данных пассажиров может повлечь уголовную ответственность.

3 вариант – предметная область «**Склад офисной техники**». В базе данных склада хранится подробная информация о ноутбуках, персональных компьютерах и принтерах (в отдельных таблицах). В базе присутствует информация о модели, производителе и технических характеристиках устройств данного типа. Разглашение информации о наличии того или иного товара может повлечь финансовые потери.

4 вариант – предметная область «**Магазин игрушек**». В базе данных магазина, продающего игрушки хранятся данные о количестве и наличии того или иного товара,

проводит операцию сравнения, поэтому в случае UNION невозможно сравнивать текстовые (такие как ntext, т.е. не строковые) типы данных. Поэтому здесь будет полезен оператор UNION ALL, который не учитывает дубликаты.

Задание для всех вариантов

Используя механизм SQL-инъекций получите следующую информацию о базе данных:

- наименование БД;
- версию СУБД;
- наименование всех таблиц в БД;
- наименование каждого поля таблицы БД.

Задание для варианта 1

С помощью SQL-инъекций извлеките следующую информацию из базы данных:

- персональные данные сотрудников колл-центра;
- информацию о сотрудниках колл-центра, обслуживающих линии с самым высоким приоритетом;
- данные и список сотрудников, нанятых после 31 декабря 2010 года.

Задание для варианта 2

С помощью SQL-инъекций извлеките следующую информацию из базы данных:

- список доступных авиакомпаний;
- список рейсов, направляющихся в Москву и в Сингапур;
- список пассажиров, забронировавших места с индексами 1a или 1b на любых рейсах.

Задание для варианта 3

С помощью SQL-инъекций извлеките следующую информацию из базы данных:

- список персональных компьютеров фирмы В или А;
- список персональных компьютеров и ноутбуков, имеющих объем памяти больше 64 ГБ и дорожке 700 условных единиц;
- список всех цветных принтеров.

Задание для варианта 4

С помощью SQL-инъекций извлеките следующую информацию из базы данных:

- информацию о покупателях и поставщиках из штата Мичиган;
- список всех заказов;
- список продуктов от поставщика BRS01, не дороже 9 условных единиц.

В этом варианте задания два последних столбца при выводе меняются порядком, учтите это в запросе с использованием UNION ALL.

Задание для варианта 5

С помощью SQL-инъекций извлеките следующую информацию из базы данных:

- список всех морских сражений, произошедших в период с 1941 года по 1950 включительно;
- список всех американских кораблей, спущенных на воду в период с 1915 по 1945 годы;

— список кораблей всех американских классов, которые были повреждены, либо потоплены.

Дополнительное задание

Необходимо полностью пройти все задания, которые предложены в тренажере по SQL-инъекциям, доступном по следующей ссылке: <https://sql.training.hackerrdom.ru/>. Необходимо предоставить подробное решение каждого задания в рамках отчета по работе и защитить данный отчет.

Задание по защите от SQL-инъекций

После успешного выполнения первой части задания, Вы можете приступить ко второй части – защите веб-приложения с помощью методов, приведенных в данном руководстве. С помощью редактора Sublime Text откройте файл с Вашей предметной областью.

Во всех вариантах выполнения задания необходимо в коде приложения своей предметной области реализовать два метода: экранирование хвоста запроса (с помощью оператора `--`) в коде запроса независимо от типа передаваемого параметра и типизацию данных.

Вам предстоит работать с классом, который был написан для взаимодействия с базами данных. Файл с кодом данного класса расположен в корне сайта и называется `injections_secured.php`. Не стоит запускать данный файл, так как вы ничего не увидите. В данном файле лишь находится класс `TCore_db` и методы данного класса. Все методы класса вызываются из его объекта, статических методов в данном классе не предусмотрено.

Для демонстрации работы данных методов, вы можете открыть через локальный сервер файл под названием `injections_secured_application.php`. Данный файл содержит те же самые методы, что и `injections_secured.php`, однако в нем есть и HTML-код, что дает возможность визуально наблюдать действия того или иного метода.

Используемые методы класса `TCore_db` помечены соответствующим текстом. Реализовать метод защиты необходимо в процедурном стиле, а не в стиле объектно-ориентированного программирования, как это сделано в классе `TCore_db`. Это означает, что реализация данных методов предусматривается либо в виде функции, которая будет объявлена заранее и будет вызываться по необходимости в том или ином участке кода, либо же данный метод защиты будет напрямую внедрен в тот участок кода, где необходимо наличие данного метода защиты.

После того, как необходимый метод защиты будет внедрен в код, следует произвести проверку работоспособности данного метода и отразить результаты проверки в отчете.

Заключение

Обеспечение конфиденциальности информации несомненно является одной из важнейших задач разработчика баз данных. Поскольку не существует единственного метода, который способен устранить абсолютно все уязвимости, специалисту необходимо владеть обширным арсеналом техник, позволяющих реализовать защиту как от мошеннических атак, так и от случайных ошибок легитимных пользователей. Разнообразие таких техник вместе с их вдумчивым использованием способствует созданию многоуровневой защиты критически важной информации.

В результате выполнения лабораторных работ в соответствии с представленными указаниями, студенты приобретают навыки шифрования данных, реализации методики разделения прав пользователей, работы с предикатами безопасности для обеспечения защиты на уровне строк, создания и использования сертификатов асимметричного шифрования. Кроме того, обучающиеся изучают принципы функционирования инструментов Microsoft SQL Server, направленных на проведение автоматизированной проверки уязвимостей базы данных. Такие инструменты позволяют сократить временные затраты на анализ базы, что особенно полезно в условиях большого количества таблиц и их частого пополнения. В последней лабораторной работе студенты знакомятся с различными типами SQL-инъекций и вариантами противодействия такому типу атак.

Методические указания позволяют углубить навыки и умения проектирования баз данных, составления SQL-запросов, управления пользователями и информацией. Освоение актуальных навыков защиты информации в области систем хранения данных, достигаемое путем выполнения предложенных лабораторных работ, способствует повышению востребованности обучающегося на рынке труда.

Литература

1. Шустова, Л. И. Базы данных : учебник / Л.И. Шустова, О.В. Тараканов. — Москва : ИНФРА-М, 2021. — 304 с.
2. Култыгин, О. П. Администрирование баз данных. СУБД MS SQL Server : учеб. пособие / О. П. Култыгин. — Москва : МФПА, 2012. — 232 с.
3. Агальцов, В. П. Базы данных : учебник : в 2 кн. Книга 1. Локальные базы данных / В. П. Агальцов. — Москва : ФОРУМ : ИНФРА-М, 2021. — 352 с.
4. Полищук Ю. В. Базы данных и их безопасность: учебное пособие / Ю.В. Полищук, А.С. Боровский. — Москва: ИНФРА-М, 2023. — 210 с.
5. Мартишин С. А. Базы данных. Практическое применение СУБД SQL- и NoSQL-типа для проектирования информационных систем: учебное пособие / С.А. Мартишин, В.Л. Симонов, М.В. Храпченко. — Москва: ФОРУМ: ИНФРА-М, 2023. — 368 с.